Расписанные вопросы к государственному экзамену магистерской программы «Интеллектуальные системы»

Москва, 2025

Содержание

1 Понятие парадигмы (стиля) программирования. Особенности императивного, функционального и логического программирования как парадигм вычисления. . . . 4

2 Ключевые принципы объектно-ориентированного программирования: абстракция, инкапсуляция, полиморфизм. Виды полиморфизма. Концепция обобщенного программирования. . . . 12

3 Основные средства современных функциональных языков (на примере языка Хаскель). . . . 15

4 Логическое программирование: вычисление как доказательство. Базовые вычислительные механизмы языка Пролог. Декларативная и процедурная семантика логических программ. . . . 19

5 Последовательная и параллельная сложность алгоритмов, информационный граф и ресурс параллелизма алгоритмов. . . . 22

6 Логическая модель представления знаний. Применение логики предикатов первого порядка: этапы представления знаний, основные сложности. Дескриптивные логики: концепты и роли, соотношение с логикой предикатов. . . . 25

7 Сетевая модель представления знаний. Семантические сети и их свойства. Фреймы, их виды и структура, межфреймовые связи. Представление значений по умолчанию, понятие немонотонного вывода. Графы знаний: концепция, примеры. . . . 28

8 Понятие онтологии в инженерии знаний: состав онтологии, типы отношений концептов. Классификация онтологий, примеры онтологий разного вида. Принципы и основные шаги построения онтологии. . . . 33

9 Лингвистические онтологии и тезаурусы: особенности состава и построения. Семантические отношения в онтологиях и тезаурусах, примеры. . . . 36

10 Продукционная модель представления знаний: структура и цикл работы продукционной системы. Нечеткие знания и их обработка в продукционных системах. . . . 41

11 Методы поиска решений в пространстве состояний. Метод ветвей и границ (Branch and Bound), лучевой поиск (Beam Search), A*-алгоритм и его частные случаи. . . . 44

12 Методы слепого поиска в пространствах состояний. Алгоритм восхождения к вершине (Hill Climbing). Метод Монте-Карло, эволюционные алгоритмы. . . . 48

13 Методы поиска решений для игр с полной информацией: игровые деревья, минимаксный алгоритм, альфа-бета-отсечение. . . . 52

14 Байесовские сети как способ представления вероятностного пространства. Построение байесовских сетей, примеры их использования. . . . 54

15 Модели прогнозирования на основе деревьев решений. Алгоритмы CART и C4.5: критерии поиска разбиений, параметры ограничения роста и отсечения. . . . 57

16 Нейронные сети прямого распространения. Архитектуры MLP и RBF: структура сетей, виды функций активации, алгоритмы обучения, проблема переобучения и локальных минимумов. . . . 61

17 Обучение ассоциативным правилам. Алгоритмы Apriori и FP-tree: построение частых эпизодов с ограничением по поддержке и формирование правил с ограничением по достоверности. . . . 66

18 Машинное обучение: обучение с учителем. Задача классификации, способы оценки качества решений. Классификация текстов: применяемые при обучении признаки текстов, методы обучения классификаторов (на примере двух методов). . . . 70

19 Машинное обучение: обучение без учителя. Задача кластеризации, способы оценки качества кластеризации. Метод К-средних, иерархическая кластеризация. Кластеризация текстовой коллекции, применяемые признаки текстов. **73**

20 Уровни естественного языка и основные этапы/модули многоуровневого анализа текста. Подходы к решению задач обработки текста: инженерный (на правилах) и основанный на машинном обучении. Лингвистические ресурсы, их виды и назначение. **77**

21 Понятие языковой модели, виды моделей. Статистическая языковая модель: униграммные и N-граммные модели, применение в задачах обработки текстов. Сглаживание модели. Перплексия и способ её подсчета. **81**

22 Дистрибутивная семантика в автоматической обработке текстов: понятие эмбеддинга (вектора слова). Нейросетевые языковые модели Word2Vec и FastText: способы обучения, семантические пространства слов. **84**

23 Современные нейросетевые языковые модели: особенности архитектуры и обучения, области применения. Понятие переноса обучения (Transfer Learning) и тонкой настройки (Fine-Tuning) на примере модели BERT. **87**

24 Машинный перевод: основные технологии автоматического перевода текста, их ограничения. Статистические и нейросетевые переводчики, принципы их построения. **90**

25 Извлечение информации из текстов: особенности задачи, виды извлекаемых данных, применяемые подходы. Задача извлечения мнений и анализа тональности текстов. **93**

26 Процессная модель управления проектом: основные процессы и их взаимодействие. Особенности управления программным проектом. **96**

27 Сетевое планирование проекта. Примеры создания сетевых графиков и назначения ресурсов. . . **100**

1 Понятие парадигмы (стиля) программирования. Особенности императивного, функционального и логического программирования как парадигм вычисления.

Парадигма программирования — это совокупность идей, концепций и методов, определяющих фундаментальный стиль написания компьютерных программ. Она представляет собой высокоуровневый способ мышления о структуре и выполнении программ, влияющий на то, как программист воспринимает задачу и формулирует её решение.

В программировании парадигма задаёт:

- Модель вычислений: Как программа будет выполняться (например, последовательное изменение состояния, вычисление функций, логический вывод).
- Структуру программы: Как программа будет организована (например, набор процедур, классов, функций, правил).
- Основные строительные блоки: Какие элементы являются ключевыми (например, переменные, объекты, функции, факты и правила).
- Подход к решению задач: Как программист должен декомпозировать проблему и конструировать решение.

Парадигма программирования не определяется однозначно языком программирования; практически все современные языки программирования в той или иной мере допускают использование различных парадигм.

а) Императивное программирование

Суть: Описывает вычисления в терминах последовательности команд (инструкций), которые изменяют состояние программы. Программа представляет собой алгоритм, который шаг за шагом указывает компьютеру, как выполнить задачу.

Модель вычисления: Основана на машине Тьюринга и архитектуре фон Неймана. Программа манипулирует данными, хранящимися в ячейках памяти (переменных), изменяя их значения.

Ключевые особенности и выкладки:

1. Явное управление состоянием:
   - Центральным понятием является состояние программы, представленное значениями переменных.
   - Инструкции (операторы присваивания, циклы, условные операторы) явно изменяют это состояние.
   - Пример: $\mathrm{x} = 5; \mathrm{x} = \mathrm{x} + 1;$ – состояние переменной x изменяется.
2. Последовательное выполнение команд:
   - Инструкции выполняются в том порядке, в котором они написаны, если только порядок не изменен управляющими конструкциями.
3. Управляющие структуры:
   - Используются для изменения порядка выполнения команд:
     - Условные операторы (if-then-else, switch-case) для выбора пути выполнения.
     - Циклы (for, while, do-while) для многократного выполнения блока кода.
     - Операторы перехода (например, goto, хотя его использование не рекомендуется в структурном программировании).
4. Побочные эффекты (Side Effects):
   - Функции или процедуры могут изменять состояние программы за пределами своей локальной области видимости (например, модифицировать глобальные переменные, выполнять операции ввода-вывода). Это является неотъемлемой частью парадигмы.

Преимущества:

- Интуитивно понятна для многих задач, так как близка к тому, как люди описывают последовательность действий.
- Эффективна, так как непосредственно отражает архитектуру большинства современных компьютеров.
- Большое количество существующих библиотек и инструментов.

Недостатки:

- Сложность управления состоянием в больших программах, что может приводить к ошибкам.
- Побочные эффекты затрудняют рассуждения о корректности программы и её параллелизацию.
- Код может быть менее декларативным ("как"вместо "что").

б) Функциональное программирование

Суть: Процесс вычислений рассматривается аналогично тому, как в математике вычисляется значение функции. Программа состоит из набора функций, которые преобразуют входные данные в выходные без изменения состояния и без побочных эффектов. Акцент делается на том, что нужно вычислить, а не как.

Модель вычисления: Основана на лямбда-исчислении Алонзо Чёрча.

Ключевые особенности и выкладки:

1. Чистые функции (Pure Functions):
   - Функция всегда возвращает один и тот же результат для одних и тех же входных данных.
   - Функция не имеет побочных эффектов (не изменяет глобальные переменные, не выполняет ввод-вывод напрямую).
   - Эта особенность называется ссылочной прозрачностью (referential transparency): вызов функции можно заменить её результатом без изменения поведения программы.

2. Неизменяемость данных (Immutability):
   - Данные (переменные, структуры данных) после создания не могут быть изменены. Вместо изменения существующих данных создаются новые.
   - Это упрощает рассуждения о программе и её параллелизацию, так как устраняет гонки данных.

3. Функции как объекты первого класса (First-Class Functions):
   - Функции могут передаваться как аргументы другим функциям, возвращаться из функций и присваиваться переменным.

4. Функции высшего порядка (Higher-Order Functions):
   - Функции, которые принимают другие функции в качестве аргументов или возвращают функции в качестве результата (например, map, filter, reduce).

5. Рекурсия вместо циклов:
   - Итеративные процессы часто реализуются с помощью рекурсии, так как традиционные циклы подразумевают изменение состояния (счетчика цикла).
   - Многие функциональные языки поддерживают оптимизацию хвостовой рекурсии (tail call optimization), чтобы избежать переполнения стека.

6. Ленивые вычисления (Lazy Evaluation):
   - Выражения вычисляются только тогда, когда их результат действительно необходим. Это позволяет работать с бесконечными структурами данных и оптимизировать производительность.

Преимущества:
- Повышенная надежность: Отсутствие побочных эффектов и неизменяемость данных уменьшают количество ошибок, связанных с состоянием.
- Упрощенное тестирование и отладка: Чистые функции легко тестировать изолированно.
- Параллелизм: Программы легче распараллеливать, так как нет общих изменяемых данных.
- Композируемость: Функции легко комбинируются для создания более сложных.
- Более декларативный стиль: Код часто описывает "что"вычислить, а не "как".

Недостатки:
- Может быть менее интуитивным для задач, где состояние играет центральную роль (хотя существуют техники для управления состоянием, например, монады).
- Работа с вводом-выводом (который по своей природе является побочным эффектом) требует специальных подходов.
- Иногда может приводить к снижению производительности из-за создания новых объектов вместо изменения существующих (хотя оптимизации компилятора и персистентные структуры данных могут это нивелировать).

Примеры языков: Haskell, Lisp, Scheme, Clojure, Scala (мультипарадигменный с сильной поддержкой ФП).

Роль лямбда-исчисления в функциональном программировании:

Фундаментальной теоретической основой для функционального программирования служит лямбда-исчисление (λ-исчисление), разработанное Алонзо Чёрчем в 1930-х годах. Это формальная система, которая определяет понятие вычисления через применение и абстракцию функций. В лямбда-исчислении всё является функцией: функции могут принимать другие функции в качестве аргументов и возвращать функции в качестве результатов. Ключевыми концепциями являются лямбда-абстракция (создание анонимных функций, например, $\lambda x.x + 1$ — функция, которая принимает x и возвращает $x + 1$) и аппликация — применение функции к аргументу (например, $(\lambda x.x + 1)3$ означает применение функции к числу 3, результат $3 + 1 = 4$). Многие функциональные языки программирования можно рассматривать как практическую реализацию или "синтаксический сахар"над лямбда-исчислением, заимствуя из него идеи чистых функций, функций высшего порядка и рекурсии как основного механизма итерации.

в) Логическое программирование

Суть: Описывает вычисления в терминах логических утверждений (фактов и правил). Программа представляет собой базу знаний, а выполнение программы — это процесс логического вывода (доказательства) некоторого утверждения (запроса) на основе этой базы знаний. Программист описывает, что является истинным и какие существуют отношения, а система сама находит решение.

Модель вычисления: Основана на исчислении предикатов первого порядка, в частности, на подмножестве, называемом дизъюнктами Хорна. Основной механизм вывода — резолюция и унификация.

Ключевые особенности и выкладки:

1. Декларативность:
   - Программист определяет факты (безусловные утверждения) и правила (условные утверждения, импликации).
   - Например: $\mathrm{parent(john, mary)}$. (факт: Джон родитель Мэри), $\mathrm{ancestor(X, Y)}$:- $\mathrm{parent(X, Y)}$. (правило: X предок Y, если X родитель Y).

2. База знаний и запросы:
   - Программа состоит из набора фактов и правил.
   - Пользователь формулирует запрос (query) к базе знаний, который система пытается доказать.
   - Запрос может содержать переменные, и система попытается найти значения этих переменных, при которых запрос становится истинным.

3. Механизм вывода (Inference Engine):
   - Использует унификацию (сопоставление термов путем нахождения подходящих подстановок для переменных) и резолюцию (правило вывода, позволяющее из двух дизъюнктов вывести новый).

4. Бэктрекинг (Backtracking):
   - Если система при поиске решения заходит в тупик или находит одно решение и нужно найти другие, она может вернуться (откатиться) к предыдущей точке выбора и попробовать другой путь. Это встроенный механизм поиска.

5. Отношения, а не функции:
   - Логическое программирование фокусируется на определении отношений между сущностями, а не на функциях, отображающих входные данные в выходные. Одно и то же отношение может использоваться для "вычисления"в разных направлениях.
   - Например, правило $\mathrm{append(L1, L2, L3)}$ (список L3 является результатом конкатенации L1 и L2) может быть использовано для:
     - $\mathrm{append([1,2], [3,4], X)}$. -> $\mathrm{X = [1,2,3,4]}$ (конкатенация)
     - $\mathrm{append(X, [3,4], [1,2,3,4])}$. -> $\mathrm{X = [1,2]}$ (найти первый список)
     - $\mathrm{append([1,2], X, [1,2,3,4])}$. -> $\mathrm{X = [3,4]}$ (найти второй список)

Преимущества:
- Очень высокий уровень абстракции, код часто близок к спецификации задачи.
- Хорошо подходит для задач, связанных с искусственным интеллектом, экспертными системами, обработкой естественного языка, базами данных и символьными вычислениями.
- Встроенный механизм поиска и сопоставления с образцом.

Недостатки:

- Может быть неэффективным для задач, требующих интенсивных численных расчетов или низкоуровневого управления ресурсами.
- Порядок правил и фактов может влиять на эффективность и даже на завершимость поиска решения (проблема управления поиском).
- Логика выполнения программы (порядок вывода) может быть не всегда очевидна для программиста.
- Ограниченная область применения по сравнению с императивным программированием.

Примеры языков: Prolog, Mercury.

Теоретические основы

1. Логика первого порядка (First-Order Logic, FOL): Логическое программирование (в частности, Пролог) основано на подмножестве логики первого порядка. FOL – это формальная система, позволяющая выражать утверждения о свойствах объектов и отношениях между ними.

- **Основные компоненты FOL:**
  - **Константы:** Символы, обозначающие конкретные объекты (например, john, apple, 5).
  - **Переменные:** Символы, которые могут представлять любой объект (например, X, Y, Person). В Прологе обычно начинаются с заглавной буквы или подчеркивания.
  - **Предикаты (или отношения):** Символы, обозначающие свойства объектов или отношения между ними. Предикат с аргументами образует **атомарную формулу** (или атом). Например, $\mathrm{parent}(\mathrm{john}, \mathrm{mary})$ (предикат parent с двумя аргументами).
  - **Функциональные символы (функторы):** Используются для построения **термов**, представляющих объекты, которые являются функциями от других объектов. Например, $\mathrm{father}(\mathrm{john})$ мог бы быть термом, обозначающим отца Джона. В Прологе структуры вида $\mathrm{functor}(\mathrm{arg1}, \mathrm{arg2}, ...)$ являются термами.
  - **Логические связки:** $\wedge$ (И, конъюнкция), $\vee$ (ИЛИ, дизъюнкция), $\neg$ (НЕ, отрицание), $\rightarrow$ (импликация, "если...то..."), $\leftrightarrow$ (эквивалентность).
  - **Кванторы:**
    * $\forall$ (всеобщности, "для всех"): $\forall X.P(X)$ означает, что $P(X)$ истинно для любого X.
    * $\exists$ (существования, "существует"): $\exists X.P(X)$ означает, что существует хотя бы один X, для которого $P(X)$ истинно.
- В логическом программировании программа состоит из утверждений, являющихся формулами FOL.

2. Дизъюнкты Хорна (Horn Clauses): Это специальный вид формул в FOL, на котором основано большинство систем логического программирования, включая Пролог. Дизъюнкт Хорна – это дизъюнкция (логическое ИЛИ) литералов (атомарных формул или их отрицаний), в которой **не более одного положительного литерала**.

- Стандартная форма дизъюнкта Хорна: $L_1 \vee L_2 \vee \cdots \vee L_k$, где каждый L_i – литерал, и не более одного L_i является положительным (не содержит отрицания).
- В Прологе дизъюнкты Хорна обычно записываются в форме импликации: $P \leftarrow Q_1, Q_2, \ldots, Q_n$ Это эквивалентно $P \vee \neg Q_1 \vee \neg Q_2 \vee \cdots \vee \neg Q_n$. Здесь P – единственный положительный литерал.
- **Типы дизъюнктов Хорна в Прологе:**
  - **Факты (Definite clauses):** Утверждения, которые всегда истинны. Содержат один положительный литерал и не содержат отрицательных.
    * Логическая форма: P.
    * Пролог: $\mathrm{p.}$ или $\mathrm{p}(\mathrm{Args})$.
    * Пример: $\mathrm{parent}(\mathrm{john}, \mathrm{mary})$. (Джон родитель Мэри.)
  - **Правила (Definite clauses with a body):** Утверждения, истинность которых зависит от истинности других утверждений. Содержат один положительный литерал (голова правила) и один или более отрицательных литералов (тело правила, после переноса в правую часть импликации становятся конъюнкцией).
    * Логическая форма: $P \leftarrow Q_1 \wedge Q_2 \wedge \cdots \wedge Q_n$.
    * Пролог: $\mathrm{p}(\mathrm{ArgsHead})$:- $\mathrm{q1}(\mathrm{Args1})$, $\mathrm{q2}(\mathrm{Args2})$, ..., $\mathrm{qN}(\mathrm{ArgsN})$.
      · $\mathrm{p}(\mathrm{ArgsHead})$ – **голова (head)** правила.
      · :- – символ импликации ("если").
      · $\mathrm{q1}(\mathrm{Args1})$, $\mathrm{q2}(\mathrm{Args2})$, ..., $\mathrm{qN}(\mathrm{ArgsN})$ – **тело (body)** правила, представляющее собой конъюнкцию (логическое И) подцелей.

    * Пример: ancestor(X, Y) :- parent(X, Y). (X предок Y, если X родитель Y.)
  - **Запросы (Цели) (Goal clauses):** Утверждения, истинность которых мы хотим проверить или для которых хотим найти значения переменных, делающие их истинными. Состоят только из отрицательных литералов.
    * Логическая форма: $\leftarrow Q_1 \wedge Q_2 \wedge \cdots \wedge Q_n$.
    * Пролог: ?- q1(Args1), q2(Args2), ..., qN(ArgsN).
    * Пример: ?- ancestor(john, Who). (Кто является потомком Джона?)
- Использование дизъюнктов Хорна делает процесс вывода более эффективным и управляемым.

Механизм вывода

1. Унификация (Unification): Фундаментальная операция в логическом программировании. Это процесс нахождения такой **подстановки** (отображения переменных в термы), которая делает два терма (или выражения) синтаксически идентичными.

- Если унификация возможна, результатом является **наиболее общий унификатор (Most General Unifier, MGU)** – это такая подстановка, что любая другая унифицирующая подстановка может быть получена из MGU путем применения дополнительной подстановки.
- **Примеры унификации:**
  - p(X, b) и p(a, Y) унифицируемы с MGU $\{X/a, Y/b\}$. Результат: p(a, b).
  - foo(X, X) и foo(a, b) не унифицируемы, так как X не может быть одновременно a и b.
  - parent(X, mary) и parent(john, mary) унифицируемы с MGU $\{X/$john$\}$.
  - f(X) и f(g(Y)) унифицируемы с MGU $\{X/$g(Y)$\}$.
- Унификация используется для сопоставления цели (или подцели) из запроса с головами правил или фактами из базы знаний.

2. Резолюция (в частности SLD-резолюция):

- **Принцип резолюции** – это правило вывода, которое позволяет из двух дизъюнктов, содержащих комплементарные литералы (например, P и $\neg P$), вывести новый дизъюнкт (резольвенту).
  - Общая форма: $(A \vee B) \wedge (\neg B \vee C) \vdash (A \vee C)$.
- **SLD-резолюция (Selective Linear Definite clause resolution)** – это стратегия резолюции, используемая в Прологе:
  - **S (Selection):** Из текущего набора целей (запроса) выбирается одна (обычно самая левая).
  - **L (Linear):** Резольвента всегда выводится из предыдущей цели (резольвенты) и одной из программных клауз (факта или правила). Это формирует линейную цепочку вывода.
  - **D (Definite clauses):** Работает с определенными дизъюнктами Хорна (фактами и правилами).
- **Как работает SLD-резолюция (упрощенно):**

  1. Начинаем с **начального запроса (цели)**, например ?- G1, G2, ..., Gk.
  2. **Выбор подцели:** Система выбирает подцель из текущего запроса (в Прологе обычно самую левую, например, G1).
  3. **Поиск унифицируемого правила/факта:** Система ищет в базе знаний (сверху вниз) факт или правило, голова которого унифицируется с выбранной подцелью G1.
     - Пусть найдено правило H :- B1, B2, ..., Bm, где H унифицируется с G1 с помощью MGU θ.
  4. **Резолюция (генерация новой цели):**
     - Выбранная подцель G1 заменяется телом правила B1, B2, ..., Bm.
     - MGU θ применяется ко всему новому списку целей.
     - Новый запрос: ?- (B1, B2, ..., Bm, G2, ..., Gk)θ.
     - Если был выбран факт F. (правило с пустым телом), то подцель G1 просто удаляется, и MGU применяется к оставшимся целям: ?- (G2, ..., Gk)θ.
  5. **Повторение:** Шаги 2-4 повторяются с новым запросом.
  6. **Успех:** Если запрос становится пустым (все цели удовлетворены), это означает, что исходный запрос доказан. MGU, накопленный в процессе, дает значения переменных, при которых запрос истинен.
  7. **Провал:** Если для выбранной подцели не удается найти унифицируемый факт или правило, происходит провал на текущем пути вывода.

3. Бэктрекинг (Backtracking): Если на каком-то шаге система не может удовлетворить текущую подцель (провал) или если пользователь запросил другие решения, система возвращается к предыдущей точке выбора (где было найдено унифицируемое правило/факт) и пытается найти другой вариант унификации (другое правило или факт, которое еще не использовалось для этой подцели). Этот механизм "отката"и пробы другого пути называется бэктрекингом. Он позволяет систематически исследовать пространство поиска решений.

Пример на Прологе с объяснением

Рассмотрим простую базу знаний:

```
% Факты
parent(john, mary).      % Джон родитель Мэри
parent(john, tom).       % Джон родитель Тома
parent(mary, ann).       % Мэри родитель Энн
parent(mary, peter).     % Мэри родитель Питера

% Правила
% ancestor(Предок, Потомок)
ancestor(X, Y) :-        % X является предком Y,
    parent(X, Y).        % ЕСЛИ X является родителем Y.
                         % Голова: ancestor(X, Y)
                         % Тело: parent(X, Y)

ancestor(X, Z) :-        % X является предком Z,
    parent(X, Y),        % ЕСЛИ X является родителем некоторого Y,
    ancestor(Y, Z).      % И Y является предком Z.
                         % Голова: ancestor(X, Z)
                         % Тело: parent(X, Y), ancestor(Y, Z)
                         % Запятая в теле означает логическое "И"
```

- **Имя предиката:** parent, ancestor. Определяет отношение.
- **Аргументы:** john, mary, tom, ann, peter (константы, атомы). X, Y, Z (переменные, начинаются с большой буквы).

Как выполняется программа (на примере запроса): Запрос: ?- ancestor(john, Who). (Кто является потомком Джона?)

1. **Начальная цель:** ancestor(john, Who).

2. Система Пролог ищет правило или факт, голова которого унифицируется с ancestor(john, Who).
   - **Вариант 1: Используем первое правило** ancestor(X, Y) :- parent(X, Y).
     - Унификация ancestor(john, Who) с ancestor(X, Y): MGU = $\{X/\text{john}, Y/\text{Who}\}$.
     - Новая цель (тело правила с примененным MGU): parent(john, Who).
     - Ищем факт, унифицируемый с parent(john, Who):
       * Находим parent(john, mary). Унификация: MGU' = {Who/mary}.
       * Цель parent(john, mary) удовлетворена (факт – правило с пустым телом).
       * **РЕШЕНИЕ 1:** Who = mary. Система сообщает об этом. Если пользователь запрашивает еще решения (обычно нажатием ;), система инициирует бэктрекинг.

3. **Бэктрекинг (после первого решения):** Система возвращается к последней точке выбора. Это был выбор факта для parent(john, Who).
   - Ищем следующий факт, унифицируемый с parent(john, Who) (после parent(john, mary).):
     - Находим parent(john, tom). Унификация: MGU" = {Who/tom}.
     - Цель parent(john, tom) удовлетворена.
     - **РЕШЕНИЕ 2:** Who = tom. Снова запрос на бэктрекинг.

4. **Бэктрекинг (после второго решения):** Система возвращается к parent(john, Who). Больше фактов, начинающихся с parent(john, ...), нет. Провал на этой ветке. Теперь система откатывается еще дальше – к выбору правила для ancestor(john, Who). Первое правило ancestor(X,Y) :- parent(X,Y). исчерпало себя для john.

   - **Вариант 2: Используем второе правило** ancestor(X, Z) :- parent(X, Y), ancestor(Y, Z).
     - Унификация ancestor(john, Who) с ancestor(X, Z): MGU = $\{X/\text{john}, Z/\text{Who}\}$.
     - Новые цели (тело правила с примененным MGU): parent(john, Y), ancestor(Y, Who).
     - **Первая подцель:** parent(john, Y).
       * Ищем факт, унифицируемый с parent(john, Y):
         · Находим parent(john, mary). Унификация: MGU_Y = $\{Y/\text{mary}\}$.
         · Подцель parent(john, mary) удовлетворена.
         · Применяем MGU_Y к оставшимся целям: ancestor(mary, Who).
       * **Вторая подцель (новая текущая цель):** ancestor(mary, Who).
         · Теперь система пытается доказать ancestor(mary, Who). Она снова начнет с первого правила для ancestor.
         · **Используем первое правило** ancestor(A, B) :- parent(A, B). (переименовываем переменные, чтобы избежать конфликтов: A, B вместо X, Y)
         · Унификация ancestor(mary, Who) с ancestor(A, B): MGU_AB = $\{A/\text{mary}, B/\text{Who}\}$.
         · Новая цель: parent(mary, Who).
         · Ищем факт, унифицируемый с parent(mary, Who):
         · Находим parent(mary, ann). Унификация: MGU_Who1 = $\{\text{Who}/\text{ann}\}$.
         · Цель parent(mary, ann) удовлетворена.
         · **РЕШЕНИЕ 3:** Who = ann. Бэктрекинг.
         · Ищем следующий факт для parent(mary, Who):
         · Находим parent(mary, peter). Унификация: MGU_Who2 = $\{\text{Who}/\text{peter}\}$.
         · Цель parent(mary, peter) удовлетворена.
         · **РЕШЕНИЕ 4:** Who = peter. Бэктрекинг.
         · Больше фактов для parent(mary, ...) нет. Провал.
         · **Бэктрекинг к** ancestor(mary, Who)**. Используем второе правило** ancestor(A, C) :- parent(A, B), ancestor(B, C).
         · Унификация ancestor(mary, Who) с ancestor(A, C): MGU_AC = $\{A/\text{mary}, C/\text{Who}\}$.
         · Новые цели: parent(mary, B), ancestor(B, Who).
         · **Первая подцель:** parent(mary, B).
         · parent(mary, ann). MGU_B1 = $\{B/\text{ann}\}$. Новая цель: ancestor(ann, Who).
         · Пытаемся доказать ancestor(ann, Who).
         · Правило 1: parent(ann, Who). Фактов parent(ann, ...) нет. Провал.
         · Правило 2: parent(ann, Y1), ancestor(Y1, Who). Фактов parent(ann, ...) нет. Провал.
         · Провал для ancestor(ann, Who). Бэктрекинг.
         · parent(mary, peter). MGU_B2 = $\{B/\text{peter}\}$. Новая цель: ancestor(peter, Who).
         · Аналогично, для ancestor(peter, Who) будет провал, так как нет фактов о родительстве Питера.
         · Больше фактов для parent(mary, ...) нет. Провал.
         · Провал для ancestor(mary, Who) через второе правило.
       * **Бэктрекинг к** parent(john, Y), ancestor(Y, Who). Мы использовали Y=mary. Теперь попробуем следующую подстановку для Y.
         · Находим parent(john, tom). Унификация: MGU_Y2 = $\{Y/\text{tom}\}$.
         · Подцель parent(john, tom) удовлетворена.
         · Применяем MGU_Y2 к оставшимся целям: ancestor(tom, Who).
         · **Вторая подцель (новая текущая цель):** ancestor(tom, Who).
         · Пытаемся доказать ancestor(tom, Who).
         · Правило 1: parent(tom, Who). Фактов parent(tom, ...) нет. Провал.
         · Правило 2: parent(tom, Y2), ancestor(Y2, Who). Фактов parent(tom, ...) нет. Провал.
         · Провал для ancestor(tom, Who). Бэктрекинг.
       * Больше фактов для parent(john, ...) нет. Провал.
     - Второе правило ancestor(X,Z) :- parent(X,Y), ancestor(Y,Z). исчерпало себя для john.

5. Больше правил для $\mathrm{ancestor}$ нет. Поиск завершен.

Система выдаст решения: $\mathrm{Who} = \mathrm{mary}$, $\mathrm{Who} = \mathrm{tom}$, $\mathrm{Who} = \mathrm{ann}$, $\mathrm{Who} = \mathrm{peter}$. Порядок их получения зависит от порядка правил и фактов в базе и стратегии Пролога.

2 Ключевые принципы объектно-ориентированного программирования: абстракция, инкапсуляция, полиморфизм. Виды полиморфизма. Концепция обобщенного программирования.

Объектно-ориентированное программирование (ООП) — это парадигма программирования, основанная на представлении программы как совокупности взаимодействующих объектов, каждый из которых является экземпляром определенного класса. Классы образуют иерархию наследования. ООП стремится повысить гибкость, масштабируемость и переиспользуемость кода.

Фундаментальные принципы ООП:

1. Абстракция (Abstraction)

Определение: Абстракция — это процесс выделения наиболее существенных характеристик объекта или системы и игнорирование второстепенных, несущественных деталей. В ООП абстракция позволяет моделировать объекты реального мира или концептуальные сущности, фокусируясь на их поведении и атрибутах, релевантных для решаемой задачи.

Цель:

- Управление сложностью: Скрытие деталей реализации позволяет разработчикам работать с объектами на более высоком уровне, не вдаваясь в подробности их внутреннего устройства.
- Фокусировка на важном: Помогает определить четкий интерфейс объекта, через который с ним можно взаимодействовать.

Реализация в ООП:

- **Классы:** Являются основной формой абстракции, описывая общую структуру (атрибуты) и поведение (методы) для группы схожих объектов. Например, класс Автомобиль может абстрагировать такие свойства, как цвет, максимальнаяСкорость, и такие действия, как ехать(), тормозить().
- **Интерфейсы и абстрактные классы:** Позволяют определить контракт (набор методов), который должны реализовать конкретные классы, не указывая деталей реализации этих методов. Это еще один уровень абстракции, определяющий "что"объект должен делать, а не "как".

Пример: При проектировании системы для библиотеки, объект "Книга"может быть абстрагирован через такие характеристики, как "Название "Автор "ISBN и такие операции, как "Выдать "Вернуть". Детали хранения информации о книге (например, в какой таблице базы данных) или физического расположения на полке скрыты от пользователя этой абстракции.

2. Инкапсуляция (Encapsulation)

Определение: Инкапсуляция — это механизм объединения данных (атрибутов) и методов, которые работают с этими данными, внутри одной сущности (объекта), а также защита этих данных от прямого внешнего доступа. Доступ к данным и их изменение возможны только через определенный интерфейс (публичные методы объекта).

Цель:

- **Сокрытие данных (Data Hiding):** Защита внутреннего состояния объекта от несанкционированного или некорректного изменения извне.
- **Модульность:** Объект становится "черным ящиком внутреннюю реализацию которого можно изменять без влияния на другие части системы, если его публичный интерфейс остается неизменным.
- **Контроль доступа:** Объект сам контролирует, как используются его данные, обеспечивая их целостность.

Реализация в ООП:

- **Модификаторы доступа:** Языки программирования предоставляют ключевые слова (например, private, protected, public) для управления видимостью атрибутов и методов класса.
  - private: доступ только внутри класса.
  - protected: доступ внутри класса и в классах-наследниках.
  - public: доступ из любого места.
- Данные обычно объявляются как private, а для их чтения и изменения предоставляются публичные методы (геттеры и сеттеры), которые могут содержать логику проверки или преобразования данных.

Пример: В классе СчетВБанке баланс (private double balance) скрыт. Изменение баланса возможно только через публичные методы внестиСредства(сумма) и снятьСредства(сумма), которые могут проверять, например, не превышает ли сумма снятия текущий баланс.

3. Полиморфизм (Polymorphism)

Определение: Полиморфизм (от греч. "много форм") — это способность объектов с одинаковым интерфейсом (или общего родительского класса) иметь различную реализацию этого интерфейса. Это позволяет обрабатывать объекты разных классов единообразно через общий тип (ссылку на базовый класс или интерфейс).

Полиморфизм — это способность использовать один интерфейс для работы с объектами разных типов, при этом вызывается реализация, соответствующая конкретному типу объекта.

Цель:

- **Гибкость и расширяемость:** Позволяет добавлять новые классы с собственной реализацией поведения без изменения существующего кода, который работает с общим интерфейсом.
- **Упрощение кода:** Уменьшает количество условных конструкций (if-else или switch) для выбора поведения в зависимости от типа объекта.

Реализация в ООП:

- Тесно связан с **наследованием** (когда класс-потомок наследует интерфейс от класса-родителя и может переопределить его методы) и **реализацией интерфейсов**.
- **Виртуальные методы (Dynamic Dispatch / Late Binding):** Конкретный метод, который будет вызван, определяется во время выполнения программы (runtime) на основе фактического типа объекта, а не типа ссылки на него.

Пример: Пусть есть базовый класс Фигура с методом нарисовать(). Классы-потомки Круг, Прямоугольник, Треугольник наследуют этот метод и каждый реализует его по-своему. Можно создать массив ссылок типа Фигура, который будет содержать объекты Круг, Прямоугольник и Треугольник. При вызове метода нарисовать() для каждого элемента массива будет вызвана соответствующая реализация для конкретной фигуры.

```
// Псевдокод
class Фигура { virtual void нарисовать() { ... } }
class Круг extends Фигура { void нарисовать() override { /* рисуем круг */ } }
class Прямоугольник extends Фигура { void нарисовать() override { /* рисуем прямоугольник */ } }

Фигура[] фигуры = { new Круг(), new Прямоугольник() };
for (Фигура ф : фигуры) {
    ф.нарисовать(); // Вызовется метод Круг.нарисовать() или Прямоугольник.нарисовать()
}
```

Виды полиморфизма

1. Ad-hoc полиморфизм (Специальный полиморфизм)

Этот вид полиморфизма позволяет одной и той же функции (или оператору) работать с аргументами разных типов, при этом для каждого типа может существовать своя, независимая реализация.

- **Перегрузка функций/методов (Function/Method Overloading):** Возможность определять в одном классе или области видимости несколько функций/методов с одинаковым именем, но разным набором параметров (разное количество, типы или порядок аргументов). Компилятор выбирает нужную версию функции на основе типов аргументов при вызове. *Пример (C++):*
  ```
  void print(int i) { /* ... */ }
  void print(double d) { /* ... */ }
  void print(std::string s) { /* ... */ }
  print(5);       // вызовет print(int)
  print("hello"); // вызовет print(std::string)
  ```
- **Перегрузка операторов (Operator Overloading):** Возможность придавать стандартным операторам языка (например, +, -, *, ==) специальное значение для пользовательских типов (классов). *Пример (C++):* Определение оператора + для сложения двух объектов класса КомплексноеЧисло.

Ad-hoc полиморфизм обычно разрешается на этапе компиляции (статический полиморфизм).

2. Полиморфизм подтипов (Subtype Polymorphism / Inclusion Polymorphism)

Это "классический"полиморфизм в ООП, тесно связанный с наследованием и виртуальными методами. Он позволяет объекту класса-потомка использоваться везде, где ожидается объект класса-родителя (или интерфейса,

который он реализует). Выбор конкретной реализации метода происходит во время выполнения (динамический полиморфизм, позднее связывание).

- **Переопределение методов (Method Overriding):** Класс-потомок предоставляет собственную реализацию метода, который уже определен в его базовом классе (с той же сигнатурой).
- Ключевым механизмом является **динамическая диспетчеризация** (dynamic dispatch), когда вызов виртуального метода разрешается во время выполнения на основе фактического типа объекта.

Пример с классом Фигура и его потомками, приведенный выше, иллюстрирует именно полиморфизм подтипов.

3. Параметрический полиморфизм (Parametric Polymorphism)

Этот вид полиморфизма позволяет писать код (функции, классы), который может работать с данными различных типов, не зная этих типов заранее. Тип указывается как параметр при создании экземпляра класса или вызове функции.

- **Обобщенные типы (Generics в Java/C#, Шаблоны/Templates в C++):** Позволяют создавать классы, интерфейсы и методы, где один или несколько типов указываются как параметры. *Пример (Java):* Класс ArrayList<T> может хранить объекты любого типа T.
  ArrayList<String> списокСтрок = new ArrayList<String>();
  ArrayList<Integer> списокЧисел = new ArrayList<Integer>();
- Параметрический полиморфизм обычно обеспечивает типобезопасность на этапе компиляции.

Этот вид полиморфизма является основой для обобщенного программирования.

Определение: Обобщенное программирование — это парадигма программирования, направленная на написание алгоритмов и структур данных таким образом, чтобы они могли работать с различными типами данных, не будучи переписанными для каждого конкретного типа. Основная идея заключается в том, чтобы отделить алгоритм от конкретных типов данных, с которыми он оперирует.

Цель:

- **Переиспользование кода (Code Reusability):** Один и тот же обобщенный алгоритм или структура данных может использоваться для множества типов.
- **Типобезопасность (Type Safety):** Ошибки типов обнаруживаются на этапе компиляции, а не во время выполнения (в большинстве реализаций, например, C++ шаблоны, Java/C# Generics).
- **Эффективность:** Обобщенный код часто может быть столь же эффективным, как и специально написанный для конкретного типа, так как специализация (конкретизация типов) происходит на этапе компиляции.

Реализация:

- **Шаблоны (Templates) в C++:** Мощный механизм, позволяющий параметризовать классы и функции типами, а также константами времени компиляции. Компилятор генерирует конкретный код для каждого используемого набора параметров шаблона.
- **Обобщения (Generics) в Java, C# и других языках:** Позволяют определять классы, интерфейсы и методы с параметрами типа. В Java дженерики реализуются через "стирание типов"(после компиляции), в C# — с сохранением информации о типах во время выполнения.

Связь с параметрическим полиморфизмом: Обобщенное программирование является практической реализацией и расширением идей параметрического полиморфизма. Параметрический полиморфизм — это теоретическая концепция, а обобщенное программирование — это методология и набор инструментов для ее применения.

Примеры:

- **Обобщенные контейнеры:** std::vector<T>, std::list<T> в C++; List<T>, Dictionary<TKey, TValue> в C#; ArrayList<E>, HashMap<K,V> в Java.
- **Обобщенные алгоритмы:** Функции сортировки, поиска, копирования, которые могут работать с различными типами контейнеров и элементов, если те удовлетворяют определенным требованиям (например, поддерживают операции сравнения). Пример: std::sort в C++.

Отличие от других видов полиморфизма:

- В отличие от *полиморфизма подтипов*, который основан на иерархии наследования и виртуальных функциях (позднее связывание), обобщенное программирование часто использует статическое связывание и генерацию кода на этапе компиляции.
- В отличие от *ad-hoc полиморфизма* (перегрузки), где для каждого типа пишется отдельная реализация, обобщенное программирование стремится написать одну реализацию, работающую для широкого класса типов, удовлетворяющих определенным интерфейсам или концепциям.

3 Основные средства современных функциональных языков (на примере языка Хаскель).

Хаскель (Haskell) — это чисто функциональный язык программирования общего назначения со статической типизацией (тип переменной фиксирован при объявлении) и ленивыми вычислениями.

1. Чистота (Purity) и отсутствие побочных эффектов

Определение: Функции в Хаскеле являются "чистыми". Это означает, что:

- Для одних и тех же входных аргументов функция всегда возвращает один и тот же результат.
- Вычисление функции не вызывает побочных эффектов (side effects), таких как изменение глобальных переменных, выполнение операций ввода-вывода, генерация случайных чисел и т.д.

Это свойство называется **ссылочной прозрачностью (referential transparency)**: любое выражение может быть заменено его значением без изменения поведения программы.

Преимущества:

- Легче рассуждать о коде и доказывать его корректность.
- Упрощается тестирование (функцию можно тестировать изолированно).
- Открываются возможности для оптимизаций компилятором (например, мемоизация, параллелизация).

Управление побочными эффектами: Для взаимодействия с внешним миром (ввод-вывод, изменение состояния) Хаскель использует специальные конструкции, такие как **монады** (например, монада IO), которые позволяют изолировать и контролировать побочные эффекты, не нарушая чистоты основной части программы. Действия с побочными эффектами имеют тип, отражающий это, например IO Int (действие, которое при выполнении вернет Int).

2. Функции высшего порядка (Higher-Order Functions)

Определение: Функции в Хаскеле являются объектами первого класса. Это означает, что они могут:

- Передаваться в качестве аргументов другим функциям.
- Возвращаться в качестве результата других функций.
- Присваиваться переменным (в Хаскеле это означает связывание имени с функцией).

Функции, которые принимают другие функции в качестве аргументов или возвращают их, называются функциями высшего порядка.

Примеры:

- map :: (a -> b) -> [a] -> [b] — применяет функцию к каждому элементу списка.

```
map (+1) [1, 2, 3] -- результат: [2, 3, 4]
```

- filter :: (a -> Bool) -> [a] -> [a] — отбирает элементы списка, удовлетворяющие предикату.

```
filter even [1, 2, 3, 4] -- результат: [2, 4]
```

- Композиция функций (.) :: (b -> c) -> (a -> b) -> (a -> c).

```
(not . even) 3 -- результат: True (эквивалентно not (even 3))
```

Функции высшего порядка являются мощным инструментом абстракции и позволяют писать более компактный и выразительный код.

3. Каррирование (Currying) и частичное применение

Каррирование - преобразование функции от нескольких аргументов в суперпозицию функций от одного аргумента.

Каррирование: В Хаскеле все функции формально принимают только один аргумент. Функции, которые кажутся многоаргументными, на самом деле являются функциями, возвращающими новые функции. Например, функция add x y = x + y имеет тип add :: Int -> Int -> Int. Это означает, что add — это функция, которая принимает Int и возвращает функцию типа Int -> Int.

```
add :: Int -> Int -> Int
add x y = x + y
```

Вызов add 5 3 интерпретируется как (add 5) 3. Сначала add 5 возвращает новую функцию, которая добавляет 5 к своему аргументу, а затем эта новая функция применяется к 3.

Частичное применение (Partial Application): Благодаря каррированию, можно легко создавать новые функции, фиксируя некоторые аргументы существующих функций.

```
addFive :: Int -> Int
addFive = add 5  -- addFive теперь функция, которая добавляет 5

result = addFive 3 -- результат: 8
```

Это очень удобный механизм для создания специализированных функций на лету.

4. Статическая типизация и вывод типов (Strong Static Typing with Type Inference)

Статическая типизация: Типы всех выражений проверяются на этапе компиляции. Это помогает отлавливать многие ошибки до запуска программы.

Вывод типов (Hindley-Milner type system): Хаскель обладает мощной системой вывода типов. В большинстве случаев программисту не нужно явно указывать типы функций и переменных; компилятор сам выводит наиболее общий (полиморфный) тип.

```
-- Явная аннотация типа (часто не обязательна, но хорошая практика)
add :: Int -> Int -> Int
add x y = x + y

-- Компилятор сам выведет тип для `myFunc` как `(a -> b) -> a -> b`
myFunc f x = f x
```

Тем не менее, явные аннотации типов считаются хорошим стилем, так как улучшают читаемость и служат документацией.

Параметрический полиморфизм: Система типов Хаскеля поддерживает параметрический полиморфизм, позволяя писать функции, работающие с данными различных типов. Типы-переменные (например, a, b) обозначают любой тип.

```
identity :: a -> a
identity x = x

head :: [a] -> a
head (x:_) = x
```

5. Ленивые вычисления (Lazy Evaluation)

Определение: Выражения в Хаскеле вычисляются только тогда, когда их результат действительно необходим ("call-by-need"). Если значение выражения не используется, оно никогда не будет вычислено.

Преимущества:

- **Работа с бесконечными структурами данных:** Можно определять и использовать потенциально бесконечные списки или другие структуры, так как вычисляются только необходимые их части.

  ```
  ones :: [Int]
  ones = 1 : ones -- Бесконечный список единиц: [1, 1, 1, ...]

  take 5 ones -- результат: [1, 1, 1, 1, 1]
  ```

- **Улучшенная композиция функций:** Функции могут быть скомбинированы таким образом, что промежуточные структуры данных не создаются полностью, если их части не нужны.
- **Собственные управляющие структуры:** Ленивость позволяет определять новые управляющие конструкции как обычные функции (например, аналог if-then-else).

Недостатки:

- Иногда сложнее предсказать использование памяти и производительность.
- Отладка может быть затруднена из-за нестрогого порядка вычислений.

6. Сопоставление с образцом (Pattern Matching)

Определение: Мощный механизм для анализа структуры данных и извлечения ее компонентов. Он широко используется в определениях функций для различных вариантов входных данных.

```
-- Факториал
factorial :: Int -> Int
factorial 0 = 1              -- Базовый случай
factorial n = n * factorial (n - 1) -- Рекурсивный случай

-- Длина списка
length' :: [a] -> Int
length' []     = 0           -- Пустой список
length' (_:xs) = 1 + length' xs -- Непустой список (голова '_' игнорируется)

-- Пример с кортежами
fst' :: (a, b) -> a
fst' (x, _) = x
```

Сопоставление с образцом делает код более читаемым и декларативным, уменьшая необходимость в условных конструкциях.

7. Алгебраические типы данных (Algebraic Data Types, ADTs)

Определение: Позволяют создавать новые пользовательские типы данных путем комбинирования существующих. Существуют два основных вида ADT:

- **Типы-суммы (Sum Types):** Представляют выбор между несколькими вариантами (конструкторами данных). Используется символ |.

  ```
  data Shape = Circle Float              -- Круг с радиусом
             | Rectangle Float Float     -- Прямоугольник с шириной и высотой
             deriving (Show)

  area :: Shape -> Float
  area (Circle r) = pi * r * r
  area (Rectangle w h) = w * h
  ```

- **Типы-произведения (Product Types):** Объединяют несколько значений в одну структуру. Кортежи и записи являются примерами.

  ```
  data Point = Point Float Float deriving (Show) -- тип-произведение

  -- Запись (синтаксический сахар для типа-произведения с именованными полями)
  data Person = Person { name :: String
                       , age :: Int
                       } deriving (Show)
  ```

ADTs являются фундаментальным инструментом для моделирования данных в Хаскеле. Тип списка [] и тип Maybe также являются ADT.

```
data Maybe a = Nothing | Just a
data [a] = [] | a : [a] -- ( : ) - это инфиксный конструктор "cons"
```

8. Классы типов (Type Classes)

Определение: Механизм для реализации *ad-hoc полиморфизма* (перегрузки) в Хаскеле. Класс типов определяет набор функций (методов), которые должны быть реализованы для типа, чтобы он стал экземпляром этого класса.

- Это похоже на интерфейсы в ООП, но классы типов оперируют типами, а не объектами.
- Позволяют определять общие операции для различных типов, которые ведут себя схожим образом.

Примеры стандартных классов типов:

- Eq: типы, значения которых можно сравнивать на равенство (==, /=).

- Ord: типы, значения которых можно упорядочивать (<, <=, >, >=, compare).
- Show: типы, значения которых можно преобразовать в строку (show).
- Read: типы, значения которых можно прочитать из строки (read).
- Num: числовые типы, поддерживающие арифметические операции (+, -, *, abs, signum, fromInteger).
- Functor, Applicative, Monad: важные классы типов для работы с контейнерами и вычислениями с контекстом.

Определение экземпляра класса типов:

```
data Temperature = Celsius Float | Fahrenheit Float

instance Eq Temperature where
    (Celsius t1) == (Celsius t2) = t1 == t2
    (Fahrenheit t1) == (Fahrenheit t2) = t1 == t2
    -- Сравнение Celsius и Fahrenheit потребовало бы их конвертации
    _ == _ = False -- По умолчанию разные конструкторы не равны

-- Можно использовать `deriving (Eq, Show)` для автоматической генерации
-- стандартных реализаций для некоторых классов типов.
data Point = Point Float Float deriving (Eq, Show)
```

Ограничения на типы в сигнатурах функций указываются с помощью =>.

```
-- Функция `areEqual` работает с любым типом `a`,
-- который является экземпляром класса `Eq`.
areEqual :: Eq a => a -> a -> Bool
areEqual x y = x == y
```

9. Монады (Monads)

Определение: Монады — это особый вид классов типов (абстрактный тип данных), который используется в Хаскеле для структурирования вычислений, особенно тех, которые включают побочные эффекты или контекст (например, состояние, ввод-вывод, возможность отсутствия значения).

- Монада предоставляет способ последовательного выполнения операций, где результат предыдущей операции влияет на следующую.
- Определяет две основные операции:
  - return :: a -> m a: "упаковывает"чистое значение в монадический контекст.
  - (»=) :: m a -> (a -> m b) -> m b (bind): извлекает значение из монадического контекста, применяет к нему функцию (которая возвращает новое монадическое значение) и возвращает результат. Just 3 »= (ª> Just (r + 1))
- do-нотация — это синтаксический сахар для работы с монадами, делающий код похожим на императивный.

Примеры монад:

- Maybe: для вычислений, которые могут не вернуть результат (безопасная работа с null).
- Either e a: для вычислений, которые могут вернуть либо ошибку типа e, либо успешный результат типа a.
- [] (List): для недетерминированных вычислений (возвращающих несколько результатов).
- IO: для инкапсуляции операций ввода-вывода и других побочных эффектов.
- State s a: для вычислений с изменяемым состоянием типа s.

Пример с IO и do-нотацией:

```
main :: IO ()
main = do
    putStrLn "What is your name?"
    name <- getLine
    putStrLn ("Hello, " ++ name ++ "!")
```

Монады являются мощным и гибким инструментом, позволяющим сохранить чистоту языка при работе с нечистыми операциями.

4 Логическое программирование: вычисление как доказательство. Базовые вычислительные механизмы языка Пролог. Декларативная и процедурная семантика логических программ.

Логическое программирование представляет собой парадигму программирования, основанную на применении математической логики для решения задач. Вместо описания последовательности шагов для достижения результата, программист определяет набор логических утверждений (фактов и правил), а система пытается вывести ответ на поставленный вопрос (запрос) путем логического доказательства.

Центральная идея логического программирования заключается в том, что **программа — это теория (набор аксиом)**, а **вычисление — это конструктивное доказательство некоторого утверждения (цели) в рамках этой теории**.

- **Программа (База знаний):** Состоит из набора **фактов** (безусловных утверждений) и **правил** (условных утверждений или импликаций). Эти факты и правила формулируются в ограниченном подмножестве логики первого порядка, чаще всего на основе **дизъюнктов Хорна**. Например:
  parent(john, mary). % Факт: Джон родитель Мэри.
  ancestor(X, Y) :- parent(X, Y). % Правило: X предок Y, если X родитель Y.

- **Запрос (Цель):** Пользователь формулирует вопрос к базе знаний в виде цели, которую система должна попытаться доказать. Запрос может содержать переменные. Например:
  ?- ancestor(john, Who). % Запрос: Кто является потомком Джона?

- **Вычисление (Доказательство):** Система логического программирования (например, интерпретатор Пролога) пытается доказать истинность запроса, используя факты и правила из базы знаний. Если запрос содержит переменные, система пытается найти такие значения (подстановки) для этих переменных, при которых запрос становится истинным.
- **Результат:**
  - Если доказательство найдено, система обычно сообщает "да"(или "true") и, если были переменные, выводит их значения (найденные подстановки).
  - Если доказательство не найдено для текущих условий, система сообщает "нет"(или "false").

Этот подход отличается от императивных парадигм, где программа описывает "как"выполнить вычисление. В логическом программировании акцент делается на "что"известно и "какие"существуют зависимости.

Пролог (PROgramming in LOGic) — наиболее известный язык логического программирования. Его вычислительные механизмы основаны на следующих принципах:

1. Унификация (Unification)

Унификация — это процесс сопоставления двух термов (констант, переменных или структур) с целью сделать их синтаксически идентичными. Если термы можно унифицировать, результатом является **наиболее общий унификатор (Most General Unifier, MGU)** — набор подстановок переменных, делающий термы равными.

- Две константы унифицируемы, если они идентичны (например, atom и atom).
- Переменная унифицируема с любым термом. В результате переменная связывается с этим термом.
- Два структурированных терма (например, f(A,b) и f(c,B)) унифицируемы, если у них одинаковый функтор (имя структуры, здесь f) и одинаковое количество аргументов, и все соответствующие пары аргументов унифицируемы.
  - f(A,b) и f(c,B) унифицируются с MGU {A/c, B/b}.
- Переменная не может быть унифицирована с термом, который содержит эту же переменную (проверка на вхождение, "occurs check"), хотя в некоторых реализациях Пролога эта проверка может быть отключена для производительности.

Унификация является ключевой операцией при поиске подходящих правил и фактов для удовлетворения текущей цели.

2. SLD-резолюция (Selective Linear Definite clause resolution)

SLD-резолюция — это стратегия логического вывода, используемая Прологом для доказательства целей.

- **S (Selection):** Из текущего набора подцелей (конъюнкции в теле запроса или правила) Пролог выбирает одну (обычно самую левую) для попытки удовлетворения.
- **L (Linear):** Доказательство строится линейно: резольвента (новый набор целей) выводится из предыдущего набора целей и одной из программных клауз (факта или правила из базы знаний).

- **D (Definite clauses):** Программа состоит из определенных дизъюнктов Хорна (факты и правила вида "голова :- тело.").

Процесс SLD-резолюции (упрощенно):

1. **Начальная цель:** Запрос пользователя.
2. **Выбор подцели:** Выбирается первая (крайняя левая) подцель из текущего списка целей.
3. **Поиск в базе знаний:** Пролог просматривает базу знаний сверху вниз, ища факт или правило, голова которого унифицируется с выбранной подцелью.
4. **Унификация и резолюция:**
   - Если найден **факт**, голова которого унифицируется с подцелью: подцель считается доказанной. MGU применяется ко всем оставшимся подцелям и переменным.
   - Если найдено **правило**, голова которого унифицируется с подцелью: выбранная подцель заменяется телом этого правила. MGU применяется ко всем подцелям в новом списке.
5. **Новый список целей:** Процесс повторяется с новым списком целей.
6. **Успех:** Если список целей становится пустым, исходный запрос успешно доказан. Накопленные подстановки переменных представляют решение.
7. **Провал:** Если для выбранной подцели не найдено унифицируемого факта или правила, происходит провал на текущем пути вывода.

3. Бэктрекинг (Backtracking, Поиск с возвратом)

Когда Пролог сталкивается с провалом (не может удовлетворить текущую подцель) или когда пользователь запрашивает дополнительные решения, активируется механизм бэктрекинга.

- Система "откатывается"к последней точке выбора (месту, где было выбрано правило или факт из нескольких возможных вариантов).
- Все переменные, связанные после этой точки выбора, развязываются (их значения отменяются).
- Система пытается найти альтернативное правило или факт для удовлетворения той подцели, на которой был сделан предыдущий выбор.
- Если альтернатива найдена, поиск продолжается по новой ветке.
- Если альтернатив нет, система откатывается еще на один уровень выше, к предыдущей точке выбора.

Бэктрекинг обеспечивает систематический поиск в пространстве решений. Порядок правил и фактов в базе знаний, а также порядок подцелей в правилах, существенно влияет на процесс поиска и его эффективность.

4. Управление поиском (Cut, Отсечение - ‘!‘)

Пролог предоставляет специальный предикат "отсечение"(cut), обозначаемый восклицательным знаком !. Отсечение — это цель, которая всегда успешно выполняется при первом вызове, но оказывает влияние на механизм бэктрекинга:

- **"Зеленое"отсечение:** Используется для оптимизации, отсекая ветви поиска, которые заведомо не приведут к новым решениям, но не меняя декларативного смысла программы. Например, если известно, что после нахождения первого решения для определенной подцели другие искать не нужно.
- **"Красное"отсечение:** Используется для изменения логики программы, отсекая ветви, которые могли бы дать другие (возможно, нежелательные) решения. Использование красных отсечений может сделать программу менее декларативной и сложнее для понимания.

Эффект отсечения:

1. Все выборы, сделанные с момента выбора правила, содержащего отсечение, до самого отсечения, становятся фиксированными (коммит). Бэктрекинг через них невозможен.
2. Если отсечение пройдено, и последующие цели в теле правила терпят неудачу, то не только это правило считается неудачным, но и исходная цель (которая унифицировалась с головой этого правила) также считается неудачной, без попыток использовать другие правила для этой исходной цели.

Декларативная и процедурная семантика логических программ

1. Декларативная семантика

Суть: Описывает программу как набор логических утверждений (фактов и правил) и определяет, *что* является истинным на основе этих утверждений. Она связана с логическим смыслом программы, независимо от того, как система выполняет вычисления.

- **Факты** (например, p(a).) утверждают, что предикат p истинен для аргумента a.
- **Правила** (например, h :- b1, b2, ..., bn.) утверждают, что голова h истинна, *если* все подцели в теле (b1, b2, ..., bn) истинны. Это логическая импликация: $(b1 \wedge b2 \wedge \cdots \wedge bn) \rightarrow h$.
- **Запрос** (например, ?- g1, g2, ..., gk.) спрашивает, можно ли логически вывести конъюнкцию целей (g1 $\wedge$ g2 $\wedge \cdots \wedge$ gk) из программы.

Декларативная семантика позволяет программисту рассуждать о корректности программы с точки зрения логики: "Что эта программа говорит о мире?"

2. Процедурная семантика

Суть: Описывает, *как* Пролог-система выполняет программу для ответа на запрос. Она связана с конкретными шагами, которые предпринимает интерпретатор: порядок выбора подцелей, порядок просмотра правил, механизм унификации и бэктрекинга.

- **Порядок целей:** Пролог обрабатывает цели в правиле (и в запросе) слева направо.
- **Порядок правил:** Пролог просматривает правила и факты в базе знаний сверху вниз, чтобы найти унифицируемое с текущей целью.
- **Унификация:** Используется для сопоставления текущей цели с головами правил/фактов.
- **Бэктрекинг:** Если текущий путь вывода приводит к неудаче, система возвращается к последней точке выбора и пробует альтернативный путь.
- **Отсечение (!):** Влияет на бэктрекинг, изменяя процедурное поведение.

Процедурная семантика важна для понимания эффективности программы, ее завершимости (избегания бесконечных циклов) и управления процессом поиска.

Взаимосвязь и расхождения: В идеальном логическом языке декларативная и процедурная семантики должны совпадать: все, что логически следует из программы, должно быть выводимо системой, и наоборот. Однако в Прологе из-за его конкретной стратегии поиска (слева направо, сверху вниз) и наличия таких конструкций, как отсечение !, могут возникать расхождения:

- **Незавершимость:** Неправильный порядок правил или целей может привести к бесконечному циклу, даже если декларативно решение существует.

```
% Плохой порядок - может зациклиться для некоторых запросов
p(X, Y) :- p(X, Z), link(Z, Y).
p(X, Y) :- link(X, Y).
link(a,b).
% Запрос: ?- p(a, W).
```

- **Порядок решений:** Порядок, в котором находятся решения, зависит от процедурной семантики.
- **Влияние отсечения:** Отсечение напрямую изменяет процедурное поведение и может привести к тому, что программа перестает быть чисто декларативной (ее логический смысл становится зависимым от порядка выполнения).

5 Последовательная и параллельная сложность алгоритмов, информационный граф и ресурс параллелизма алгоритмов.

1. Последовательная сложность алгоритмов

Определение: Последовательная сложность алгоритма — это мера ресурсов (обычно времени или памяти), необходимых для выполнения алгоритма на однопроцессорной (последовательной) вычислительной машине в зависимости от размера входных данных.

- **Временная сложность** $T(n)$**:** Количество элементарных операций, выполняемых алгоритмом, как функция от размера входа n. Элементарные операции — это базовые операции, время выполнения которых считается постоянным (например, арифметические операции, сравнения, присваивания).
- **Пространственная сложность (емкостная сложность)** $S(n)$**:** Объем памяти, используемый алгоритмом, как функция от размера входа n. Включает память для входных данных, выходных данных и промежуточных вычислений.

Асимптотический анализ сложности: Поскольку точное количество операций или байт памяти часто зависит от конкретной машины и компилятора, для анализа сложности используется асимптотический подход, который описывает поведение функции сложности при увеличении размера входа $n \to \infty$.

- **О-большое ($\mathcal{O}$):** Верхняя граница. $f(n) = \mathcal{O}(g(n))$, если существуют положительные константы c и n_0 такие, что $0 \leq f(n) \leq c \cdot g(n)$ для всех $n \geq n_0$. Описывает наихудший случай.
- **Омега-большое (Ω):** Нижняя граница. $f(n) = \Omega(g(n))$, если существуют положительные константы c и n_0 такие, что $0 \leq c \cdot g(n) \leq f(n)$ для всех $n \geq n_0$. Описывает наилучший случай.
- **Тета (Θ):** Точная (асимптотически тесная) граница. $f(n) = \Theta(g(n))$, если $f(n) = \mathcal{O}(g(n))$ и $f(n) = \Omega(g(n))$. Описывает средний или типичный случай, когда верхняя и нижняя границы совпадают по порядку роста.

Классы сложности (примеры):

- $\mathcal{O}(1)$ — константная сложность (например, доступ к элементу массива по индексу).
- $\mathcal{O}(\log n)$ — логарифмическая сложность (например, бинарный поиск).
- $\mathcal{O}(n)$ — линейная сложность (например, поиск в несортированном массиве).
- $\mathcal{O}(n \log n)$ — линеарифмическая сложность (например, эффективные алгоритмы сортировки: слиянием, быстрая).
- $\mathcal{O}(n^2)$ — квадратичная сложность (например, сортировка пузырьком, вставками).
- $\mathcal{O}(n^c)$, $c > 1$ — полиномиальная сложность.
- $\mathcal{O}(c^n)$, $c > 1$ — экспоненциальная сложность (например, решение задачи коммивояжера полным перебором).
- $\mathcal{O}(n!)$ — факториальная сложность.

Алгоритмы с полиномиальной сложностью считаются "эффективными"или "быстрыми в то время как алгоритмы с экспоненциальной или факториальной сложностью считаются "медленными"для больших n.

2. Параллельная сложность алгоритмов

Определение: Параллельная сложность алгоритма — это мера ресурсов, необходимых для выполнения алгоритма на параллельной вычислительной машине. Основные метрики:

- **Параллельное время выполнения** $T_p(n)$**:** Время, прошедшее от момента начала вычислений на первом процессоре до момента окончания вычислений на последнем процессоре, при использовании p процессоров для задачи размера n.
- **Работа (Work)** $W(n)$ **или** $T_1(n)$**:** Суммарное количество элементарных операций, выполненных всеми процессорами. Эквивалентно времени выполнения на одном процессоре.
- **Стоимость (Cost)** $C_p(n)$**:** Произведение параллельного времени на количество используемых процессоров: $C_p(n) = p \cdot T_p(n)$. Алгоритм считается стоимостно-оптимальным, если $C_p(n) = \Theta(T_1(n))$.

Метрики эффективности параллельных алгоритмов:

- **Ускорение (Speedup)** $S_p(n)$**:** Отношение времени выполнения наилучшего последовательного алгоритма $T_s(n)$ (или $T_1(n)$ для того же параллельного алгоритма) ко времени выполнения параллельного алгоритма на p процессорах: $S_p(n) = \frac{T_s(n)}{T_p(n)}$ или $S_p(n) = \frac{T_1(n)}{T_p(n)}$. Идеальное (линейное) ускорение $S_p(n) = p$. На практике обычно $S_p(n) \leq p$ (закон Амдала, накладные расходы на синхронизацию и коммуникацию).
- **Эффективность (Efficiency)** $E_p(n)$**:** Отношение ускорения к количеству процессоров: $E_p(n) = \frac{S_p(n)}{p} = \frac{T_1(n)}{p \cdot T_p(n)}$. Идеальная эффективность $E_p(n) = 1$.

Закон Амдала: Указывает на теоретический предел ускорения, который может быть достигнут при распараллеливании алгоритма. Если f — доля последовательной части алгоритма (которую нельзя распараллелить), то максимальное ускорение S_{max} даже при неограниченном количестве процессоров ($p \to \infty$) равно: $S_{\text{max}} = \frac{1}{f+\frac{1-f}{p}} \xrightarrow{p\to\infty} \frac{1}{f}$. Например, если 5% алгоритма не может быть распараллелено, максимальное ускорение не превысит $1/0.05 = 20$.

3. Информационный граф алгоритма

Определение: Информационный граф (или граф зависимостей по данным, граф потока данных) алгоритма — это ориентированный ациклический граф (DAG), вершины которого представляют элементарные операции (вычисления) алгоритма, а дуги — информационные зависимости между ними. Дуга из вершины u в вершину v означает, что результат операции u необходим для выполнения операции v.

- **Вершины:** Элементарные операции (например, сложение, умножение, чтение/запись). Каждой вершине может быть приписан "вес соответствующий времени выполнения операции.
- **Дуги:** Показывают поток данных и порядок выполнения. Если есть дуга (u, v), то операция v не может начаться до тех пор, пока операция u не завершится.

Свойства и использование информационного графа:

- **Визуализация параллелизма:** Граф наглядно показывает, какие операции могут выполняться параллельно (те, между которыми нет пути зависимости).
- **Определение критического пути:** Самый длинный путь в информационном графе (по сумме весов вершин на пути) называется **критическим путем**. Длина критического пути определяет минимально возможное время выполнения алгоритма даже при неограниченном количестве процессоров. $T_\infty(n)$ — время выполнения на неограниченном числе процессоров, равно длине критического пути.
- **Оценка работы:** Суммарный вес всех вершин графа (или общее число вершин, если все операции единичны) соответствует общей работе $W(n)$ или $T_1(n)$.
- **Структура параллелизма:** Форма графа (ширина, высота) дает представление о возможностях распараллеливания. "Широкий"и "неглубокий"граф обычно указывает на хороший потенциал параллелизма.

Пример: Вычисление $(a + b) * (c + d)$.

- Вершины: v_1 : temp1 $= a + b$; v_2 : temp2 $= c + d$; v_3 : result $=$ temp1 $*$ temp2.
- Дуги: (v_1, v_3), (v_2, v_3).
- Операции v_1 и v_2 могут выполняться параллельно.
- Критический путь: например, $v_1 \to v_3$ (если сложение и умножение занимают по 1 единице времени, то длина 2).

4. Ресурс параллелизма алгоритмов

Определение: Ресурс параллелизма (или средний параллелизм, степень параллелизма) — это мера, показывающая, сколько операций в среднем может выполняться параллельно в ходе работы алгоритма.

Расчет ресурса параллелизма $\Pi(n)$: Обычно определяется как отношение общей работы алгоритма к длине его критического пути (минимальному параллельному времени): $\Pi(n) = \frac{W(n)}{T_\infty(n)} = \frac{T_1(n)}{T_\infty(n)}$.

- $T_1(n)$ — общее число операций (работа).
- $T_\infty(n)$ — длина критического пути в информационном графе (время выполнения на неограниченном числе процессоров).

Ресурс параллелизма показывает, во сколько раз можно в принципе ускорить алгоритм за счет параллелизма, или, иными словами, какое максимальное количество процессоров может быть эффективно утилизировано этим алгоритмом. Если реальное число процессоров $p > \Pi(n)$, то дополнительные процессоры, скорее всего, будут простаивать, так как алгоритм не сможет обеспечить их работой.

Значение ресурса параллелизма:

- **Оценка масштабируемости:** Алгоритмы с высоким ресурсом параллелизма лучше масштабируются на системах с большим числом процессоров.
- **Выбор алгоритма:** При наличии нескольких алгоритмов для решения задачи, ресурс параллелизма может быть одним из критериев выбора для параллельной реализации.
- **Предел ускорения:** Ресурс параллелизма $\Pi(n)$ является верхней границей для достижимого ускорения $S_p(n)$, т.е., $S_p(n) \leq \Pi(n)$. Также $S_p(n) \leq p$. Следовательно, $S_p(n) \leq \min(p, \Pi(n))$.

Пример: Суммирование n чисел с помощью дерева.

- $T_1(n) = \mathcal{O}(n)$ (последовательное суммирование).
- $T_\infty(n) = \mathcal{O}(\log n)$ (на каждом уровне дерева пары чисел суммируются параллельно, высота дерева $\log n$).
- $\Pi(n) = \frac{\mathcal{O}(n)}{\mathcal{O}(\log n)} = \mathcal{O}\left(\frac{n}{\log n}\right)$.

Это означает, что для эффективного суммирования n чисел можно задействовать до $\mathcal{O}(n/\log n)$ процессоров.

Конечный параллелизм определяется информационной независимостью некоторых фрагментов в тексте программы. (Фрагменты можно запустить параллельно)

Массовый параллелизм определяется информационной независимостью итераций циклов программы. (Итерации цикла можно запустить параллельно)

6 Логическая модель представления знаний. Применение логики предикатов первого порядка: этапы представления знаний, основные сложности. Дескриптивные логики: концепты и роли, соотношение с логикой предикатов.

Представление знаний — это область искусственного интеллекта, занимающаяся разработкой формализмов для описания знаний о мире таким образом, чтобы компьютерные системы могли их использовать для решения интеллектуальных задач. Логические модели являются одним из наиболее формализованных и хорошо изученных подходов к представлению знаний.

Определение: Логическая модель представления знаний использует формальные языки логики для кодирования знаний в виде набора логических выражений (формул). Основная идея заключается в том, что знания представляются как совокупность утверждений об объектах, их свойствах и отношениях между ними, а также правил вывода, позволяющих получать новые знания из уже существующих.

Компоненты логической модели:

- **Формальный язык:** Определяет синтаксис для построения логических выражений (формул). Примеры: логика высказываний, логика предикатов первого порядка, модальные логики, дескриптивные логики.
- **Семантика:** Определяет значение (истинность или ложность) формул языка в некоторой интерпретации (модели мира).
- **Механизм вывода (Inference Engine):** Набор правил и процедур, позволяющих выводить новые формулы (знания) из заданного набора формул (базы знаний). Примеры: резолюция, modus ponens, натуральный вывод.

Преимущества логической модели:

- **Формальность и точность:** Логика предоставляет строгий и однозначный способ представления знаний.
- **Декларативность:** Акцент делается на том, *что* известно, а не на том, *как* это использовать.
- **Мощные механизмы вывода:** Существуют хорошо изученные и эффективные алгоритмы логического вывода.
- **Выразительность:** Логика предикатов позволяет описывать сложные отношения и квантифицировать утверждения.

Недостатки:

- **Вычислительная сложность:** Логический вывод в общих случаях (например, для полной логики предикатов первого порядка) может быть очень трудоемким или даже неразрешимым.
- **Трудности представления некоторых видов знаний:** Например, нечетких, неполных, противоречивых знаний, знаний о времени или причинно-следственных связях (требуются расширения стандартных логик).
- **Проблема фрейма (Frame Problem):** Сложность описания того, что *не* изменяется в результате некоторого действия.

2. Применение логики предикатов первого порядка (ЛППП)

Логика предикатов первого порядка (First-Order Predicate Logic, FOPL) является мощным и выразительным формализмом для представления знаний. Она позволяет описывать объекты, их свойства (предикаты) и отношения между ними, а также использовать кванторы всеобщности ($\forall$) и существования ($\exists$).

Этапы представления знаний с использованием ЛППП

1. **Анализ предметной области:**
   - Определение ключевых объектов, сущностей и концепций в рассматриваемой области.
   - Выявление важных свойств этих объектов и отношений между ними.
   - Формулировка знаний, которые необходимо представить, на естественном языке.

2. **Выбор словаря (онтологии):**
   - **Константы:** Символы для обозначения конкретных объектов (например, John, $\mathrm{Book123}$, Paris).
   - **Предикаты:** Символы для обозначения свойств объектов или отношений между ними. У каждого предиката есть арность (число аргументов). Например: $\mathrm{Student}(x)$ (x – студент), $\mathrm{Likes}(x, y)$ (x любит y), $\mathrm{Parent}(x, y)$ (x – родитель y).
   - **Функции (опционально):** Символы для обозначения функций, которые отображают объекты в другие объекты. Например: $\mathrm{mother_of}(x)$ (мать x), $\mathrm{sum}(x, y)$ (сумма x и y).

3. **Формализация знаний (построение аксиом):**

- Перевод утверждений из естественного языка в формулы ЛППП с использованием выбранного словаря.
- **Факты:** Представляются как атомарные формулы или их отрицания. Например: $\text{Student}(\text{Alice})$., $\text{Likes}(\text{Bob}, \text{Pizza})$.
- **Правила (общие утверждения):** Представляются с использованием логических связок ($\wedge, \vee, \neg, \rightarrow, \leftrightarrow$) и кванторов ($\forall, \exists$). Например:
  - "Все студенты умные": $\forall x(\text{Student}(x) \rightarrow \text{Smart}(x))$.
  - "У каждого человека есть мать": $\forall x(\text{Person}(x) \rightarrow \exists y(\text{Mother}(y, x)))$.
  - "Если X родитель Y и Y родитель Z, то X предок Z": $\forall x \forall y \forall z((\text{Parent}(x, y) \wedge \text{Parent}(y, z)) \rightarrow \text{Ancestor}(x, z))$.

4. **Построение базы знаний:** Совокупность формализованных фактов и правил образует базу знаний.

5. **Формулировка запросов и вывод новых знаний:**
   - Запросы к базе знаний формулируются как цели, которые нужно доказать.
   - Используются механизмы логического вывода для получения ответов на запросы и вывода новых, неявных знаний. Например, запрос "Является ли Алиса умной?": ?- $\text{Smart}(\text{Alice})$.

Основные сложности при применении ЛППП

- **Выразительность vs. Вычислительная сложность:** Полная ЛППП очень выразительна, но общезначимость формул в ней полуразрешима (существует алгоритм, который найдет доказательство, если оно есть, но может работать бесконечно, если его нет), а выполнимость неразрешима. Это делает автоматический вывод в полной ЛППП очень сложным. На практике используются подмножества ЛППП (например, дизъюнкты Хорна в Прологе) или эвристические методы.
- **Трудность формализации знаний естественного языка:** Перевод неформальных знаний, особенно нюансов и контекста естественного языка, в строгие логические формулы является сложной и трудоемкой задачей.
- **Монотонность логики:** В классической ЛППП добавление новых аксиом не может сделать ранее выводимые утверждения ложными. Это затрудняет работу с изменяющимися знаниями и рассуждениями "по умолчанию"(non-monotonic reasoning).
- **Представление неполной и неопределенной информации:** Стандартная ЛППП плохо справляется с ситуациями, когда информация неполна ("неизвестно") или вероятностна.
- **Масштабируемость:** Для больших баз знаний логический вывод может стать неэффективным.
- **Проблема выбора адекватного словаря (онтологии):** Создание хорошей онтологии, которая полно и непротиворечиво описывает предметную область, — сложная задача.

3. Дескриптивные логики (Description Logics, DL)

Определение: Дескриптивные логики (ДЛ) — это семейство формализмов для представления знаний, которые являются разрешимыми фрагментами логики предикатов первого порядка. Они ориентированы на описание терминологии (понятий и их свойств) предметной области. ДЛ лежат в основе языков веб-онтологий, таких как OWL (Web Ontology Language).

Ключевые элементы ДЛ:

- **Индивиды (Individuals):** Конкретные объекты в мире (аналоги констант в ЛППП). Например, $\text{john}, \text{myCar}$.
- **Концепты (Concepts):** Множества индивидов со схожими свойствами (аналоги унарных предикатов или классов). Например, $\text{Person}, \text{Car}, \text{Student}$.
- **Роли (Roles):** Бинарные отношения между индивидами (аналоги бинарных предикатов). Например, hasChild, $\text{owns}, \text{worksFor}$.

Конструкторы концептов и ролей: ДЛ предоставляют набор конструкторов для построения сложных концептов и ролей из атомарных (базовых):

- **Атомарные концепты** (например, A) и **атомарные роли** (например, R).
- **Верхний концепт ($\top$):** Концепт, содержащий все индивиды (универсум).
- **Нижний концепт ($\bot$):** Пустой концепт.
- **Логические конструкторы концептов:**
  - **Пересечение (Conjunction) $\sqcap$:** $C \sqcap D$ (индивиды, принадлежащие и C, и D). Аналог $\wedge$.
  - **Объединение (Disjunction) $\sqcup$:** $C \sqcup D$ (индивиды, принадлежащие C или D). Аналог $\vee$.

- **Отрицание (Negation) $\neg$:** $\neg C$ (индивиды, не принадлежащие C).
- **Кванторные ограничения на роли (Role Restrictions):**
  - **Квантор существования $\exists R.C$:** "Существует роль R в концепт C". Индивиды x, для которых существует индивид y такой, что $(x, y) \in R$ и $y \in C$. Например, Person $\sqcap$ $\exists$hasChild.Male (человек, имеющий хотя бы одного сына).
  - **Квантор всеобщности $\forall R.C$:** "Все значения роли R принадлежат концепту C". Индивиды x, у которых все y, связанные с x ролью R, принадлежат концепту C. Например, Person$\sqcap\forall$hasChild.Doctor (человек, все дети которого являются докторами).
- **Числовые ограничения (Number Restrictions) (в более выразительных ДЛ):**
  - $\geq nR$ (имеет по крайней мере n заполнителей роли R).
  - $\leq nR$ (имеет не более n заполнителей роли R).
  - $= nR$ (имеет ровно n заполнителей роли R).
- **Конструкторы ролей (в некоторых ДЛ):** инверсия роли (R^-), композиция ролей ($R \circ S$), иерархия ролей.

База знаний в ДЛ (Онтология): Состоит из двух частей:

- **TBox (Terminological Box):** Содержит аксиомы, описывающие терминологию (концепты и роли) и их взаимосвязи.
  - **Включение концептов (Subsumption):** $C \sqsubseteq D$ (концепт C является подконцептом D, все экземпляры C также являются экземплярами D).
  - **Эквивалентность концептов:** $C \equiv D$ (эквивалентно $C \sqsubseteq D$ и $D \sqsubseteq C$).
  - Аналогичные аксиомы для ролей (например, $R \sqsubseteq S$).

  Пример: Man $\equiv$ Person $\sqcap$ Male.
- **ABox (Assertional Box):** Содержит утверждения об индивидах.
  - **Утверждения о принадлежности концепту:** $C(a)$ (индивид a принадлежит концепту C).
  - **Утверждения о принадлежности роли:** $R(a, b)$ (индивиды a и b связаны ролью R).

  Пример: Person(John), hasChild(John, Mary).

Задачи вывода в ДЛ:

- **Проверка выполнимости (Satisfiability):** Существует ли интерпретация, в которой концепт C не пуст?
- **Проверка включения (Subsumption):** Является ли $C \sqsubseteq D$ следствием базы знаний?
- **Проверка экземпляра (Instance Checking):** Является ли $C(a)$ следствием базы знаний?
- **Извлечение экземпляров (Retrieval):** Найти всех индивидов a, для которых $C(a)$ является следствием.

Важным свойством ДЛ является то, что эти задачи вывода обычно **разрешимы**, и их вычислительная сложность хорошо изучена для различных ДЛ.

Соотношение ДЛ с логикой предикатов первого порядка

Дескриптивные логики являются фрагментами ЛППП. Каждое выражение ДЛ может быть переведено в эквивалентную формулу ЛППП.

- **Концепты** переводятся в унарные предикаты. $C(x)$.
- **Роли** переводятся в бинарные предикаты. $R(x, y)$.
- **Индивиды** переводятся в константы.
- **Конструкторы ДЛ переводятся следующим образом (примеры):**
  - $C \sqcap D \quad \Rightarrow \quad C(x) \wedge D(x)$
  - $C \sqcup D \quad \Rightarrow \quad C(x) \vee D(x)$
  - $\neg C \quad \Rightarrow \quad \neg C(x)$
  - $\exists R.C \quad \Rightarrow \quad \exists y(R(x, y) \wedge C(y))$
  - $\forall R.C \quad \Rightarrow \quad \forall y(R(x, y) \rightarrow C(y))$
- **Аксиомы TBox переводятся в импликации с кванторами всеобщности:**
  - $C \sqsubseteq D \quad \Rightarrow \quad \forall x(C(x) \rightarrow D(x))$
  - $C \equiv D \quad \Rightarrow \quad \forall x(C(x) \leftrightarrow D(x))$
- **Утверждения ABox переводятся в основные формулы (ground atomic formulas):**
  - $C(a) \quad \Rightarrow \quad C(a)$
  - $R(a, b) \quad \Rightarrow \quad R(a, b)$

Преимущество ДЛ заключается в том, что, ограничивая выразительность по сравнению с полной ЛППП, они обеспечивают разрешимость и часто более низкую вычислительную сложность задач вывода. Это делает их практичными для построения больших онтологий и систем, основанных на знаниях.

7 Сетевая модель представления знаний. Семантические сети и их свойства. Фреймы, их виды и структура, межфреймовые связи. Представление значений по умолчанию, понятие немонотонного вывода. Графы знаний: концепция, примеры.

Определение: Семантическая сеть — это ориентированный граф, вершины которого представляют понятия (объекты, события, концепты), а дуги (ребра) — семантические отношения между этими понятиями.

Компоненты семантической сети:

- **Узлы:** Представляют концепты. Концепты могут быть:
  - *Конкретными объектами* (экземплярами): "Собака Жучка "Город Москва".
  - *Абстрактными понятиями* (классами): "Собака "Город "Млекопитающее".
  - *Свойствами*: "Цвет красный "Размер большой".
  - *Действиями или событиями*: "Бежать "Читать книгу".
- **Дуги:** Представляют семантические отношения между узлами. Дуги обычно имеют метки, указывающие тип отношения.

Типичные виды отношений (меток дуг) в семантических сетях:

- **IS-A (является экземпляром / является подклассом):** Отношение наследования или классификации.
  - "Жучка" $\xrightarrow{\text{IS-A}}$ "Собака"(Жучка — это собака, экземпляр класса)
  - "Собака" $\xrightarrow{\text{IS-A}}$ "Млекопитающее"(Собака — это вид млекопитающего, подкласс)
- **PART-OF (является частью):** Отношение меронимии (часть-целое).
  - "Двигатель" $\xrightarrow{\text{PART-OF}}$ "Автомобиль"
- **HAS-A (имеет / свойство):** Отношение атрибуции или обладания.
  - "Собака" $\xrightarrow{\text{HAS-A}}$ "Хвост"
  - "Птица" $\xrightarrow{\text{HAS-PROPERTY}}$ "Способность летать"
- **Агентивные отношения:**
  - AGENT (агент действия): "Мальчик" $\xrightarrow{\text{AGENT}}$ "Бросать"
  - OBJECT (объект действия): "Мяч" $\xrightarrow{\text{OBJECT}}$ "Бросать"
- **Пространственные, временные, причинно-следственные** и другие отношения.

Свойства семантических сетей:

- **Наглядность:** Легко воспринимаются человеком.
- **Ассоциативность:** Позволяют моделировать ассоциативные связи, схожие с человеческим мышлением.
- **Наследование свойств:** Благодаря отношению IS-A, свойства и отношения, определенные для более общих понятий (классов), автоматически наследуются более частными понятиями (подклассами и экземплярами). Это позволяет избежать избыточности в представлении знаний.
- **Гибкость:** Можно добавлять новые узлы и отношения по мере необходимости.
- **Механизмы вывода:** Основной механизм вывода — это поиск путей в графе и наследование свойств. Например, чтобы ответить на вопрос "Умеет ли Жучка лаять? можно проследить путь: "Жучка" $\xrightarrow{\text{IS-A}}$ "Собака" $\xrightarrow{\text{HAS-ABILITY}}$ "Лаять".

Недостатки и ограничения семантических сетей:

- **Отсутствие стандарта:** Нет единого общепринятого набора типов узлов и отношений, что затрудняет унификацию и обмен знаниями между системами.
- **Проблема интерпретации меток:** Смысл меток дуг может быть неоднозначным и зависеть от контекста или разработчика.
- **Сложность представления кванторов и сложных логических выражений:** Семантические сети плохо приспособлены для выражения утверждений с кванторами всеобщности или существования, отрицаний, дизъюнкций и т.п..
- **Различие между типом и экземпляром:** Иногда нечетко проводится различие между узлом, представляющим класс, и узлом, представляющим экземпляр.
- **Вычислительная сложность вывода** на больших и сложных сетях.

2. Фреймы, их виды и структура, межфреймовые связи

Определение: Фрейм — это структура данных, которая используется для представления типичных ситуаций, объектов или понятий. Фрейм объединяет знания, относящиеся к одному понятию, в виде набора атрибутов (слотов) и их значений.

Структура фрейма: Фрейм обычно состоит из:

- **Имя фрейма (Frame Name):** Уникальный идентификатор фрейма (например, "Собака "Комната "Покупка в магазине").
- **Слоты (Slots):** Атрибуты или характеристики, описывающие фрейм. Каждый слот имеет:
  - **Имя слота (Slot Name):** Например, "Цвет "Размер "Возраст "Имеет частью".
  - **Значение слота (Slot Value):** Конкретное значение атрибута. Значение может быть:
    * Простым значением (число, строка, булево).
    * Именем другого фрейма (ссылка на другой фрейм, реализующая связь).
    * Процедурой (демоном), которая вычисляется при обращении к слоту или изменении его значения.
  - **Фасеты (Facets) или Аспекты слота (необязательно):** Уточняют информацию о слоте или его значении. Примеры фасетов:
    * VALUE: Основное значение слота.
    * DEFAULT: Значение по умолчанию (используется, если конкретное значение не задано).
    * IF-NEEDED: Процедура, которая вызывается, если значение слота необходимо, но не задано.
    * IF-ADDED: Процедура, которая вызывается при добавлении значения в слот.
    * IF-REMOVED: Процедура, которая вызывается при удалении значения из слота.
    * RANGE или TYPE: Ограничения на возможные значения слота (тип данных, диапазон).
    * CARDINALITY: Ограничения на количество значений (например, "одно "много").

Пример фрейма "Собака":

Имя фрейма: Собака
Слоты:
  IS-A:
    VALUE: Млекопитающее
  ИМЕЕТ_ЧАСТЬЮ:
    VALUE: Голова, Туловище, Лапы (4), Хвост
  ЗВУК:
    VALUE: "Гав"
    DEFAULT: "Лай"
  КОЛИЧЕСТВО_ЛАП:
    VALUE: 4
    TYPE: Integer
  ОКРАС:
    IF-NEEDED: процедура_определения_окраса_по_породе()

Виды фреймов:

- **Фреймы-прототипы (Фреймы-классы):** Описывают общие характеристики класса объектов или понятий (например, фрейм "Собака"выше).
- **Фреймы-экземпляры:** Описывают конкретный объект или ситуацию, являясь экземпляром фрейма-прототипа (например, фрейм "Моя собака Рекс который наследует слоты от "Собака"и заполняет их конкретными значениями).

Межфреймовые связи:

Фреймы организуются в системы фреймов (иерархии) с помощью связей между ними:

- **Связь IS-A (AKO – A Kind Of):** Связывает фрейм-экземпляр с его фреймом-прототипом или фрейм-прототип с более общим фреймом-прототипом. Обеспечивает наследование слотов, значений и процедур.
- **Связи через значения слотов:** Если значением слота одного фрейма является имя другого фрейма, это устанавливает связь между ними (например, слот "ВЛАДЕЛЕЦ"во фрейме "Собака"может ссылаться на фрейм "Человек").
- **Descendents / Instances:** Слот для указания всех фреймов-потомков (видовых фреймов-прототипов), которые являются более конкретными вариантами данного фрейма.
- **Examples:** Слот для перечисления всех фреймов-экземпляров (конкретных случаев или объектов), которые реализуют данный фрейм-прототип.

3. Представление значений по умолчанию, понятие немонотонного вывода

Значения по умолчанию (Default Values): Фреймы позволяют задавать значения по умолчанию для слотов. Если для экземпляра фрейма конкретное значение слота не указано, используется значение по умолчанию,

унаследованное от фрейма-прототипа. *Пример:* Если у фрейма "Птица"есть слот "СПОСОБНОСТЬ ЛЕТАТЬ"со значением по умолчанию "Да то для конкретной птицы (например, "Воробей"), если не указано иное, будет считаться, что она умеет летать.

Немонотонный вывод (Non-monotonic Reasoning): Классическая логика является монотонной: добавление новых знаний (аксиом) в базу знаний не может сделать ранее выводимые утверждения ложными. Однако в реальном мире часто приходится делать выводы на основе неполной информации, которые могут быть пересмотрены при поступлении новых данных.

- **Определение:** Немонотонный вывод — это тип рассуждений, при котором выводы могут быть отменены или изменены при добавлении новой информации.
- **Связь с фреймами и значениями по умолчанию:** Наследование значений по умолчанию является примером немонотонного вывода. *Пример:*
  1. Фрейм "Птица слот "СПОСОБНОСТЬ ЛЕТАТЬ DEFAULT: "Да".
  2. Фрейм "Пингвин IS-A: "Птица". Слот "СПОСОБНОСТЬ ЛЕТАТЬ VALUE: "Нет".
  3. Если мы знаем только, что "Твити — это птица мы по умолчанию заключаем, что Твити летает.
  4. Если мы затем узнаем, что "Твити — это пингвин то предыдущий вывод о способности летать отменяется, и мы заключаем, что Твити не летает (значение из более специфичного фрейма "Пингвин"переопределяет значение по умолчанию из "Птица").

  Это отличается от монотонной логики, где если бы мы вывели "Твити летает добавление факта "Твити — пингвин"не отменило бы предыдущий вывод без явного противоречия.

Немонотонный вывод важен для моделирования здравого смысла и рассуждений в условиях неполной информации.

4. Графы знаний (Knowledge Graphs): концепция, примеры

Концепция: Граф знаний (Knowledge Graph, KG) — это структурированное представление фактов, состоящее из сущностей, их атрибутов и отношений между ними, организованное в виде графа. По сути, это современное развитие идей семантических сетей и фреймовых систем, но с акцентом на масштаб, формальную семантику (часто основанную на стандартах Semantic Web, таких как RDF, RDFS, OWL) и использование в реальных приложениях.

Ключевые характеристики графов знаний:

- **Сущности (Entities):** Узлы графа, представляющие реальные или абстрактные объекты (люди, места, организации, концепты). Каждая сущность обычно имеет уникальный идентификатор (URI).
- **Отношения (Relationships/Predicates):** Меченые дуги, соединяющие сущности и описывающие семантические связи между ними (например, "родился в "работает в "является столицей").
- **Атрибуты (Attributes/Literals):** Свойства сущностей, значения которых являются литералами (строки, числа, даты), а не ссылками на другие сущности. (Например, сущность "Человек"может иметь атрибут "Дата рождения"со значением "1990-01-01").
- **Тройки (Triples):** Основная единица информации в многих графах знаний — это тройка вида (субъект, предикат, объект), например, (Барак_Обама, был_президентом, США).
- **Схема/Онтология (Schema/Ontology):** Определяет типы сущностей (классы), типы отношений и атрибутов, а также ограничения на их использование (например, домен и диапазон предиката). Часто используется RDFS или OWL.
- **Масштабируемость:** Графы знаний могут содержать миллиарды фактов.
- **Интеграция данных:** Позволяют интегрировать данные из различных источников.

Отличие от традиционных баз данных: В отличие от реляционных баз данных с жесткой схемой, графы знаний более гибкие и лучше подходят для представления сложных, разнородных (все вместе, не разные таблицы) и слабоструктурированных данных.

Пример: Google Knowledge Graph (упрощённо)

Представим, что пользователь вводит запрос “Леонардо да Винчи”. Справа Google показывает информационную панель. Эти данные берутся из графа знаний.

Сущности (узлы):

- id_leo — Леонардо да Винчи; типы: Человек, Художник, Изобретатель

- id_mona_lisa — Мона Лиза; тип: Картина
- id_louvre — Лувр; тип: Музей
- id_florence — Флоренция; тип: Город
- id_milan — Милан; тип: Город
- id_painting — Живопись; тип: Область_искусства

Отношения (дуги, предикаты):

- написал_картину
- находится_в_музее
- место_рождения
- место_смерти
- работал_в_городе
- область_деятельности

Атрибуты (литеральные значения):

Для id_leo**:**
- дата_рождения: “1452-04-15”
- дата_смерти: “1519-05-02”
- полное_имя: “Леонардо ди сер Пьеро да Винчи”

Для id_mona_lisa**:**
- год_создания_примерно: “1503”

Факты в виде троек (subject, predicate, object):

- (id_leo, написал_картину, id_mona_lisa)
- (id_mona_lisa, находится_в_музее, id_louvre)
- (id_leo, место_рождения, id_florence)
- (id_leo, работал_в_городе, id_milan)
- (id_leo, область_деятельности, id_painting)
- (id_leo, дата_рождения, “1452-04-15”)

Как работает Google с этим графом?

Если пользователь спрашивает: “Кто написал Мону Лизу?”, поисковая система:

1. Находит узел id_mona_lisa
2. Ищет входящую дугу с предикатом написал_картину
3. Получает субъект — id_leo
4. Выводит информацию о Леонардо да Винчи

Улучшение семантического поиска:

Обычный поиск по ключевым словам: Запрос “высокие здания в Париже” находит документы, где есть эти слова.

Семантический поиск с графом знаний:
- Граф знает, что “Париж” — это город во Франции.
- Он знает, какие сущности являются зданиями и имеют атрибут высота.
- Он может найти здания, у которых находится_в_городе = Париж и высота > заданного порога.
- Он может интерпретировать сложный запрос вроде “столица Франции достопримечательности”.
- **Семантическая сеть:** более ранняя и простая концепция, фокусируется на понятиях и их связях; часто неформальна.
- **Граф знаний:** современная и формальная модель; поддерживает масштаб, типы, свойства, источники, форматы (RDF, OWL); применяется в поиске, рекомендациях и системах вопросов-ответов.

Можно сказать, что графы знаний – это реализация идей семантических сетей на новом технологическом уровне, с учетом потребностей обработки больших объемов данных и создания интеллектуальных систем.

8 Понятие онтологии в инженерии знаний: состав онтологии, типы отношений концептов. Классификация онтологий, примеры онтологий разного вида. Принципы и основные шаги построения онтологии.

Онтология — это формальное описание предметной области (ПО), которое включает:

- **Понятия (концепты)** и их определения;
- **Связи между понятиями**, включая возможную иерархию (например, отношения типа IS-A или part-of);
- **Правила использования понятий**, определяющие допустимые значения и ограничения в рамках данной предметной области.

2. Состав онтологии

Онтология обычно включает следующие основные компоненты:

1. **Концепты (Понятия, Классы - Concepts, Classes):** Абстрактные группы, наборы или коллекции объектов. Могут быть организованы в иерархию. *Примеры:* "Человек "Животное "Автомобиль "Заболевание "Процесс".

2. **Свойства (Атрибуты - Properties, Attributes):** Характеристики концептов, описывающие их свойства или связывающие их с другими концептами. Свойства делятся на:
   - **Свойства объектов (Object Properties) / Роли (Roles):** Связывают экземпляры одного класса с экземплярами другого класса (или того же класса). *Примеры:* "имеет_частью"(связывает "Автомобиль"и "Двигатель"), "работает_в"(связывает "Человек"и "Организация").
   - **Свойства данных (Datatype Properties) / Атрибуты в узком смысле:** Связывают экземпляры класса с литеральными значениями (строки, числа, даты и т.д.). *Примеры:* "имеет_возраст"(для "Человек значение — число), "имеет_название"(для "Книга значение — строка).

3. **Отношения (Relationships):** Способы, которыми концепты могут быть связаны друг с другом. Это более общее понятие, часто реализуемое через свойства объектов. Отношения могут иметь характеристики, такие как транзитивность, симметричность, функциональность и т.д.

4. **Аксиомы (Axioms):** Утверждения (правила, ограничения), которые всегда считаются истинными в рамках данной онтологии. Аксиомы используются для определения концептов, установления ограничений на свойства и вывода новых знаний. *Примеры:*
   - "Все люди смертны."(Аксиома, связывающая концепт "Человек"со свойством "смертен").

3. Типы отношений концептов

  - **IS-A / subClassOf (является подклассом / вид-род):** Основное отношение наследования. Указывает, что один класс является специализацией другого (более общего) класса. Экземпляры подкласса также являются экземплярами надкласса. *Пример:* "Млекопитающее" $\xrightarrow{\text{subClassOf}}$ "Животное".
  - **instanceOf (является экземпляром):** Связывает конкретный индивид с его классом. *Пример:* "Собака Жучка" $\xrightarrow{\text{instanceOf}}$ "Собака".
  - **partOf / hasPart (является частью / имеет частью):** Описывает агрегацию или композицию. *Пример:* "Двигатель" $\xrightarrow{\text{partOf}}$ "Автомобиль"; "Автомобиль" $\xrightarrow{\text{hasPart}}$ "Колесо".
- **Ассоциативные отношения:** Описывают различные неиерархические связи между концептами. Типов таких отношений может быть множество, и они часто специфичны для предметной области. *Примеры:*
  - causes (вызывает): "Вирус" $\xrightarrow{\text{causes}}$ "Заболевание"
  - treats (лечит): "Лекарство" $\xrightarrow{\text{treats}}$ "Заболевание"
- **Атрибутивные отношения:** Связывают концепт с его характеристиками или свойствами (реализуются через свойства данных). *Пример:* "Человек" $\xrightarrow{\text{hasAge}}$ (литерал типа Integer).

4. Классификация онтологий, примеры онтологий разного вида

- **Глубина проработки**
  - *Heavy-weighted:* тяжёлые онтологии, содержащие аксиомы
  - *Light-weighted:* легковесные онтологии без аксиом

- **Степень охвата областей**
  - верхнего уровня (Upper Model) или General (общие знания для нескольких предметных областей)
  - предметных областей (Domain-oriented)
  - прикладных задач (представление модели корпорации)
- **Степень формализации, язык описания**
  - неформальные
  - формальные: на специальных языках (часто языки логики)
- **Владелец/пользователь онтологии**
  - индивидуальная
  - групповая (сообщество, компания)
  - всеобщие
- **Общая методология построения**
  - от слов к понятиям — лингвистическая онтология
  - от понятий — понятийная онтология
- **Тип отношений между понятиями**
  - неоднородные
  - однородные:
    * Таксономии: иерархия по связи IS-A
    * Партономии: иерархия по связи PART-OF
    * Генеалогии: отношение Предок-Потомок
    * Атрибутивные онтологии
    * Причинно-следственные

Несмотря на близость контекстов, слова *«семантическая сеть»* и *«онтология»* обозначают два никак не связанных понятия.

Одно из слов характеризует **способ представления знаний**, а другое — **способ хранения знаний**.

Примеры неоднородных онтологий

WordNet — тезаурус, лексический ресурс для английского языка (Принстонский университет):
- общая (верхнего уровня), легкая онтология;
- лингвистическая, неоднородная;
- формализованная.

Прикладные лингвистические онтологии: *рубрикаторы, тезаурусы*
- нет внутренней структуры понятий;
- непосредственная связь с терминами языка.

CYC (Encyclopedia) — представление общечеловеческих знаний и знаний здравого смысла (пример: наполненный стакан стоит держать донышком вниз):
- онтология верхнего уровня;
- весомая понятийная база;
- неоднородная формальная (язык CYCL).

5. Принципы и основные шаги построения онтологии

Основные принципы построения онтологий:
- **Ясность (Clarity):** Определения должны быть объективными, недвусмысленными и документированными на естественном языке.
- **Непротиворечивость (Coherence):** Онтология не должна содержать противоречивых утверждений. Выводы, следующие из аксиом, не должны противоречить определениям.
- **Расширяемость (Extendibility):** Онтология должна быть спроектирована так, чтобы можно было легко добавлять новые термины и определения без необходимости пересмотра существующих.

Основные шаги построения онтологии (обобщенный подход):

1. **Определение области и назначения онтологии:**
   - Какую предметную область будет покрывать онтология?
   - Для каких целей она будет использоваться? (например, для интеграции данных, для поддержки принятия решений, для семантического поиска).

- Какие вопросы должна помогать решать онтология?
- Кто будет пользователями онтологии?

2. **Рассмотрение возможности переиспользования существующих онтологий:** Поиск и анализ уже существующих онтологий (верхнеуровневых, доменных), которые могут быть использованы полностью или частично.

3. **Выделение ключевых концептов (терминов) онтологии:** Составление списка важных терминов и понятий в предметной области. Можно использовать различные источники: экспертные знания, тексты, существующие словари.

4. **Определение иерархии классов (концептов):** Организация концептов в таксономическую иерархию (с использованием отношения IS-A). Подходы:
   - *Сверху вниз (Top-down):* Начать с наиболее общих концептов и детализировать их.
   - *Снизу вверх (Bottom-up):* Начать с наиболее специфичных концептов и обобщать их.
   - *Комбинированный (Middle-out):* Начать с наиболее важных концептов и двигаться вверх и вниз по иерархии.

5. **Определение свойств классов (атрибутов и отношений):** Для каждого класса определить его свойства (атрибуты, описывающие его внутренние характеристики) и отношения, связывающие его с другими классами. Определить характеристики свойств (домен, диапазон, кардинальность, транзитивность и т.д.).

6. **Определение ограничений (фасетов) для слотов/свойств:** Уточнение типов значений, диапазонов, кардинальности и других ограничений для свойств.

7. **Формализация и кодирование онтологии:** Выбор языка представления онтологий (например, OWL, RDF-S) и кодирование онтологии с использованием специализированных редакторов (например, Protégé).

8. **Оценка и верификация онтологии:** Проверка на непротиворечивость, полноту, корректность, соответствие целям. Может включать автоматическую проверку с помощью логических выводителей (reasoners) и оценку экспертами.

9. **Документирование онтологии.**

10. **Сопровождение и развитие онтологии:** Онтологии редко бывают статичными; они требуют обновления и расширения по мере изменения предметной области или появления новых требований.

9 Лингвистические онтологии и тезаурусы: особенности состава и построения. Семантические отношения в онтологиях и тезаурусах, примеры.

Определение: Тезаурус — это словарь, в котором слова сгруппированы на основе семантической близости (синонимии) или других семантических отношений. В отличие от толкового словаря, который определяет слова, тезаурус предназначен для помощи в поиске слов с похожими или противоположными значениями, а также для навигации по семантическим связям.

Особенности состава тезауруса:

- **Лексические единицы (Лексемы):** Основными элементами являются слова или устойчивые словосочетания.
- **Группировка по значениям (Синсеты):** Ключевой особенностью многих электронных тезаурусов (например, WordNet) является группировка слов, имеющих одинаковое или очень близкое значение, в **синсеты (synsets - synonym sets)**. Каждый синсет представляет одно лексическое понятие. *Пример синсета для понятия "автомобиль":* {автомобиль, машина, авто, тачка (разг.)}.
- **Краткие определения:** Для каждого синсета часто приводится краткое толкование или пример употребления, помогающий различить значения омонимичных слов.
- **Семантические отношения:** Синсеты (или отдельные слова) связаны между собой различными семантическими отношениями (рассматриваются ниже).
- **Частеречная информация:** Слова в синсетах обычно привязаны к определенной части речи (существительное, глагол, прилагательное, наречие).

Особенности построения тезауруса:

- **Ручное или полуавтоматическое создание:** Традиционно тезаурусы создавались вручную лексикографами. Современные подходы могут включать полуавтоматические методы извлечения синонимов и других отношений из текстовых корпусов, но требуют значительной ручной верификации.
- **Опора на существующие словари и лингвистические знания.**
- **Фокус на лексическом уровне:** Основное внимание уделяется словам и их значениям, а не сложным концептуальным структурам предметных областей.
- **Итеративный процесс:** Построение и расширение тезауруса — это длительный итеративный процесс.

Примеры тезаурусов:

- **WordNet (Принстонский Ворднет):** Наиболее известный и широко используемый электронный тезаурус для английского языка. Содержит синсеты для существительных, глаголов, прилагательных и наречий, связанные различными семантическими отношениями. Существуют ворднеты и для других языков (например, Russian WordNet, EuroWordNet).
- **Тезаурус Роже (Roget's Thesaurus):** Классический печатный тезаурус, где слова организованы по тематическим категориям.

Разновидности тезаурусов

- **Идеографический (семантический) словарь** типа Тезауруса Роже:
  - цель — систематизация и классификация слов ЕЯ (для освоения лексики, подбора синонимов и близких по смыслу слов и т.п.);
  - связи слов, как правило, не специфицированы.
- **Информационно-поисковый тезаурус:**
  - цель — систематизация и категоризация лексики для индексирования документов и/или поисковых запросов в информационном поиске, повышения его эффективности.
- **Тезаурус типа WordNet:**
  - цель — детальное описание лексической системы конкретного ЕЯ для задач КЛ.

2. Лингвистические онтологии: особенности состава и построения

Определение: Лингвистическая онтология — это онтология, объектом описания которой является сам естественный язык или его аспекты. Она формализует знания о структуре языка, его единицах, их свойствах и отношениях.

Отличие от доменных онтологий: Если доменная онтология описывает понятия и отношения в некоторой предметной области (например, "медицина "география"), то лингвистическая онтология описывает сам язык, который используется для разговора об этих областях. Однако, на практике, лингвистические онтологии часто

тесно связаны с лексическими ресурсами и могут включать элементы, похожие на тезаурусы (например, WordNet иногда рассматривают как лексическую онтологию).

Особенности состава лингвистической онтологии:

- **Лингвистические концепты (классы):** Могут включать:
  - *Лексические единицы:* Слова, морфемы, фразы.
  - *Грамматические категории:* Часть речи, падеж, число, время, наклонение.
  - *Синтаксические конструкции:* Типы фраз (именная группа, глагольная группа), типы предложений.
  - *Семантические роли:* Агент, пациенс, инструмент.
  - *Фонемы, графемы.*
- **Свойства и отношения между лингвистическими концептами:**
  - Лексико-семантические отношения (см. ниже, часто пересекаются с тезаурусами).
  - Грамматические отношения (например, "словоформа имеет часть речи Существительное "глагол имеет время Прошедшее").
  - Синтаксические отношения (например, "именная группа является субъектом для глагольная группа").
- **Аксиомы и правила:** Формальные утверждения о лингвистических закономерностях. Например, правила согласования, правила словообразования.
- **Связь с лексическими ресурсами:** Лингвистические онтологии часто используют или расширяют существующие словари и тезаурусы.

Особенности построения лингвистической онтологии:

- **Опора на лингвистические теории:** Построение требует глубоких знаний в области лингвистики (морфологии, синтаксиса, семантики, прагматики).
- **Формализация:** Используются формальные языки представления онтологий (OWL, RDF-S) для обеспечения точности и возможности автоматического вывода.
- **Высокая сложность:** Язык — очень сложная и многогранная система, поэтому создание полной и непротиворечивой лингвистической онтологии является чрезвычайно сложной задачей.
- **Цель:** Обеспечение более глубокого "понимания"языка компьютерными системами, поддержка сложных задач NLP, таких как машинный перевод, семантический анализ, генерация текста.

Особенности:

- **Методология построения: от слов к понятиям**
  - **На пальцах:** Мы не придумываем абстрактные понятия из головы, а смотрим на реальные слова, которые люди используют, и пытаемся сгруппировать их по смыслу, чтобы выделить из них понятия.
  - **Пример:** Мы видим слова “кошка”, “кот”, “котенок”, “мурлыка”. Анализируя, как они используются, мы понимаем, что все они относятся к одному понятию “ДОМАШНЯЯ КОШКА” или “ПРЕДСТАВИТЕЛЬ СЕМЕЙСТВА КОШАЧЬИХ”. Мы идем от конкретных слов к обобщенному понятию.
- **Предполагается изучение конкретного ЕЯ и привлечение корпусов текстов**
  - **ЕЯ =** Естественный Язык (русский, английский, китайский и т.д.).
  - **Корпус текстов =** Огромная коллекция текстов на этом языке (книги, статьи, сайты).
  - **На пальцах:** Чтобы понять, как слова связаны с понятиями, нужно изучить, как эти слова реально используются. Для этого лингвисты и программисты анализируют большие объемы текстов.
  - **Пример:** Чтобы понять все значения слова “ключ” (дверной ключ, гаечный ключ, ключ к разгадке, родник), нужно посмотреть много предложений, где это слово встречается.
- **На выходе — менее формальный ресурс онтологического характера — лингвистическая онтология**
  - **Менее формальный:** Язык очень гибкий и многозначный. Поэтому лингвистическая онтология не такая строгая и однозначная, как, например, математическая теория. В ней больше гибкости и учета нюансов.
  - **На пальцах:** Это не жесткая таблица “слово = понятие”. Это скорее сложная сеть, где слова могут относиться к нескольким понятиям, а понятия могут выражаться разными словами, с разными оттенками смысла.
  - **Пример:** Слово “звезда” может быть астрономическим объектом, известной личностью или геометрической фигурой. Лингвистическая онтология попытается отразить эти разные смыслы и связи.
- **Поскольку понятия так или иначе связаны со значениями языковых единиц, для переноса на другой язык требуется серьезная адаптация**
  - **На пальцах:** Если мы сделали “карту понятий” для русского языка, она не подойдет один в один для английского. Слова и их связи в разных языках устроены по-разному, даже если понятия кажутся похожими.

  - **Пример:** В русском есть слова “друг”, “приятель”, “знакомый”. В английском есть “friend”, “acquaintance”. Прямого и полного соответствия нет. Поэтому лингвистическую онтологию приходится сильно переделывать для каждого нового языка.

Виды (типы лингвистических онтологий):

- **Однородные классификации (-ономии) — понятия узкой области в одном определенном отношении**
  - **-ономии:** Окончание, как в словах “таксономия” (классификация живых организмов), “гастрономия” (искусство приготовления пищи).
  - **На пальцах:** Это когда мы берем какую-то очень узкую тему и классифицируем понятия в ней по одному главному признаку.
  - **Пример:**
    * “Цветоонимия”: список названий цветов (красный, синий, желтый) – все они относятся к понятию “цвет”.
    * “Болезни сердца”: перечень конкретных заболеваний сердца (инфаркт миокарда, стенокардия, аритмия) – все они относятся к понятию “заболевание сердца”.
- **Тезаурус — понятия предметной области (или лексики языка) в отношениях синонимии, иерархии, ассоциации**
  - **На пальцах:** Это самый известный тип. Тезаурус – это словарь, который показывает не только значения слов, но и как они связаны друг с другом.
  - **Отношения:**
    * **Синонимия:** Слова с похожим значением (например, “умный”, “сообразительный”, “мудрый”).
    * **Иерархия (гипонимия/гиперонимия):** Отношения “вид-род” (например, “пудель” (вид) – это “собака” (род), “собака” (вид) – это “млекопитающее” (род)).
    * **Ассоциации:** Слова, которые часто используются вместе или связаны по смыслу (например, “врач” ассоциируется с “больница”, “лекарство”, “пациент”).
  - **Пример:** *WordNet* – знаменитый тезаурус для английского языка. В России есть *RuWordNet* или *RuThes*. Когда вы в текстовом редакторе ищете синонимы к слову – вы используете элементы тезауруса.
- **Рубрикатор (классификатор, каталог) — иерархическая/фасетная классификация понятий (терминов) предметной области**
  - **На пальцах:** Это способ упорядочить понятия (часто термины из какой-то области) в виде структуры, похожей на каталог в библиотеке или разделы на сайте.
  - **Иерархическая классификация:** Как дерево – от общего к частному (например, “Наука” → “Физика” → “Квантовая механика”).
  - **Фасетная классификация:** Позволяет классифицировать одно и то же понятие по разным независимым признакам (фасетам).
    * **Пример фасетной классификации:** Представьте, что вы выбираете одежду в интернет-магазине. Вы можете фильтровать по:
      · Типу (фасет 1): Футболка, Рубашка, Брюки
      · Цвету (фасет 2): Красный, Синий, Зеленый
      · Размеру (фасет 3): S, M, L

      Вы можете выбрать “Рубашка” + “Синий” + “M”.
  - **Пример рубрикатора:** Универсальная десятичная классификация (УДК) в библиотеках, классификатор товаров в интернет-магазине, медицинский классификатор болезней (МКБ).

3. Семантические отношения в онтологиях и тезаурусах, примеры

Семантические отношения являются ядром как тезаурусов, так и лингвистических онтологий, хотя в онтологиях они могут быть более формализованы и типизированы.

Основные лексико-семантические отношения:

1. **Синонимия:** Отношение между словами или синсетами, имеющими одинаковое или очень близкое значение. *Примеры:*
   - {автомобиль, машина, авто}
   - {красивый, прекрасный, великолепный}
   - {быстро, скоро}

В WordNet синонимы объединены в синсеты.

2. **Антонимия:** Отношение между словами или синсетами с противоположными значениями. *Примеры:*
   - "хорошийплохой"
   - "большоймаленький"
   - "приходитьуходить"

3. **Гипонимия / Гиперонимия (Hyponymy / Hypernymy):** Отношение "вид-род"или "частное-общее". Если A является гипонимом B, то B является гиперонимом A. Это отношение формирует иерархию IS-A (в более широком смысле, чем subClassOf для классов).
   - **Гипоним (Hyponym):** Более специфичное понятие.
   - **Гипероним (Hypernym):** Более общее понятие.

   Примеры:
   - "пудель"(гипоним) – "собака"(гипероним)
   - "собака"(гипоним) – "млекопитающее"(гипероним)
   - "стул"(гипоним) – "мебель"(гипероним)

4. **Меронимия / Холонимия (Meronymy / Holonymy):** Отношение "часть-целое". Если A является меронимом B, то B является холонимом A.
   - **Мероним (Meronym):** Часть.
   - **Холоним (Holonym):** Целое.

   Примеры:
   - "колесо"(мероним) – "автомобиль"(холоним)
   - "рука"(мероним) – "тело"(холоним)
   - "глава"(мероним) – "книга"(холоним)

   Существуют разные типы меронимии (например, компонент-объект, член-коллекция, материал-объект).

5. **Тропонимия (Troponymy):** Отношение между глаголами, где один глагол описывает более специфичный способ выполнения действия, описываемого другим глаголом (его гиперонимом). *Примеры:*
   - "шептать"(тропоним) – "говорить"(гипероним)
   - "хромать"(тропоним) – "идти"(гипероним)

6. **Причинность (Cause/Entailment):** Отношение, где одно действие или состояние вызывает другое или влечет за собой другое. *Примеры (для глаголов в WordNet):*
   - "убить"causes "умереть"
   - "храпеть"entails "спать"(если кто-то храпит, он спит)

Лингвистические онтологии, создаваемые с использованием формальных языков представления знаний (например, Web Ontology Language - OWL, RDF Schema - RDFS), используют стандартизированные конструкции для описания классов, свойств и отношений. Эти конструкции позволяют точно и недвусмысленно (для машинной обработки) представлять семантические связи, включая аналоги лексико-семантических отношений.

- rdfs:subClassOf **(Таксономическое отношение):** Определяет, что один класс является подклассом другого. Это основной способ построения иерархии классов (таксономии). Это формальный аналог отношения гиперонимии на уровне классов понятий. *Пример:* Класс Собака rdfs:subClassOf класс Млекопитающее.
- rdf:type **(Отношение инстанцирования/принадлежности классу):** Связывает конкретный экземпляр (индивида, объекта) с его классом. *Пример:* Экземпляр Шарик rdf:type класс Собака.
- owl:ObjectProperty **и** owl:DatatypeProperty **(Свойства):** Это способы определения пользовательских типов отношений (свойств).
  - owl:ObjectProperty**:** Свойство, связывающее два экземпляра (индивида) из некоторых классов. Может использоваться для формализации отношений, таких как меронимия (партономия). *Пример:* Свойство имеет_часть может быть определено как owl:ObjectProperty. Тогда можно утверждать: экземпляр МойАвтомобиль имеет_часть экземпляр ЕгоДвигатель.

  - owl:DatatypeProperty**:** Свойство, связывающее экземпляр со значением простого типа данных (например, строка, число, дата). *Пример:* Свойство имеет_дату_рождения может быть определено как owl:DatatypeProperty. Тогда: экземпляр ИванИванов имеет_дату_рождения "1990-01-15"^^xsd:date.

  Для свойств часто указываются:
  - rdfs:domain**:** Класс, экземпляры которого могут быть субъектом данного свойства (от кого исходит свойство). *Пример для* имеет_часть*:* rdfs:domain Автомобиль (т.е. свойством имеет_часть обладают автомобили).
  - rdfs:range**:** Класс (для owl:ObjectProperty) или тип данных (для owl:DatatypeProperty), экземпляры-/значения которого могут быть объектом данного свойства (на что указывает свойство). *Пример для* имеет_часть*:* rdfs:range Компонент_автомобиля (т.е. имеет_часть указывает на компоненты автомобиля).

  Таким образом, отношение **партономии** (частеречное отношение, как меронимия) может быть формализовано через owl:ObjectProperty с соответствующими доменом и диапазоном.
- owl:equivalentClass**,** owl:disjointWith**:** Логические связи между классами.
  - owl:equivalentClass: Указывает, что два класса эквивалентны (описывают одно и то же множество экземпляров). Может использоваться для формализации синонимии на уровне концептов.
  - owl:disjointWith: Указывает, что два класса не могут иметь общих экземпляров (например, класс Человек owl:disjointWith класс Кошка).
- **Характеристики свойств:** Например, owl:TransitiveProperty (транзитивность: если А связано с В свойством Р, и В связано с С свойством Р, то А связано с С свойством Р. Пример: "быть предком "быть частью компонента"). owl:SymmetricProperty (если А связано с Б, то Б связано с А, например "быть коллегой").
- **Ограничения на свойства (Property Restrictions):** Позволяют более точно описывать классы, например, через ограничения кардинальности (сколько значений может иметь свойство), ограничения на значения свойств. *Пример:* "Каждый Человек должен иметь ровно одного (owl:cardinality 1) Биологического_отца, который является экземпляром класса Мужчина".

10 Продукционная модель представления знаний: структура и цикл работы продукционной системы. Нечеткие знания и их обработка в продукционных системах.

Определение: Продукционная система (или система, основанная на правилах) — это система, которая использует набор правил (продукций) для представления знаний и механизм вывода для манипулирования этими знаниями с целью решения задач.

Основные компоненты продукционной системы:

1. **База знаний (Knowledge Base) / Память правил (Rule Memory):**
   - Содержит набор **продукционных правил (продукций)**.
   - Каждое правило имеет вид: **ЕСЛИ <условие> ТО <действие>**.
     - **<Условие> (Антецедент, левая часть):** Логическое выражение, которое описывает ситуацию или набор фактов, при истинности которых правило может быть активировано. Условие обычно представляет собой конъюнкцию (логическое "И") элементарных условий (шаблонов, фактов). *Пример условия:* "Температура пациента > 38"И "У пациента есть кашель".
     - **<Действие> (Консеквент, правая часть):** Операция или набор операций, которые должны быть выполнены, если условие правила истинно. Действия могут:
       * Добавлять новые факты в рабочую память.
       * Модифицировать существующие факты.
       * Удалять факты.
       * Выполнять внешние процедуры (например, запросить информацию у пользователя, взаимодействовать с другими системами).
       * Управлять процессом вывода (например, остановить вывод, изменить цель).

       Пример действия: "Предположить диагноз: Простуда"ИЛИ "Рекомендовать: Измерить давление".
   - Правила в базе знаний обычно независимы друг от друга и могут быть добавлены, удалены или изменены без существенного влияния на другие правила (модульность).

2. **Рабочая память (Working Memory) / База фактов (Fact Base / Data Memory):**
   - Содержит текущее состояние решаемой задачи в виде набора **фактов**.
   - Факты — это утверждения о предметной области, которые считаются истинными на данный момент. Они могут быть:
     - Исходными данными, введенными пользователем.
     - Промежуточными результатами, полученными в ходе работы системы.
     - Целями, которые система пытается достичь.
   - Рабочая память является динамической: ее содержимое изменяется в результате срабатывания правил. *Примеры фактов:* "Температура пациента = 38.5 "У пациента есть кашель "Диагноз = Грипп".

3. **Механизм вывода (Inference Engine) / Интерпретатор правил:**
   - Это "мозг"продукционной системы, который управляет процессом применения правил к фактам.
   - Он выполняет **цикл "Распознавание-Действие"** или **цикл "Сопоставление-Выбор-Выполнение"**.

2. Цикл работы продукционной системы (Цикл "Распознавание-Действие")

Механизм вывода работает в циклическом режиме:

1. **Сопоставление (Match / Распознавание):**
   - Механизм вывода сравнивает условия (левые части) всех правил из базы знаний с текущими фактами в рабочей памяти.
   - Определяется подмножество правил, условия которых удовлетворяются (инстанцируются) текущим состоянием рабочей памяти. Эти правила называются **активированными** или **срабатывающими (triggered rules)**.
   - Множество активированных правил вместе с конкретными фактами, которые их активировали, называется **конфликтным множеством (conflict set)**.

2. **Разрешение конфликтов (Conflict Resolution / Выбор):**

- Если конфликтное множество содержит более одного правила (или одно правило может быть инстанцировано несколькими способами), необходимо выбрать одно правило (или одну инстанциацию) для выполнения.
- Стратегии разрешения конфликтов могут быть различными:
  - **Приоритет правил (Rule Priority):** Правилам заранее присваиваются приоритеты.
  - **Специфичность (Specificity):** Предпочтение отдается более специфичным правилам (с более сложными условиями).
  - **Свежесть данных (Recency):** Предпочтение правилам, которые оперируют с более "свежими"(недавно добавленными или измененными) фактами.
  - **Произвольный выбор (Arbitrary Choice):** Например, первое по порядку правило.

3. **Выполнение (Execute / Действие):**
   - Действие (правая часть) выбранного правила выполняется.
   - Это приводит к изменению рабочей памяти (добавление, удаление, модификация фактов) или выполнению других операций.

4. **Повторение цикла:** После выполнения действия система возвращается к шагу 1 (Сопоставление) с обновленной рабочей памятью. Цикл продолжается до тех пор, пока:
   - Не будет достигнута цель.
   - Конфликтное множество не станет пустым (нет правил, которые могут сработать).
   - Не будет выполнено специальное действие "стоп".

Направления вывода в продукционных системах:

- **Прямой вывод:** Начинается с известных фактов и применяет правила для вывода новых фактов до тех пор, пока не будет достигнута цель или не останется правил для применения. Движение от условий к заключениям. Подходит для задач мониторинга, интерпретации данных, планирования.
- **Обратный вывод:** Начинается с гипотезы (цели) и пытается найти факты и правила, которые ее подтверждают. Если условие правила, ведущего к цели, неизвестно, оно становится новой подцелью. Движение от заключений (целей) к условиям. Подходит для задач диагностики, консультирования, поиска объяснений.

Достоинства продукционной модели ПЗ

- Простота создания и понимания правил продукций, выражающих самостоятельные фрагменты знаний;
- Удобство и простота пополнения и модификации базы знаний, т.е. лёгкость построения продукционной БЗ;
- Возможность изменения управляющей системы без изменения БЗ и БД;
- Естественная возможность асинхронного и параллельного выполнения продукций (применимых в текущий момент);
- Возможность программирования немонотонных выводов.

Недостатки продукционной модели ПЗ

- Эвристический метод: слабый теоретический базис, не хватает строгой теории;
- Неэффективность вывода в продукционной системе: с увеличением числа продукций существенно замедляется скорость вывода;
- Сложность управления выводом в ПС: метаправила повышают гибкость, но ещё больше снижают эффективность;
- Сложность проверки непротиворечивости БЗ с ростом числа продукций (из-за их независимости). При уверенности в правильности любой продукции всё же априори нельзя быть уверенным в полноте и непротиворечивости модели ПЗ в виде ПС.

3. Нечеткие знания и их обработка в продукционных системах

Классические продукционные системы работают с четкими (булевыми) фактами и правилами: факт либо истинен, либо ложен; условие правила либо выполняется, либо нет. Однако многие знания в реальном мире являются нечеткими, неопределенными или неполными.

Нечеткие знания: Знания, которые содержат нечеткие понятия, лингвистические переменные или неопределенные утверждения. *Примеры:*

- "Температура *высокая*"(насколько высокая?)
- "Пациент *молодой*"
- "Если давление *значительно повышено* И пульс *частый*, ТО риск *средний*"

Обработка нечетких знаний в продукционных системах: Основана на теории нечетких множеств (fuzzy sets) и нечеткой логики (fuzzy logic).

1. **Представление нечетких фактов:** Факты представляются с использованием **лингвистических переменных** и их **нечетких значений (термов)**, которые описываются **функциями принадлежности**.
   - **Лингвистическая переменная:** Переменная, значениями которой являются слова или фразы естественного языка (например, "Температура "Возраст "Скорость").
   - **Нечеткий терм:** Нечеткое значение лингвистической переменной (например, для "Температура": "низкая "нормальная "высокая").
   - **Функция принадлежности $\mu_A(x)$:** Определяет степень принадлежности элемента x нечеткому множеству A (терму). Значение $\mu_A(x)$ находится в диапазоне [0, 1]. *Пример факта:* "Температура есть высокая с уверенностью 0.8"$\rightarrow \mu_{\text{высокая}}(\text{текущая_температура}) = 0.8$.

2. **Представление нечетких правил:** Правила имеют тот же вид "ЕСЛИ <условие> ТО <действие> но условия и/или действия могут содержать нечеткие термы. *Пример правила:* ЕСЛИ Температура IS Высокая И Давление IS Повышенное ТО Риск IS Средний.

3. **Нечеткий логический вывод:** Процесс получения заключений из нечетких фактов и нечетких правил. Включает этапы:
   - **Фаззификация:** Преобразование четких входных значений (например, конкретное значение температуры) в степени принадлежности нечетким термам, используемым в условиях правил.
   - **Агрегирование условий:** Если условие правила состоит из нескольких подусловий, соединенных логическими связками (И, ИЛИ), вычисляется общая степень истинности условия. Для "И"обычно используется минимум (min) или произведение (prod) степеней истинности подусловий. Для "ИЛИ"— максимум (max) или вероятностная сумма.
   - **Активация (Импликация):** Степень истинности условия правила используется для определения степени истинности заключения (действия) этого правила. Распространенные методы: min-импликация, prod-импликация. Результатом является нечеткое множество для выходной переменной, "срезанное"или "масштабированное"на уровне истинности условия.
   - **Аккумуляция заключений:** Если несколько правил приводят к заключениям относительно одной и той же выходной переменной, их нечеткие множества объединяются (обычно с помощью операции max) для получения итогового нечеткого множества для этой переменной.
   - **Дефаззификация:** Преобразование итогового нечеткого множества для выходной переменной в четкое (числовое) значение, которое может быть использовано для принятия решений или управления. Распространенные методы: центр максимумов (Находят точки, где функция принадлежности максимальна, и берут их центр или среднее).

Преимущества нечетких продукционных систем:

- Способность моделировать знания, выраженные на естественном языке с использованием нечетких понятий.
- Устойчивость к неточным или неполным входным данным.
- Возможность построения более "гладких"и интуитивно понятных моделей управления и принятия решений.

Применение: Системы управления (например, управление климатом, бытовая техника), экспертные системы, системы поддержки принятия решений, распознавание образов.

Механизм неточного вывода в ПС

Простейший механизм неточного вывода присоединенного характера для ПС с правилами продукции вида $r : p \to q$ получается путем введения функций для пересчета:

Меры достоверности $t(p)$ левой части правила по мерам достоверности составляющих его частей:

$$t(p) = f_p(t(e_1), t(e_2\ \ ), \ldots, t(e_n)).$$

Для вычисления меры достоверности $t(p)$ левой части правила могут быть использованы операции нечеткой логики (их суперпозиции):

- $t(e_1 \& e_2) = min\{t(e_1), t(e_2)\}$
- $t(e_1 \vee e_2) = max\{t(e_1), t(e_2)\}$
- $t(\neg e) = 1 - t(e)$

Меры достоверности правой части (заключения правила) по мерам достоверности левой части и самого правила:

$$t(q) = f_r(t(r), t(p))$$

где $t(e_k)$ — мера достоверности утверждения/факта e_kв РП, $t(r)$ — мера достоверности правила r (достоверности заключения при истинности посылок).

Например, $t(q) = min\{t(r), t(p)\}$.

Рис. 1: Изображение 8

11 Методы поиска решений в пространстве состояний. Метод ветвей и границ (Branch and Bound), лучевой поиск (Beam Search), A*-алгоритм и его частные случаи.

Многие задачи в искусственном интеллекте, исследовании операций и других областях могут быть сформулированы как задачи поиска решения в пространстве состояний. Пространство состояний представляет собой граф, где узлы — это состояния задачи, а дуги — это операторы (действия), переводящие из одного состояния в другое. Цель — найти путь от начального состояния к одному из целевых состояний.

1. Поиск в пространстве состояний

Основные компоненты задачи поиска:

- **Пространство состояний (State Space):** Множество всех возможных конфигураций или состояний, в которых может находиться решаемая задача.
- **Начальное состояние (Initial State):** Состояние, с которого начинается поиск.
- **Операторы (Actions/Operators):** Правила или действия, которые переводят систему из одного состояния в другое. Применение оператора к состоянию порождает новое состояние.
- **Целевое условие (Goal Test):** Условие, которое определяет, является ли данное состояние целевым (решением). Может быть несколько целевых состояний.
- **Стоимость пути (Path Cost):** Функция, присваивающая числовую стоимость пути. Обычно это сумма стоимостей операторов вдоль пути. В простейшем случае каждый шаг имеет стоимость 1.

Цель поиска — найти путь от начального состояния к целевому. Если есть стоимости, то часто ищется путь с минимальной стоимостью (оптимальное решение).

Общая стратегия поиска: Большинство алгоритмов поиска работают по схожему принципу:

1. Начать с начального состояния.
2. Поддерживать множество еще не исследованных состояний (часто называемое *границей*, *фронтом* или *открытым списком*).
3. Итеративно выбирать состояние из границы для раскрытия.
4. Раскрытие состояния: применение всех возможных операторов к нему, генерация дочерних состояний.
5. Проверка дочерних состояний: если состояние целевое — успех; иначе, если оно новое или путь к нему лучше, добавить его в границу.
6. Если граница пуста и цель не найдена — неудача.

Различные алгоритмы поиска отличаются стратегией выбора следующего состояния из границы.

2. Метод ветвей и границ (Branch and Bound, B&B)

Определение: Метод ветвей и границ — это общий алгоритмический метод для нахождения оптимальных решений в задачах дискретной и комбинаторной оптимизации, а также в задачах математического программирования. Он систематически исследует пространство решений, отсекая (pruning) те ветви дерева поиска, которые заведомо не могут привести к оптимальному решению.

Принцип работы:

1. **Ветвление (Branching):** Пространство решений рекурсивно разделяется на меньшие подмножества (подзадачи). Каждая подзадача соответствует узлу в дереве поиска.

2. **Оценка границ (Bounding):** Для каждой подзадачи (узла дерева) вычисляется оценка (нижняя граница для задач минимизации или верхняя граница для задач максимизации) значения целевой функции для всех решений, которые могут быть получены из этой подзадачи.

3. **Отсечение (Pruning):**
   - Если оценка для текущей подзадачи хуже (например, нижняя граница больше), чем уже найденное лучшее решение (текущий рекорд), то эта ветвь отсекается, так как она не может привести к улучшению рекорда.
   - Если решение найдено в текущей подзадаче, и оно лучше текущего рекорда, рекорд обновляется.
   - Если подзадачу нельзя разделить дальше (например, достигнут лист дерева), и она представляет допустимое решение, оно сравнивается с рекордом.

4. **Выбор узла для ветвления:** Стратегия выбора следующего узла из активных (не отсеченных и не полностью исследованных) для дальнейшего ветвления. Распространенные стратегии:
   - Поиск в глубину (DFS-like).
   - Поиск по наилучшей оценке (Best-First Search): выбирается узел с наилучшей (самой перспективной) оценкой границы.

Метод завершается, когда все узлы либо исследованы, либо отсечены. Текущий рекорд на момент завершения является оптимальным решением.

Ключевые компоненты B&B:

- **Правило ветвления:** Как разделять задачу на подзадачи.
- **Функция оценки (Bounding Function):** Как вычислять оценки для подзадач. Качество оценки критично: чем точнее оценка (ближе к реальному оптимуму подзадачи), тем эффективнее отсечение.
- **Стратегия обхода дерева поиска.**

Применение: Задача коммивояжера, задачи целочисленного программирования, задача о назначениях, планирование.

3. Лучевой поиск (Beam Search)

Определение: Лучевой поиск — это эвристический алгоритм поиска, который исследует граф, расширяя наиболее перспективные узлы в ограниченном наборе. Это оптимизация поиска по первому наилучшему совпадению (best-first search), которая уменьшает требования к памяти.

Принцип работы:

1. На каждом шаге (уровне дерева поиска) генерируются все возможные преемники (дочерние узлы) для узлов предыдущего шага.

2. Эти преемники оцениваются с помощью эвристической функции.

3. Выбирается только **B** наилучших преемников (где **B** — это **ширина луча**, заранее заданный параметр). Остальные преемники отбрасываются.

4. Процесс повторяется для выбранных **B** узлов.

Особенности:

- **Неполный поиск:** Лучевой поиск не гарантирует нахождения оптимального решения (или вообще какого-либо решения), так как перспективные ветви могут быть отброшены, если они временно не попадают в число **B** лучших.

- **Компромисс между качеством и ресурсами:**
  - Малая ширина луча (**B**): экономит память и время, но повышает риск потери оптимального решения. При **B=1** лучевой поиск вырождается в "жадный"поиск по первому наилучшему совпадению (greedy best-first search).
  - Большая ширина луча (**B**): увеличивает шансы найти хорошее решение, но приближается по затратам к поиску по первому наилучшему совпадению.
- **Эвристическая функция:** Качество используемой эвристики сильно влияет на результат. Эвристическая функция (или просто эвристика) — это способ оценить, насколько близко текущее состояние к цели, и насколько оно перспективное для продолжения.

Применение: Машинный перевод (поиск наилучшей последовательности слов), распознавание речи, задачи планирования, где пространство поиска очень велико.

4. Алгоритм A* (A-star)

Определение: A* — это алгоритм поиска по первому наилучшему совпадению, который находит путь с наименьшей стоимостью от начальной вершины к целевой. Он использует эвристическую функцию для оценки стоимости пути до цели.

Принцип работы: A* поддерживает приоритетную очередь узлов для раскрытия. Приоритет узла n определяется функцией оценки $f(n)$: $f(n) = g(n) + h(n)$ где:

- $g(n)$ — реальная стоимость пути от начального узла до узла n.
- $h(n)$ — эвристическая оценка стоимости пути от узла n до ближайшего целевого узла.

Алгоритм всегда выбирает для раскрытия узел с наименьшим значением $f(n)$.

Свойства A*:

- **Оптимальность:** A* гарантированно находит путь с наименьшей стоимостью, если эвристическая функция $h(n)$ является **допустимой (admissible)** (или **оптимистичной**), то есть $h(n)$ никогда не переоценивает реальную стоимость достижения цели из n. Формально, $0 \leq h(n) \leq h^*(n)$, где $h^*(n)$ — истинная минимальная стоимость пути от n до цели.
- **Полнота:** A* является полным (всегда находит решение, если оно существует) при условии, что стоимости ребер положительны ($c > 0$) и граф конечен или локально конечен (каждый узел имеет конечное число преемников).
- **Оптимальная эффективность:** Среди всех оптимальных алгоритмов поиска, использующих ту же эвристику, A* раскрывает наименьшее количество узлов (при условии **согласованности** эвристики).

Согласованность (Consistency / Monotonicity) эвристики: Эвристика $h(n)$ называется согласованной, если для любого узла n и любого его преемника n', порожденного действием a, выполняется условие треугольника: $h(n) \leq \text{cost}(n, a, n') + h(n')$ где $\text{cost}(n, a, n')$ — стоимость действия a из n в n'. Если эвристика согласована, то она автоматически является и допустимой. Согласованность также гарантирует, что когда A* выбирает узел n для раскрытия, он уже нашел оптимальный путь к n (т.е. $g(n)$ является минимальной стоимостью).

Алгоритм A* (шаги):

1. Инициализировать открытый список (приоритетная очередь) начальным узлом. $g(\text{start}) = 0$, $f(\text{start}) = h(\text{start})$.
2. Инициализировать закрытый список (посещенные узлы) как пустой.
3. Пока открытый список не пуст:
   (a) Извлечь узел n с наименьшим значением $f(n)$ из открытого списка.
   (b) Если n — целевой узел, то путь найден. Восстановить путь и завершить.
   (c) Переместить n из открытого списка в закрытый.
   (d) Для каждого преемника m узла n:
      i. Вычислить $g(m) = g(n) + \text{cost}(n, \text{action}, m)$.
      ii. Если m находится в закрытом списке и новый путь к m не лучше, пропустить m.
      iii. Если m находится в открытом списке и новый путь к m не лучше, пропустить m.

iv. Иначе (новый узел или найден лучший путь):

- Установить $g(m)$ и родителя для m.
- Вычислить $h(m)$ и $f(m) = g(m) + h(m)$.
- Добавить m в открытый список (или обновить его приоритет, если он там уже был с худшим $f(m)$).

4. Если открытый список пуст и цель не найдена, решения нет.

Частные случаи A*

- **Алгоритм Дейкстры (Dijkstra's Algorithm):** Если $h(n) = 0$ для всех узлов n. В этом случае $f(n) = g(n)$, и A* выбирает узел с наименьшей стоимостью пути от начального узла. Алгоритм Дейкстры находит кратчайшие пути от одной вершины до всех остальных в графе с неотрицательными весами ребер. Он оптимален и полон.
- **Поиск в ширину (Breadth-First Search, BFS):** Если $h(n) = 0$ и стоимость всех операторов (ребер) равна 1 ($g(n)$ — это глубина узла). BFS находит путь с наименьшим количеством шагов. Он оптимален (по числу шагов) и полон. BFS использует очередь (FIFO). Это значит: Сначала в очередь кладутся соседи старта → они будут раскрыты первыми Потом соседи этих соседей → раскроются во вторую волну И так далее
- **Жадный поиск по первому наилучшему совпадению (Greedy Best-First Search):** Если $g(n) = 0$ (или игнорируется), то $f(n) = h(n)$. Алгоритм выбирает узел, который эвристически кажется наиболее близким к цели, не учитывая уже пройденный путь. Этот поиск не является ни оптимальным, ни полным в общем случае, но часто находит решение быстро.

A* является одним из наиболее широко используемых и эффективных алгоритмов поиска благодаря своей оптимальности и способности использовать эвристическую информацию для направления поиска. Выбор хорошей допустимой эвристики критичен для его производительности.

12 Методы слепого поиска в пространствах состояний. Алгоритм восхождения к вершине (Hill Climbing). Метод Монте-Карло, эволюционные алгоритмы.

Методы поиска решений в пространстве состояний можно разделить на информированные (использующие эвристическую информацию о близости к цели, как A*) и слепые (неинформированные). Слепые методы поиска используют только информацию, доступную в определении задачи (начальное состояние, операторы, целевое условие), без дополнительных знаний о предметной области. Также существуют методы локального поиска и стохастические методы, которые исследуют пространство состояний специфическим образом.

1. Методы слепого поиска (Неинформированный поиск)

Слепые методы поиска систематически исследуют пространство состояний, не используя никакой информации о том, насколько "близко"текущее состояние к целевому. Они отличаются только порядком, в котором узлы раскрываются.

а) Поиск в ширину (Breadth-First Search, BFS)

- **Стратегия:** Раскрывает узлы уровень за уровнем. Сначала все узлы на глубине d раскрываются перед тем, как раскрывать узлы на глубине $d + 1$. Использует очередь FIFO (First-In, First-Out) для хранения узлов на границе.
- **Полнота:** Да (если фактор ветвления конечен).
- **Оптимальность:** Да, если стоимость пути — это число шагов (все операторы имеют одинаковую стоимость). Находит решение с наименьшей глубиной.
- **Временная сложность:** $\mathcal{O}(b^d)$, где b — фактор ветвления, d — глубина решения.
- **Пространственная сложность:** $\mathcal{O}(b^d)$ (хранит все узлы на границе).
- **Недостаток:** Требует много памяти для больших пространств состояний.

б) Поиск с равномерной стоимостью (Uniform-Cost Search, UCS)

- **Стратегия:** Раскрывает узел n с наименьшей стоимостью пути $g(n)$ от начального узла. Использует приоритетную очередь. Является обобщением BFS для случая разных стоимостей операторов. (Эквивалентен A* с $h(n) = 0$).
- **Полнота:** Да (если стоимости операторов $\geq \epsilon > 0$).
- **Оптимальность:** Да (находит путь с наименьшей стоимостью).
- **Временная сложность:** $\mathcal{O}(b^{1+\lfloor C^*/\epsilon \rfloor})$, где C^* — стоимость оптимального решения, ϵ — минимальная стоимость оператора.
- **Пространственная сложность:** Аналогична временной.
- **Недостаток:** Может исследовать "дорогие"пути, если они кажутся короткими по числу шагов, не учитывая эвристику.

в) Поиск в глубину (Depth-First Search, DFS)

- **Стратегия:** Раскрывает самый "глубокий"узел в дереве поиска. Идет по одной ветви до конца, затем возвращается (бэктрекинг). Использует стек LIFO (Last-In, First-Out) для хранения узлов на границе.
- **Полнота:** Нет, если есть бесконечные пути (может зациклиться). Полон для конечных графов без циклов или при использовании списка посещенных узлов.
- **Оптимальность:** Нет (может найти неоптимальное решение, которое глубже, но достигнуто раньше).
- **Временная сложность:** $\mathcal{O}(b^m)$, где m — максимальная глубина дерева поиска (может быть намного больше d).
- **Пространственная сложность:** $\mathcal{O}(b \cdot m)$ (хранит только узлы на текущем пути и их нераскрытых соседей). Это главное преимущество DFS.

г) Поиск с ограничением глубины (Depth-Limited Search, DLS)

- **Стратегия:** DFS с заранее заданным пределом глубины L. Узлы на глубине L рассматриваются как не имеющие преемников.
- **Полнота:** Нет, если глубина решения $d > L$.
- **Оптимальность:** Нет.

- **Временная сложность:** $\mathcal{O}(b^L)$.
- **Пространственная сложность:** $\mathcal{O}(b \cdot L)$.

д) Итеративное углубление поиска в глубину (Iterative Deepening DFS, IDDFS)

- **Стратегия:** Комбинирует преимущества BFS и DFS. Выполняет DLS для предела глубины $L = 0, 1, 2, \ldots$ до тех пор, пока не будет найдено решение.
- **Полнота:** Да.
- **Оптимальность:** Да (если стоимость пути — это число шагов, как BFS).
- **Временная сложность:** $\mathcal{O}(b^d)$ (удивительно, но ненамного хуже BFS, так как большинство узлов находятся на последнем уровне).
- **Пространственная сложность:** $\mathcal{O}(b \cdot d)$ (как DFS).
- Считается предпочтительным слепым методом, когда пространство состояний велико и глубина решения неизвестна.

2. Алгоритм восхождения к вершине (Hill Climbing)

Определение: Алгоритм восхождения к вершине (или "градиентного подъема") — это метод локального поиска, который итеративно движется в направлении увеличения значения целевой (оценочной) функции. Он пытается найти максимум (или минимум) этой функции.

Принцип работы:

1. Начать с некоторого случайного или заданного начального состояния (текущего решения).
2. На каждой итерации рассмотреть соседние состояния (полученные применением небольших изменений к текущему состоянию).
3. Выбрать из соседей то состояние, которое имеет наилучшее значение целевой функции и лучше текущего состояния.
4. Перейти в это новое состояние (оно становится текущим).
5. Повторять шаги 2-4 до тех пор, пока не будет найдено состояние, у которого все соседи имеют худшее или такое же значение целевой функции. Это состояние объявляется локальным оптимумом.

Особенности и проблемы:

- **Локальные оптимумы:** Алгоритм может застрять в локальном максимуме, который не является глобальным максимумом.
- **Плато (Plateaus):** Области пространства состояний, где целевая функция имеет одинаковое значение для многих состояний. Алгоритм может бесцельно блуждать по плато.
- **Гребни (Ridges):** Области, где движение в любом одном направлении приводит к ухудшению, но последовательность шагов могла бы привести к улучшению.
- **Зависимость от начальной точки:** Результат сильно зависит от выбора начального состояния.
- **Жадность:** Принимает локально оптимальные решения, не заглядывая вперед.
- **Простота и эффективность:** Очень прост в реализации и требует мало памяти (хранит только текущее состояние).

Варианты алгоритма восхождения к вершине:

- **Простейшее восхождение (Simple Hill Climbing):** Выбирает первого соседа, который лучше текущего.
- **Наискорейшее восхождение (Steepest-Ascent Hill Climbing):** Оценивает всех соседей и выбирает наилучшего.
- **Стохастическое восхождение (Stochastic Hill Climbing):** Выбирает соседа случайным образом, с вероятностью, зависящей от улучшения целевой функции. Может иногда выбирать шаги "вниз"для преодоления локальных оптимумов.
- **Восхождение с рестартами (Random-Restart Hill Climbing):** Многократно запускает алгоритм из разных случайных начальных точек и выбирает лучший из найденных локальных оптимумов. Повышает шансы найти глобальный оптимум.

3. Метод Монте-Карло (Monte Carlo Method)

Определение: Метод Монте-Карло — это широкий класс вычислительных алгоритмов, которые полагаются на повторную случайную выборку для получения численных результатов. Они часто используются, когда детерминированное решение задачи невозможно или слишком трудоемко.

Принцип работы (в контексте поиска/оптимизации):

1. **Генерация случайных выборок:** Случайным образом генерируются точки (решения) из пространства поиска.

2. **Оценка выборок:** Каждая сгенерированная точка оценивается с помощью целевой функции.

3. **Агрегация результатов:** Результаты оценок используются для получения приближенного решения. Например, если ищется оптимум, то выбирается лучшая из всех сгенерированных точек. Если оценивается интеграл, то усредняются значения функции.

В простейшем варианте для оптимизации, метод Монте-Карло заключается в генерации большого числа случайных решений и выборе наилучшего из них.

Особенности:

- **Вероятностный характер:** Результаты являются случайными и могут варьироваться от запуска к запуску. Качество решения обычно улучшается с увеличением числа выборок.
- **Простота реализации:** Легко реализовать для многих задач.
- **Применимость к сложным пространствам:** Может применяться к задачам с очень большим или даже непрерывным пространством состояний, где систематический перебор невозможен.
- **Не использует градиент или информацию о структуре пространства поиска** (в своих базовых формах).
- **Конвергенция:** Скорость сходимости к точному решению часто медленная (порядка $1/\sqrt{N}$, где N — число выборок).

Применение в поиске:

- **Поиск в деревьях методом Монте-Карло (Monte Carlo Tree Search, MCTS):** Популярный эвристический алгоритм поиска для игр и задач принятия решений. Он строит дерево поиска, используя случайные симуляции (проигрыши) для оценки перспективности узлов.
- **Глобальная оптимизация:** Случайный поиск для нахождения приближенного глобального оптимума.

4. Эволюционные алгоритмы (Evolutionary Algorithms, EA)

Определение: Эволюционные алгоритмы — это методы оптимизации, которые ищут решения с помощью имитации процесса эволюции: они работают с популяцией возможных решений, оценивают их «качество» и постепенно улучшают их через отбор, скрещивание и мутацию, чтобы найти наилучшее.

Общая схема эволюционного алгоритма:

1. **Инициализация:** Создается начальная *популяция* индивидуумов (потенциальных решений). Каждый индивидуум обычно представляется в виде некоторой структуры данных (например, битовой строки, вектора чисел, дерева).

2. **Оценка (Fitness Evaluation):** Для каждого индивидуума в популяции вычисляется значение *функции приспособленности* (fitness function), которая оценивает, насколько хорошо данный индивидуум решает задачу.

3. **Цикл эволюции (пока не выполнено условие остановки):**

   (a) **Отбор (Selection):** Индивидуумы выбираются из текущей популяции для участия в размножении. Более приспособленные индивидуумы имеют больше шансов быть отобранными (например, рулеточный отбор, турнирный отбор).

   (b) **Генетические операторы (Variation Operators):** К отобранным индивидуумам (родителям) применяются генетические операторы для создания новых индивидуумов (потомков):

   - **Скрещивание (Crossover / Recombination):** Комбинирование генетического материала двух или более родителей для создания одного или нескольких потомков.
   - **Мутация (Mutation):** Внесение небольших случайных изменений.

(c) **Формирование новой популяции:** Потомки (и, возможно, часть родителей из предыдущей популяции) формируют новую популяцию для следующего поколения (например, замена всех родителей, элитизм — сохранение лучших индивидуумов).

(d) **Оценка новой популяции.**

4. **Результат:** Лучший индивидуум, найденный за все время работы алгоритма, считается решением.

Основные виды эволюционных алгоритмов:

- **Генетические алгоритмы (Genetic Algorithms, GA):** Обычно работают с битовыми строками или другими дискретными представлениями. Активно используют скрещивание и мутацию.
- **Эволюционное программирование (Evolutionary Programming, EP):** Фокусируется на мутации как основном операторе изменения. Представление индивидуумов часто связано с конечными автоматами или действительными числами.
- **Эволюционные стратегии (Evolution Strategies, ES):** Обычно работают с векторами действительных чисел. Используют специфические операторы мутации (например, с самоадаптацией параметров мутации) и отбора.
- **Генетическое программирование (Genetic Programming, GP):** Эволюционируют программы, обычно представленные в виде деревьев выражений или синтаксических деревьев.

Особенности эволюционных алгоритмов:

- **Популяционный подход:** Работают с набором решений одновременно, что позволяет исследовать разные области пространства поиска.
- **Стохастичность:** Содержат случайные компоненты (инициализация, отбор, генетические операторы).
- **Глобальный поиск:** Способны находить глобальные или близкие к глобальным оптимумы, преодолевая локальные экстремумы.
- **Применимость к "черным ящикам":** Требуют только возможности оценить качество решения (функцию приспособленности), не обязательно знания о градиентах или структуре задачи.
- **Вычислительная сложность:** Могут быть вычислительно затратными, особенно для больших популяций и многих поколений.
- **Настройка параметров:** Эффективность сильно зависит от выбора параметров алгоритма (размер популяции, вероятности мутации и скрещивания, стратегия отбора и т.д.).

Что можно эволюционировать в нейросетях

1. **Архитектуру (topology)**
   Эволюционный алгоритм может изменять структуру нейросети: количество слоёв, количество нейронов в слоях, и схему соединений.
   Пример: сеть с 3 слоями и 16 нейронами может превратиться в сеть с 4 слоями и 20 нейронами.

2. **Веса сети**
   Вместо обучения с помощью обратного распространения ошибки (backpropagation), веса могут эволюционировать через мутации и скрещивание. Это особенно полезно, если градиент трудно вычислить или модель обучается в симуляции (например, игра или физическая среда).

Зачем вообще нужны эволюционные алгоритмы (ЭА)?

Эволюционные алгоритмы применяются там, где другие методы не работают, потому что:

- **Функция сложная:** может быть разрывной, нелинейной, шумной, с множеством локальных экстремумов.
- **Нет производной:** не удаётся применить градиентные методы, так как нет аналитического выражения или оно слишком сложное.
- **Нет формулы:** задача задана в виде чёрного ящика (например, симуляция), и можно только наблюдать выход по входу.
- **Пространство решений большое и неперебираемое:** множество параметров, большое количество допустимых конфигураций.
- **Комбинаторный характер задачи:** нельзя построить граф состояний или аналитически вычислить лучший путь.

13 Методы поиска решений для игр с полной информацией: игровые деревья, минимаксный алгоритм, альфа-бета-отсечение.

Игры с полной информацией, такие как шахматы, шашки, го или крестики-нолики, представляют собой класс задач, где все аспекты текущего состояния игры известны всем игрокам. Поиск решений в таких играх часто сводится к анализу возможных ходов и их последствий для определения оптимальной стратегии.

1. Игровые деревья (Game Trees)

Определение: Игровое дерево — это формальное представление игры, где узлы дерева соответствуют состояниям (позициям) игры, а дуги (ребра) — возможным ходам игроков.

- **Корень дерева:** Начальное состояние игры.
- **Узлы (Nodes):** Представляют собой состояния игры (например, расстановка фигур на доске).
- **Дуги (Edges):** Представляют собой возможные ходы, которые переводят игру из одного состояния в другое.
- **Листья (Terminal Nodes):** Представляют собой конечные состояния игры, где игра завершена (например, выигрыш, проигрыш, ничья). Каждому листовому узлу присваивается числовое значение (оценка полезности, utility value) с точки зрения одного из игроков (обычно первого, MAX-игрока).
- **Уровни дерева:** Часто соответствуют ходам разных игроков. Если игра для двух игроков, то уровни чередуются между ходами MAX-игрока (который пытается максимизировать свою оценку) и MIN-игрока (который пытается минимизировать оценку MAX-игрока, т.е. максимизировать свою собственную).

Пример: В крестиках-ноликах корень — пустая доска. Первый уровень — все возможные первые ходы крестиком. Второй уровень — все возможные ответы ноликом на каждый из этих ходов, и т.д. Листья — это состояния, где один из игроков выиграл или все клетки заполнены (ничья).

2. Минимаксный алгоритм (Minimax Algorithm)

Определение: Минимаксный алгоритм — это рекурсивный алгоритм принятия решений, используемый в играх с полной информацией для двух игроков с нулевой суммой (выигрыш одного игрока равен проигрышу другого). Алгоритм определяет оптимальный ход для текущего игрока, предполагая, что оба игрока играют оптимально.

Принцип работы: Алгоритм строит (или обходит) игровое дерево до определенной глубины или до терминальных узлов.

1. **Оценка терминальных узлов:** Каждому конечному состоянию игры (листу дерева) присваивается оценка полезности $U(s)$ с точки зрения MAX-игрока.
   - Высокое положительное значение: выигрыш MAX.
   - Низкое отрицательное значение: выигрыш MIN (проигрыш MAX).
   - Нулевое значение: ничья.

2. **Распространение оценок вверх по дереву:**
   - **Для MAX-узлов (ходы MAX-игрока):** Оценка узла равна *максимуму* оценок его дочерних узлов. MAX-игрок выбирает ход, ведущий к состоянию с максимальной для него оценкой. $\text{MinimaxValue}(s_{\text{MAX}})$ $\max_{s' \in \text{Successors}(s_{\text{MAX}})} \text{MinimaxValue}(s')$.
     Когда минимакс находится в позиции s_{MAX}, он вычисляет:
     $$\text{MinimaxValue}(s_{\text{MAX}}) = \max_{s' \in \text{Successors}(s_{\text{MAX}})} \text{MinimaxValue}(s')$$
     То есть:
     - рассматривает все возможные ходы $s' \in \text{Successors}(s_{\text{MAX}})$,
     - для каждого состояния s' рекурсивно вызывает минимакс,
     - получает оценки $\text{MinimaxValue}(s')$ для всех s',
     - выбирает максимальную из них.
   - **Для MIN-узлов (ходы MIN-игрока):** Оценка узла равна *минимуму* оценок его дочерних узлов. MIN-игрок выбирает ход, ведущий к состоянию с минимальной для MAX-игрока (и, следовательно, максимальной для себя) оценкой. $\text{MinimaxValue}(s_{\text{MIN}}) = \min_{s' \in \text{Successors}(s_{\text{MIN}})} \text{MinimaxValue}(s')$

3. **Выбор хода:** В корневом узле MAX-игрок выбирает ход, который ведет к дочернему узлу с вычисленной максимальной минимаксной оценкой.

Ограничение глубины и оценочная функция: Поскольку полное построение дерева для сложных игр невозможно, поиск обычно ограничивается некоторой глубиной d. Для нетерминальных узлов на этой глубине используется **эвристическая оценочная функция (heuristic evaluation function)**, $Eval(s)$, которая оценивает перспективность позиции с точки зрения MAX-игрока. Качество этой функции критично для силы игрока.

Простой пример эвристической функции в шахматах:

Один из самых популярных и простых вариантов — **материальный счёт**:

$$\text{Оценка}(s) = \text{ценность белых фигур} - \text{ценность чёрных фигур}$$

Свойства минимаксного алгоритма:

- **Оптимальность:** Если дерево игры конечно и полностью исследовано, минимакс находит оптимальную стратегию (при условии, что противник также играет оптимально).
- **Полнота:** Да, для конечных деревьев.
- **Временная сложность:** $\mathcal{O}(b^m)$, где b — фактор ветвления (среднее число ходов из позиции), m — максимальная глубина дерева (или глубина поиска). Это экспоненциальная сложность, что делает его неприменимым для глубокого анализа сложных игр.
- **Пространственная сложность:** $\mathcal{O}(b \cdot m)$ при использовании поиска в глубину для обхода дерева.
  - Поиск в глубину (DFS) идёт по одному пути вниз до конца (до глубины m).
  - В памяти одновременно хранятся только: текущий путь длиной m (одна ветвь дерева) и потомки текущего узла (до b вариантов).
  - Следовательно, максимальное использование памяти: m уровней по b вариантов.

3. Альфа-бета-отсечение (Alpha-Beta Pruning)

Определение: Альфа-бета-отсечение — это оптимизация минимаксного алгоритма, которая позволяет значительно сократить количество исследуемых узлов в игровом дереве без изменения результата (т.е. выбирается тот же оптимальный ход, что и полным минимаксом).

Принцип работы: Алгоритм отслеживает два значения во время обхода дерева:

- **Альфа (α):** Наилучшее (максимальное) значение, гарантированное для MAX-игрока на текущем пути до корня. Инициализируется $-\infty$.
- **Бета (β):** Наилучшее (минимальное) значение, гарантированное для MIN-игрока на текущем пути до корня (т.е. наихудшее для MAX). Инициализируется $+\infty$.

Эти значения передаются рекурсивно вниз по дереву.

Правила отсечения:

1. **Отсечение в MIN-узле (Бета-отсечение):** Если текущее значение v в MIN-узле становится *меньше или равно* α (значение, которое MAX уже может себе гарантировать на более высоком уровне), то дальнейшее исследование потомков этого MIN-узла можно прекратить. MIN-игрок никогда не выберет этот путь, так как MAX-игрок не позволит ему дойти до ситуации, где оценка будет $\leq \alpha$, если у MAX есть альтернатива с оценкой α. Формально: если $v \leq \alpha$, то отсечь. В MIN-узле значение β обновляется: $\beta = \min(\beta, v)$.

2. **Отсечение в MAX-узле (Альфа-отсечение):** Если текущее значение v в MAX-узле становится *больше или равно* β (значение, которое MIN уже может себе гарантировать на более высоком уровне), то дальнейшее исследование потомков этого MAX-узла можно прекратить. MAX-игрок никогда не выберет этот путь, так как MIN-игрок не позволит ему достичь оценки $\geq \beta$, если у MIN есть альтернатива с оценкой β. Формально: если $v \geq \beta$, то отсечь. В MAX-узле значение α обновляется: $\alpha = \max(\alpha, v)$.

Эффективность альфа-бета-отсечения:

- **Сохранение оптимальности:** Алгоритм гарантированно находит тот же оптимальный ход, что и минимакс.
- **Уменьшение временной сложности:** В лучшем случае (при идеальном упорядочивании ходов, когда лучшие ходы рассматриваются первыми) альфа-бета-отсечение может уменьшить эффективный фактор ветвления с b до $\sqrt{b}$. Это означает, что временная сложность может снизиться с $\mathcal{O}(b^m)$ до $\mathcal{O}((\sqrt{b})^m) = \mathcal{O}(b^{m/2})$. На практике это позволяет за то же время просматривать дерево примерно в два раза глубже.
- **Зависимость от порядка ходов:** Эффективность отсечения сильно зависит от порядка, в котором рассматриваются дочерние узлы. Использование хороших эвристик для упорядочивания ходов (например, сначала рассматривать ходы, которые кажутся наиболее перспективными) может значительно улучшить производительность.
- **Пространственная сложность:** Остается $\mathcal{O}(b \cdot m)$ при использовании поиска в глубину.

14 Байесовские сети как способ представления вероятностного пространства. Построение байесовских сетей, примеры их использования.

Определение: Байесовская сеть (БС) — это ориентированный ациклический граф (DAG), в котором:

- **Узлы (Nodes)** представляют случайные переменные (дискретные или непрерывные).
- **Дуги (Directed Edges)** представляют прямые вероятностные зависимости (часто причинно-следственные) между переменными. Если есть дуга из узла X в узел Y, то X называется *родителем* Y, а Y — *потомком* X.
- **Количественная часть:** Каждому узлу X_i сопоставлена таблица условных вероятностей (Conditional Probability Table, CPT) $P(X_i|\text{Parents}(X_i))$, которая определяет вероятность каждого возможного значения X_i при каждой комбинации значений его непосредственных родителей $\text{Parents}(X_i)$. Если у узла нет родителей, ему сопоставляется априорное распределение вероятностей $P(X_i)$.

Представление вероятностного пространства: Байесовская сеть позволяет компактно представить совместное распределение вероятностей $P(X_1, X_2, \ldots, X_n)$ для n переменных. Благодаря условной независимости, закодированной в структуре графа, совместное распределение может быть разложено в произведение условных вероятностей: $P(X_1, X_2, \ldots, X_n) = \prod_{i=1}^{n} P(X_i|\text{Parents}(X_i))$ Это разложение значительно уменьшает количество параметров, необходимых для описания совместного распределения, по сравнению с полной таблицей совместных вероятностей (которая требует $2^n - 1$ параметров для n бинарных переменных, тогда как БС требует гораздо меньше, если граф разреженный).

Ключевые концепции условной независимости в БС: Структура DAG в байесовской сети кодирует утверждения об условной независимости. Основное правило: **переменная условно независима от своих не-потомков при заданных ее родителях.** Более формально, для любого узла X_i: $P(X_i|X_1, \ldots, X_{i-1}) = P(X_i|\text{Parents}(X_i))$ (при условии топологической сортировки узлов, где родители всегда предшествуют своим потомкам).

Это позволяет эффективно выполнять **вероятностный вывод (Probabilistic Inference)** — вычисление апостериорных вероятностей некоторых переменных (переменных запроса) при наличии свидетельств (наблюдений) о значениях других переменных. Основные типы запросов:

- Вычисление условной вероятности: $P(\text{Запрос}|\text{Свидетельства})$.
- Поиск наиболее вероятного объяснения (Most Probable Explanation, MPE): Найти наиболее вероятную конфигурацию значений всех не наблюдаемых переменных при заданных свидетельствах.

Алгоритмы вывода в БС могут быть точными (например, исключение переменных, алгоритм распространения доверия для деревьев) или приближенными (например, методы Монте-Карло по схеме марковских цепей (MCMC), вариационный вывод).

2. Построение байесовских сетей

Построение байесовской сети включает два основных этапа:

а) Определение структуры сети (качественный аспект)

Это включает определение:

1. **Переменных:** Какие случайные переменные релевантны для моделируемой предметной области? Каковы их возможные значения (состояния)?

2. **Зависимостей (дуг):** Какие переменные напрямую влияют на другие? Направление дуг часто отражает причинно-следственные связи, хотя это не всегда обязательно (модель может отражать корреляции).

Структура сети может быть построена:

- **Экспертным путем:** На основе знаний экспертов в данной области. Эксперты определяют переменные и причинно-следственные связи между ними. Это наиболее распространенный подход для многих практических приложений.
- **Автоматически из данных (Structural Learning):** С использованием алгоритмов обучения структуры из наблюдательных данных. Это сложная задача, особенно для больших сетей. Методы включают:
  - *Методы на основе ограничений (Constraint-based):* Используют тесты условной независимости для определения наличия или отсутствия дуг.
  - *Методы на основе оценок (Score-based):* Ищут структуру графа, которая максимизирует некоторую оценочную функцию (например, байесовский информационный критерий (BIC) или минимальную

длину описания (MDL)). Это часто NP-трудная задача, требующая эвристического поиска (например, жадный поиск, имитация отжига).
- **Комбинированный подход:** Использование экспертных знаний для задания начальной структуры или ограничений, а затем дообучение или уточнение структуры на данных.

Важно, чтобы полученный граф был **ациклическим**.

б) Определение параметров сети (количественный аспект)

После определения структуры необходимо задать условные вероятности $P(X_i|\text{Parents}(X_i))$ для каждого узла X_i.

- **Экспертным путем:** Эксперты предоставляют оценки условных вероятностей на основе своего опыта и знаний. Это может быть сложно и субъективно, особенно для узлов с большим числом родителей.
- **Автоматически из данных (Parameter Learning):** Если есть размеченный набор данных (где известны значения всех или некоторых переменных для ряда наблюдений), параметры (вероятности в CPT) могут быть оценены статистически.
  - *Для полных данных:* Если значения всех переменных известны для каждого примера, условные вероятности можно оценить методом максимального правдоподобия (по сути, подсчетом частот). Например, $P(X_i = x|\text{Parents}(X_i) = \mathbf{pa}) = \frac{\text{Count}(X_i=x,\text{Parents}(X_i)=\mathbf{pa})}{\text{Count}(\text{Parents}(X_i)=\mathbf{pa})}$. Может потребоваться сглаживание (например, добавление псевдосчетчиков), если данных мало.
  - *Для неполных данных (с пропущенными значениями):* Используются более сложные алгоритмы, такие как EM-алгоритм (Expectation-Maximization) или градиентные методы.

3. Примеры использования байесовских сетей

Пример 1: Медицинская диагностика

Рассмотрим упрощенную сеть для диагностики простуды:

- **Переменные:**
  - Сезон (Лето, Осень, Зима, Весна) - родительская
  - Аллергия (Да, Нет) - родительская
  - Простуда (Да, Нет) - зависит от Сезон
  - Насморк (Да, Нет) - зависит от Простуда, Аллергия
  - Кашель (Да, Нет) - зависит от Простуда
  - Температура (Высокая, Нормальная) - зависит от Простуда
- **Структура (Дуги):**
  - Сезон $\rightarrow$ Простуда
  - Простуда $\rightarrow$ Насморк
  - Аллергия $\rightarrow$ Насморк
  - Простуда $\rightarrow$ Кашель
  - Простуда $\rightarrow$ Температура
- **CPT:** Для каждой переменной задается таблица условных вероятностей. Например, $P(\text{Простуда}|\text{Сезон})$:

Сезон	$P(\text{Простуда=Да})$	$P(\text{Простуда=Нет})$
Лето	0.1	0.9
Осень	0.4	0.6
Зима	0.6	0.4
Весна	0.3	0.7

  Аналогично для $P(\text{Насморк}|\text{Простуда}, \text{Аллергия})$, $P(\text{Кашель}|\text{Простуда})$ и т.д. $P(\text{Аллергия})$ - априорная вероятность.

Использование:

- **Диагностика:** Если пациент жалуется на Насморк=Да и Кашель=Да, можно вычислить апостериорную вероятность $P(\text{Простуда=Да}|\text{Насморк=Да}, \text{Кашель=Да})$.
- **Прогнозирование:** Какова вероятность Насморка, если известно, что сейчас Сезон=Зима?

Пример 2: Фильтрация спама

- **Переменные:**

  - Спам (Да, Нет) - целевая переменная (корень, если мы моделируем $P(\text{Слова}|\text{Спам})$, или лист, если $P(\text{Спам}|\text{Слова})$ - как в наивном Байесе)
  - Слово1присутствует (Да, Нет),
    Слово2присутствует(Да, Нет), ...,
    СловоNприсутствует (Да, Нет) - признаки (наличие определенных слов в письме).
- **Структура (Наивный Байес):** Простейший случай байесовской сети, где предполагается, что все признаки (слова) условно независимы при заданном классе (Спам/Не спам). Дуги идут от узла Спам к каждому узлу СловоХприсутствует.
  Спам $\rightarrow$ Слово1присутствует
  Спам $\rightarrow$ Слово2присутствует, ...
- **CPT:** $P(\text{Спам})$ (априорная вероятность спама) и $P(\text{СловоХприсутствует} \mid \text{Спам})$(вероятность наличия слова X, если письмо спам, и если не спам).

Использование: Получив новое письмо (набор слов), вычисляется $P(\text{Спам=Да}|\text{Слова})$ и $P(\text{Спам=Нет}|\text{Слова})$. Письмо классифицируется как спам, если $P(\text{Спам=Да}|\text{Слова})$ выше.

Другие области применения:

- **Биоинформатика:** Моделирование генных регуляторных сетей, анализ экспрессии генов.
- **Финансовое моделирование:** Оценка рисков, прогнозирование рынков.
- **Мониторинг и диагностика неисправностей** в сложных системах.

15 Модели прогнозирования на основе деревьев решений. Алгоритмы CART и C4.5: критерии поиска разбиений, параметры ограничения роста и отсечения.

Деревья решений — это популярный и интуитивно понятный метод машинного обучения, используемый как для задач классификации (Classification Trees), так и для задач регрессии (Regression Trees). Они представляют собой иерархическую структуру правил вида "если... то... которая разбивает пространство признаков на области, соответствующие определенному прогнозу.

1. Модели прогнозирования на основе деревьев решений

Структура дерева решений:

- **Корневой узел (Root Node):** Представляет всю совокупность данных (обучающую выборку).
- **Внутренние узлы (Internal Nodes / Decision Nodes):** Представляют собой проверку некоторого признака (атрибута). Каждый узел имеет несколько исходящих ветвей (ребер).
- **Ветви (Branches / Edges):** Соответствуют возможным значениям (или диапазонам значений) проверяемого признака.
- **Листовые узлы (Leaf Nodes / Terminal Nodes):** Представляют собой конечные решения (прогнозы).
  - В задачах **классификации** лист содержит метку класса (наиболее вероятный класс для объектов, попавших в этот лист) или распределение вероятностей классов.
  - В задачах **регрессии** лист содержит числовое значение (например, среднее значение целевой переменной для объектов в этом листе).

Для получения прогноза для нового объекта, он "пропускается"через дерево от корня к одному из листьев в соответствии со значениями его признаков.

Преимущества деревьев решений:

- **Интерпретируемость:** Легко понять и объяснить логику принятия решений.
- **Работа с разными типами данных:** Могут обрабатывать как категориальные, так и числовые признаки.
- **Не требуют сложной подготовки данных:** Например, не требуют масштабирования признаков.
- **Устойчивость к выбросам (относительная):** Влияние отдельных выбросов может быть локализовано в одной ветви.
- **Непараметрическая модель:** Не делают предположений о распределении данных.

Недостатки деревьев решений:

- **Склонность к переобучению (Overfitting):** Могут строить слишком сложные деревья, которые хорошо описывают обучающую выборку, но плохо обобщают на новых данных.
- **Нестабильность:** Небольшие изменения в данных могут привести к построению совершенно другого дерева.
- **Проблема представления некоторых функций:** Сложно аппроксимировать некоторые типы зависимостей (например, линейные или диагональные границы).
- **Жадный характер построения:** На каждом шаге выбирается локально оптимальное разбиение, что не гарантирует глобальной оптимальности дерева.

Процесс построения дерева (индукция дерева): Деревья обычно строятся "сверху вниз"с использованием рекурсивного разделения (recursive partitioning):

1. Начать с корневого узла, содержащего все обучающие примеры.

2. Выбрать наилучший признак и точку (или значение) для разбиения данных в текущем узле на два или более дочерних узла. "Наилучший"определяется с помощью некоторого **критерия качества разбиения**.

   Gini impurity для одного узла:

$$G = 1 - \sum_{i=1}^{C} p_i^2$$

   где:

   - C — число классов,
   - p_i — доля объектов класса i в узле.

   Взвешенное среднее Gini после разбиения:

$$G_{\text{split}} = \frac{n_{\text{left}}}{n} \cdot G_{\text{left}} + \frac{n_{\text{right}}}{n} \cdot G_{\text{right}}$$

где:

- n — общее число объектов,
- n_{left} и n_{right} — число объектов в левой и правой группах,
- G_{left}, G_{right} — Gini impurity этих групп.

Пример:

Допустим, в результате разбиения получились такие группы:

- **Левая группа:** 3 объекта — 2 "Да 1 "Нет"
- **Правая группа:** 2 объекта — 1 "Да 1 "Нет"

Вычислим Gini для каждой группы:

$$G_{\text{left}} = 1 - \left(\frac{2}{3}\right)^2 - \left(\frac{1}{3}\right)^2 = 1 - \frac{4}{9} - \frac{1}{9} = \frac{4}{9} \approx 0.444$$

$$G_{\text{right}} = 1 - \left(\frac{1}{2}\right)^2 - \left(\frac{1}{2}\right)^2 = 1 - \frac{1}{4} - \frac{1}{4} = \frac{1}{2} = 0.5$$

Общий Gini после разбиения:

$$G_{\text{split}} = \frac{3}{5} \cdot 0.444 + \frac{2}{5} \cdot 0.5 = 0.266 + 0.2 = 0.466$$

3. Разделить данные на подмножества в соответствии с выбранным разбиением и создать дочерние узлы.

4. Рекурсивно повторить шаги 2-3 для каждого дочернего узла до тех пор, пока не будет выполнено одно из **условий остановки** (например, все примеры в узле принадлежат одному классу, достигнута максимальная глубина, в узле слишком мало примеров).

5. (Опционально) Применить процедуру **отсечения (pruning)** для уменьшения переобучения (Ограничения при построении: max depth, min samples leaf).

2. Алгоритм CART (Classification and Regression Trees)

CART — это алгоритм, который строит бинарные деревья решений (каждый внутренний узел имеет ровно два потомка). Он может использоваться как для классификации, так и для регрессии.

Критерии поиска разбиений в CART

а) Для задач классификации: Цель — найти такое разбиение, которое максимизирует "чистоту"(это насколько объекты в этом узле принадлежат одному и тому же классу) дочерних узлов по отношению к классам.

- **Индекс Джини (Gini Impurity / Gini Index):** Мера того, насколько вероятно, что случайно выбранный элемент из набора будет неправильно классифицирован, если его класс будет случайно присвоен в соответствии с распределением классов в наборе. Для узла m с K классами, где p_{mk} — доля объектов класса k в узле m: $Gini(m) = 1 - \sum_{k=1}^{K} p_{mk}^2$ Значение $Gini(m)$ варьируется от 0 (все объекты в узле принадлежат одному классу — идеальная чистота) до $1 - 1/K$ (равномерное распределение классов). При выборе разбиения, CART ищет такое, которое максимизирует уменьшение индекса Джини (Gini Gain): $\Delta Gini = Gini(parent) - \sum_j \frac{N_j}{N_{parent}} Gini(child_j)$ где N_j — число объектов в дочернем узле j, N_{parent} — число объектов в родительском узле.

б) Для задач регрессии: Цель — минимизировать дисперсию целевой переменной в дочерних узлах.

- **Среднеквадратичная ошибка (Mean Squared Error, MSE) или Сумма квадратов остатков (Sum of Squared Residuals, SSR):** Для узла m, где y_i — значение целевой переменной для i-го объекта, а $\bar{y}_m$ — среднее значение целевой переменной для всех объектов в узле m: $MSE(m) = \frac{1}{N_m} \sum_{i \in m} (y_i - \bar{y}_m)^2$ При выборе разбиения, CART ищет такое, которое максимизирует уменьшение MSE (или SSR). $\Delta MSE = MSE(parent) - \sum_j \frac{N_j}{N_{parent}} MSE(child_j)$

Для числовых признаков CART перебирает все возможные точки разбиения (обычно середины между отсортированными уникальными значениями признака). Для категориальных признаков (с L категориями) в бинарном дереве CART может рассматривать все $2^{L-1} - 1$ возможных бинарных разбиений на два подмножества категорий (для небольших L) или использовать эвристики.

3. Алгоритм C4.5

Алгоритм строит не обязательно бинарные деревья и может работать с пропущенными значениями (тогда используется пропорциональное ветвление по вероятностям).

Критерии поиска разбиений в C4.5

а) Для задач классификации (регрессию не поддерживает):

- **Прирост информации (Information Gain):** Основан на понятии энтропии из теории информации. Энтропия измеряет неопределенность (нечистоту) в наборе данных. Энтропия для узла m: $Entropy(m) = -\sum_{k=1}^{K} p_{mk} \log_2(p_{mk})$ Значение $Entropy(m)$ варьируется от 0 (идеальная чистота) до $\log_2(K)$ (равномерное распределение). Прирост информации при разбиении по признаку A: $Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$ где S — набор данных в текущем узле, $Values(A)$ — множество значений признака A, S_v — подмножество S, где признак A имеет значение v. ID3 использует прирост информации. Однако, он имеет тенденцию предпочитать признаки с большим количеством уникальных значений (например, идентификаторы).
- **Коэффициент прироста информации (Gain Ratio):** C4.5 использует модификацию прироста информации, чтобы уменьшить это смещение. Коэффициент прироста нормализует прирост информации на "собственную информацию"признака (Split Information или Intrinsic Information), которая измеряет, насколько широко и равномерно признак разбивает данные. $SplitInfo(S, A) = -\sum_{v \in Values(A)} \frac{|S_v|}{|S|} \log_2 \left(\frac{|S_v|}{|S|} \right)$ $GainRatio(S, A) = \frac{Gain(S,A)}{SplitInfo(S,A)}$ Выбирается признак с максимальным коэффициентом прироста информации.

C4.5 может обрабатывать числовые признаки, находя оптимальную точку бинарного разбиения (аналогично CART). Для категориальных признаков он по умолчанию создает ветвь для каждого значения.

4. Параметры ограничения роста и отсечения (Pruning)

Эти методы используются для борьбы с переобучением деревьев решений.

а) Ограничение роста (Pre-pruning / Ранняя остановка)

Остановка процесса построения дерева до того, как оно идеально подгонится под обучающие данные.

- **Минимальное количество объектов в узле для разбиения:** Если число объектов в узле меньше заданного порога, узел не разбивается и становится листом.
- **Минимальное количество объектов в листовом узле:** Если после разбиения какой-либо из дочерних узлов будет содержать меньше заданного порога объектов, то разбиение не выполняется.
- **Максимальная глубина дерева:** Ограничивает максимальную длину пути от корня до листа.
- **Минимальное улучшение критерия качества:** Если наилучшее возможное разбиение не дает достаточного улучшения (например, $\Delta Gini$ или $GainRatio$ слишком мал), то узел не разбивается.

Pre-pruning может быть слишком "жадным"и остановить рост дерева преждевременно, упустив хорошие разбиения на более глубоких уровнях.

б) Отсечение (Pruning / Post-pruning)

Сначала строится полное (или почти полное) дерево, а затем удаляются (отсекаются) некоторые его части (ветви или узлы), которые считаются менее полезными или приводящими к переобучению. Post-pruning обычно дает лучшие результаты, чем pre-pruning.

- **Отсечение на основе ошибки (Error-Based Pruning):**
  - **Reduced Error Pruning (REP):** Использует отдельный набор данных (validation set). Для каждого внутреннего узла рассматривается замена его поддерева на листовой узел (с меткой большинства). Если такая замена не ухудшает (или улучшает) точность на валидационном наборе, отсечение выполняется. Процесс идет снизу вверх.

- **Pessimistic Error Pruning (PEP - используется в C4.5):** Не требует отдельного валидационного набора. Оценивает ошибку узла и его поддерева на обучающих данных, добавляя к листу штраф за сложность (например, на основе биномиального распределения). Если ошибка листа меньше, чем ошибка поддерева, то поддерево заменяется листом.

- **Отсечение по стоимости-сложности (Cost-Complexity Pruning / Weakest Link Pruning - используется в CART):** Строится последовательность вложенных поддеревьев исходного полного дерева. Для каждого поддерева T вычисляется параметр стоимости-сложности α: $R_\alpha(T) = R(T) + \alpha \cdot |L(T)|$ где $R(T)$ — ошибка дерева на обучающей выборке (например, число неправильно классифицированных примеров), $|L(T)|$ — количество листьев в дереве T. Цель — найти дерево, минимизирующее $R_\alpha(T)$. Параметр α варьируется, и для каждого α находится оптимальное поддерево. Окончательный выбор лучшего дерева из этой последовательности обычно делается с помощью кросс-валидации.

  1. Посчитай значение α_t для каждого поддерева (внутреннего узла) по формуле:

     $$\alpha_t = \frac{R(t) - R(T_t)}{|\mathcal{L}(T_t)| - 1}$$

     где:
     - $R(T_t)$ — ошибка поддерева (всех его листьев),
     - $R(t)$ — ошибка, если всё поддерево заменить на один лист (класс большинства),
     - $|\mathcal{L}(T_t)|$ — число листьев в поддереве T_t.
  2. Найди минимальное значение α_t — это соответствует "самой слабой ветви".
  3. Обрежь эту ветвь: замени всё поддерево T_t на один лист с классом большинства.
  4. Сохрани текущее дерево в цепочку упрощённых деревьев.
  5. Повтори шаги 1–4 на новом дереве, пока не останется одно узловое дерево (единственный лист).

  Как найти лучшее дерево из полученной цепочки:
  1. Разбиваем обучающую выборку на k фолдов (например, $k = 5$).
  2. Для каждого дерева T_i из цепочки:
     (a) Обучаем дерево T_i на $k - 1$ фолдах.
     (b) Оцениваем его точность (или ошибку) на оставшемся фолде.
     (c) Повторяем это для всех k фолдов и вычисляем среднюю ошибку.
  3. Выбираем дерево T_i с минимальной средней ошибкой на кросс-валидации.

16 Нейронные сети прямого распространения. Архитектуры MLP и RBF: структура сетей, виды функций активации, алгоритмы обучения, проблема переобучения и локальных минимумов.

Нейронные сети прямого распространения (Feedforward Neural Networks) — это класс искусственных нейронных сетей, в которых информация движется только в одном направлении: от входного слоя через один или несколько скрытых слоев к выходному слою. В таких сетях отсутствуют циклы или обратные связи (в отличие от рекуррентных сетей). Они являются основой для многих моделей машинного обучения, используемых для задач классификации, регрессии и аппроксимации функций.

Двумя распространенными архитектурами сетей прямого распространения являются многослойный перцептрон (MLP) и сети на основе радиальных базисных функций (RBF).

1. Многослойный перцептрон (Multilayer Perceptron, MLP)

Структура сети MLP

MLP состоит из нескольких слоев нейронов:

- **Входной слой (Input Layer):** Получает входные данные (вектор признаков x). Количество нейронов равно размерности входного вектора. Нейроны этого слоя обычно не выполняют вычислений, а просто передают значения дальше.
- **Скрытые слои (Hidden Layers):** Один или несколько слоев нейронов, расположенных между входным и выходным слоями. Эти слои выполняют основную обработку информации. Каждый нейрон j в скрытом слое l вычисляет взвешенную сумму своих входов $z_j^{(l)} = \sum_i w_{ji}^{(l)} a_i^{(l-1)} + b_j^{(l)}$, где $a_i^{(l-1)}$ — активация нейрона i из предыдущего слоя $(l-1)$, $w_{ji}^{(l)}$ — вес связи между нейронами i и j, $b_j^{(l)}$ — смещение (bias) нейрона j. Затем к этой сумме применяется нелинейная **функция активации** ϕ: $a_j^{(l)} = \phi(z_j^{(l)})$.
- **Выходной слой (Output Layer):** Производит окончательный результат (прогноз) сети. Количество нейронов и их функции активации зависят от типа задачи (например, один нейрон с линейной функцией для регрессии, K нейронов с функцией Softmax для K-классовой классификации).

Обычно слои в MLP являются **полносвязными (fully connected)**, то есть каждый нейрон одного слоя соединен со всеми нейронами следующего слоя.

1. Инициализация весов

Когда нейросеть только создаётся, она не знает, как решать задачу. Поэтому все веса w_{ji} и смещения b_j инициализируются случайными числами (обычно из нормального распределения или равномерного диапазона).

- Пример: $w = 0.3$, другой вес может быть $w = -0.8$.

2. Прямое распространение (предсказание)

Сеть получает на вход вектор признаков, например:

Входной вектор: $[180, 80]$, где первый элемент — рост, второй — вес.

С помощью текущих весов вычисляется результат. Допустим, выход сети:

$$\hat{y} = 0.6$$

(вероятность того, что у человека ожирение — 60%).

3. Расчёт ошибки

Сравниваем предсказание $\hat{y}$ с правильным ответом y (в данном случае $y = 1$ — ожирение есть):

Ошибка: $\hat{y} - y = 0.6 - 1 = -0.4$

4. Обратное распространение ошибки и градиентный спуск

Сеть должна понять, какие веса привели к ошибке. Это делается методом **градиентного спуска**.

- Вычисляется вклад каждого веса в ошибку — **градиент**:

$$\frac{\partial \text{error}}{\partial w}$$

- Если вес увеличивает ошибку — его нужно уменьшить.

5. Обновление весов

Каждый вес w обновляется по формуле:

$$w \leftarrow w - \eta \cdot \frac{\partial \text{error}}{\partial w}$$

где η — скорость обучения.
где η — **скорость обучения** (например, 0.01).

6. Повторение процесса

1. Сеть делает предсказание
2. Сравнивает его с правильным ответом
3. Распространяет ошибку назад
4. Обновляет веса
5. Повторяет всё для следующего примера

Так, шаг за шагом, нейросеть обучается. Через множество примеров она «понимает», какие веса дают хорошие результаты.

7. Зачем нужна функция активации?

Если убрать функцию активации, сеть будет просто линейной моделью:

$$y = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b$$

Это значит, что она не сможет различать сложные, **нелинейные зависимости**, например, отличить круг от квадрата.

Функция активации делает модель **нелинейной** и мощной.

Виды функций активации в MLP

Функции активации вводят нелинейность в модель, позволяя сети аппроксимировать сложные зависимости.

- **Пороговая функция (Threshold/Step function):** $\phi(z) = 1$ если $z \geq 0$, иначе 0. Исторически использовалась в перцептронах, но недифференцируема, что затрудняет обучение градиентными методами.
- **Сигмоидная функция (Sigmoid/Logistic):** $\phi(z) = \frac{1}{1+e^{-z}}$. Диапазон (0, 1). Гладкая, дифференцируемая, но страдает от проблемы *затухающих градиентов* (vanishing gradients) при больших $|z|$ и не центрирована относительно нуля. Часто используется в выходном слое для бинарной классификации.
- **Гиперболический тангенс (tanh):** $\phi(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = 2 \cdot \text{sigmoid}(2z) - 1$. Диапазон (-1, 1). Центрирована относительно нуля, что может ускорять сходимость, но проблема затухающих градиентов остается.
- **ReLU (Rectified Linear Unit):** $\phi(z) = \max(0, z)$. Наиболее популярная функция для скрытых слоев. Вычислительно эффективна, не насыщается для $z > 0$ (избегает затухания градиента). Проблема: "мертвые ReLU"(нейроны, которые всегда выдают 0 для любых входов).
- **Leaky ReLU:** $\phi(z) = \max(\alpha z, z)$, где α — малая константа (например, 0.01). Решает проблему мертвых ReLU, позволяя небольшой ненулевой градиент при $z < 0$. Вариант: PReLU (Parametric ReLU), где α является обучаемым параметром.
- **ELU (Exponential Linear Unit):** $\phi(z) = z$ если $z > 0$, иначе $\alpha(e^z - 1)$. Похожа на Leaky ReLU, но использует экспоненту для $z < 0$. Может давать отрицательные выходы и приближаться к нулевому среднему.

- **Линейная функция (Linear/Identity):** $\phi(z) = z$. Используется в выходном слое для задач регрессии.
- **Softmax:** $\phi(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$. Преобразует вектор действительных чисел (логитов) z в вектор вероятностей, сумма которых равна 1. Используется в выходном слое для многоклассовой классификации.

Алгоритм обучения MLP (Backpropagation)

Обучение MLP — это процесс настройки весов $w_{ji}^{(l)}$ и смещений $b_j^{(l)}$ для минимизации некоторой **функции потерь (loss function)** L (например, среднеквадратичная ошибка для регрессии, перекрестная энтропия для классификации) на обучающей выборке. Основным алгоритмом является **метод обратного распространения ошибки (Backpropagation)**:

1. **Инициализация:** Веса и смещения инициализируются малыми случайными значениями.

2. **Прямое распространение (Forward Pass):** Для входного примера x вычисляются активации всех нейронов слой за слоем, от входа к выходу, до получения прогноза $\hat{y}$.

3. **Вычисление ошибки:** Вычисляется значение функции потерь $L(\hat{y}, y)$, где y — истинное значение.

4. **Обратное распространение (Backward Pass):** Вычисляются градиенты функции потерь по отношению ко всем весам и смещениям сети. Это делается с использованием правила цепного дифференцирования, начиная с выходного слоя и двигаясь назад к входному. $\frac{\partial L}{\partial w_{ji}^{(l)}} = \frac{\partial L}{\partial a_j^{(l)}} \frac{\partial a_j^{(l)}}{\partial z_j^{(l)}} \frac{\partial z_j^{(l)}}{\partial w_{ji}^{(l)}} = \delta_j^{(l)} a_i^{(l-1)}$, где $\delta_j^{(l)} = \frac{\partial L}{\partial z_j^{(l)}}$ — ошибка нейрона j слоя l. Ошибка $\delta^{(l)}$ вычисляется через ошибки следующего слоя $\delta^{(l+1)}$.

5. **Коррекция весов:** Веса и смещения обновляются в направлении, противоположном градиенту, с использованием некоторого **алгоритма оптимизации** (например, градиентный спуск): $w_{ji}^{(l)} \leftarrow w_{ji}^{(l)} - \eta \frac{\partial L}{\partial w_{ji}^{(l)}}$ $b_j^{(l)} \leftarrow b_j^{(l)} - \eta \frac{\partial L}{\partial b_j^{(l)}}$ где η — **скорость обучения (learning rate)**. Популярные оптимизаторы: SGD (Stochastic Gradient Descent), Mini-batch GD, Adam, RMSprop, Adagrad.

6. **Повторение:** Шаги 2-5 повторяются для многих примеров (эпох) до сходимости.

2. Сети на основе радиальных базисных функций (RBF Networks)

Структура сети RBF

Сети RBF обычно имеют трехслойную архитектуру:
- **Входной слой:** Принимает входной вектор x.
- **Скрытый слой:** Состоит из нейронов, использующих **радиальную базисную функцию** в качестве функции активации. Каждый нейрон i характеризуется **центром** c_i (вектор той же размерности, что и вход) и **параметром ширины** σ_i.
- **Выходной слой:** Обычно состоит из одного или нескольких линейных нейронов, которые вычисляют взвешенную сумму активаций нейронов скрытого слоя.

В отличие от MLP, здесь обычно только один скрытый слой, и он не является полносвязным в том же смысле (связь определяется близостью входа к центру нейрона).

Функции активации RBF (скрытый слой)

Активация $\phi_i(x)$ нейрона i скрытого слоя зависит от расстояния между входным вектором x и центром нейрона c_i. Наиболее распространенной является Гауссова RBF: $\phi_i(x) = \exp\left(-\frac{\|x-c_i\|^2}{2\sigma_i^2}\right)$ где $\|\cdot\|$ — евклидово расстояние. Активация максимальна (равна 1), когда $x = c_i$, и экспоненциально убывает с увеличением расстояния.

Выходной слой

Выход сети y_k (для k-го выходного нейрона) является линейной комбинацией активаций скрытых нейронов: $y_k = \sum_{i=1}^{M} w_{ki}\phi_i(x) + b_k$ где M — количество нейронов в скрытом слое, w_{ki} — вес связи между i-м скрытым нейроном и k-м выходным нейроном, b_k — смещение выходного нейрона.

Алгоритмы обучения RBF

Обучение RBF-сети часто происходит в два (или три) этапа, используя гибридный подход:

1. **Обучение параметров скрытого слоя (неконтролируемое):**
   - **Определение центров c_i:** Часто используется алгоритм кластеризации (например, K-means) на входных обучающих данных x. Центры кластеров становятся центрами RBF-нейронов. Другие методы: случайный выбор центров из обучающих данных.
   - **Определение ширин σ_i:** Могут быть установлены одинаковыми для всех нейронов или индивидуально. Часто ширина σ_i связывается со средним расстоянием до k ближайших соседей центра c_i или с дисперсией данных в кластере.

2. **Обучение весов выходного слоя (контролируемое):** После того как параметры скрытого слоя c_i, σ_i зафиксированы, активации $\phi_i(x)$ для всех обучающих примеров x могут быть вычислены. Задача нахождения весов w_{ki} становится задачей линейной регрессии (минимизация ошибки $\sum(y - \hat{y})^2$). Ее можно решить точно с помощью псевдообращения матрицы активаций или итеративно с помощью градиентного спуска (например, алгоритм LMS - Least Mean Squares).

Существуют также полностью контролируемые подходы, где все параметры (c_i, σ_i, w_{ki}) настраиваются одновременно с помощью градиентного спуска, аналогично MLP, но это встречается реже.

Сравнение MLP и RBF:

- MLP обычно требует больше времени на обучение (из-за backpropagation через все слои), но может требовать меньше нейронов для аппроксимации сложных функций.
- RBF обучаются быстрее (особенно с гибридным подходом), но могут требовать больше скрытых нейронов.
- MLP выполняет глобальную аппроксимацию, RBF — локальную (активация нейрона значима только в окрестности его центра). RBF часто лучше справляются с интерполяцией.
- MLP использует скалярное произведение и нелинейную активацию; RBF использует расстояние и радиальную активацию.

3. Проблема переобучения и локальных минимумов

Переобучение (Overfitting)

Описание: Ситуация, когда нейронная сеть слишком хорошо подстраивается под особенности обучающей выборки, включая шум, и теряет способность к обобщению на новых, ранее не виденных данных. Ошибка на обучающей выборке продолжает уменьшаться, а ошибка на тестовой (или валидационной) выборке начинает расти. **Причины:**

- Слишком сложная модель (много слоев, нейронов, параметров).
- Недостаточное количество обучающих данных.
- Слишком долгое обучение.

Методы борьбы:

- **Регуляризация:** Добавление штрафа к функции потерь за большие значения весов.
  - L2-регуляризация (Weight Decay): $L_{reg} = L + \frac{\lambda}{2} \sum w^2$. Штрафует большие веса, способствует получению более гладких моделей.
  - L1-регуляризация: $L_{reg} = L + \lambda \sum |w|$. Может приводить к обнулению некоторых весов, способствуя разреженности модели (отбору признаков).
- **Dropout:** Во время обучения случайным образом "отключается"(обнуляется выход) некоторый процент нейронов на каждом шаге. Это заставляет сеть обучаться более обобщающим представлениям, не полагаясь слишком сильно на отдельные нейроны. Каждый из них должен научиться быть универсальным и хорошо работать с любой комбинацией.
- **Ранняя остановка (Early Stopping):** Обучение прекращается, когда ошибка на отдельной валидационной выборке (не участвующей в обновлении весов) перестает уменьшаться или начинает расти.
- **Аугментация данных (Data Augmentation):** Искусственное увеличение размера обучающей выборки путем применения различных преобразований к существующим данным (например, поворот, сдвиг, изменение яркости изображений).
- **Уменьшение сложности сети:** Уменьшение числа слоев или нейронов.

Локальные минимумы

Описание: Функция потерь нейронной сети обычно является невыпуклой и имеет множество локальных минимумов. Градиентный спуск может сойтись к одному из этих локальных минимумов, который не является глобальным минимумом (наилучшим возможным решением). **Современный взгляд:** В пространствах высокой размерности, характерных для глубоких нейронных сетей, проблема плохих локальных минимумов (значительно хуже глобального) считается менее серьезной, чем проблема **седловых точек** (точек, где градиент равен нулю, но которые не являются экстремумами). Седловые точки могут значительно замедлять обучение. **Методы борьбы/смягчения:**

- **Алгоритмы оптимизации с моментом:** SGD с моментом, Adam, RMSprop помогают "проскакивать"мелкие локальные минимумы и быстрее проходить седловые точки.
- **Стохастичность:** Использование стохастического или мини-батчевого градиентного спуска вносит шум в процесс обновления весов, что может помочь выбраться из локальных минимумов.
- **Адаптивные скорости обучения:** Алгоритмы типа Adam или RMSprop подстраивают скорость обучения для каждого параметра, что может помочь в навигации по сложному ландшафту функции потерь.
- **Инициализация весов:** Использование продуманных стратегий инициализации (например, Xavier/Glorot, He) помогает начать обучение в "хорошей"области пространства параметров.
- **Запуск нескольких обучений:** Обучение сети несколько раз с разными случайными инициализациями и выбор наилучшего результата.

17 Обучение ассоциативным правилам. Алгоритмы Apriori и FP-tree: построение частых эпизодов с ограничением по поддержке и формирование правил с ограничением по достоверности.

Обучение ассоциативным правилам — это метод интеллектуального анализа данных (Data Mining), направленный на выявление интересных зависимостей между элементами в больших наборах данных. Эти зависимости представляются в виде *ассоциативных правил* (Если A, то B). Классическим примером является анализ покупательских корзин, где правило может выглядеть как "{Хлеб, Молоко} $\Rightarrow$ {Масло}".

1. Основные понятия

- **Набор элементов (Itemset):** Множество из одного или более элементов (items). Например, {Хлеб, Молоко, Масло}.
- **Транзакция (Transaction):** Конкретный набор элементов, приобретенных вместе. Например, одна покупательская корзина.
- **База данных транзакций (Transaction Database):** Совокупность всех транзакций.
- **Ассоциативное правило (Association Rule):** Выражение вида $X \Rightarrow Y$, где X и Y — это наборы элементов, и $X \cap Y = \emptyset$.
  - X называется **антецедентом** (antecedent) или посылкой.
  - Y называется **консеквентом** (consequent) или следствием.

  Правило $X \Rightarrow Y$ означает, что если в транзакции присутствует набор X, то с определенной вероятностью там же будет и набор Y.

Меры интересности правил: Для оценки "интересности"или "силы"ассоциативных правил используются две основные метрики:

1. **Поддержка (Support):** Доля транзакций в базе данных, которые содержат все элементы из набора $X \cup Y$.

   Насколько часто вообще встречается определенный набор (или правило). Это показатель "популярности"или "распространенности"комбинации.

   $$\text{supp}(X \Rightarrow Y) = \text{supp}(X \cup Y) = \frac{|\{t \in T \mid X \cup Y \subseteq t\}|}{|T|}$$

   Здесь T — множество всех транзакций. Поддержка показывает, насколько часто такое сочетание элементов встречается в данных. Правила с низкой поддержкой могут быть случайными. Обычно устанавливают минимальный порог поддержки (minSupport). Если поддержка правила ниже этого порога, оно считается неинтересным.

2. **Достоверность (Confidence):** Условная вероятность того, что транзакция, содержащая X, также будет содержать Y.

   Если покупатель уже использует X, то какова вероятность, что он будет использовать Y? Это показатель "надежности"правила.

   $$\text{conf}(X \Rightarrow Y) = \frac{\text{supp}(X \cup Y)}{\text{supp}(X)} = \frac{|\{t \in T \mid X \cup Y \subseteq t\}|}{|\{t \in T \mid X \subseteq t\}|}$$

   Зачем нужна достоверность?

   Достоверность показывает, насколько можно "положиться"на правило. Высокая достоверность означает, что если условие (X) выполнено, то следствие (Y) наступает с большой вероятностью. Обычно также устанавливают минимальный порог достоверности (minConfidence).

   Важный момент: высокая достоверность не всегда означает интересное правило. Представь правило: {Что_угодно} $\Rightarrow$ {Хлеб}. Хлеб покупают почти все. Допустим, $\text{supp}(\{\text{Хлеб}\}) = 0.9$ (90% чеков содержат хлеб). Тогда для любого товара X правило $\{X\} \Rightarrow$ {Хлеб} будет иметь высокую достоверность, просто потому что хлеб и так популярен. Здесь нам поможет *лифт*.

Другие метрики, такие как **лифт (lift)**, также могут использоваться для оценки правил, особенно для выявления правил, которые не являются очевидными или случайными. $\text{lift}(X \Rightarrow Y) = \frac{\text{supp}(X \cup Y)}{\text{supp}(X) \cdot \text{supp}(Y)} = \frac{\text{conf}(X \Rightarrow Y)}{\text{supp}(Y)}$. Лифт > 1 указывает на положительную корреляцию между X и Y. Насколько покупка товара X увеличивает (или

уменьшает) вероятность покупки товара Y, по сравнению с тем, как если бы эти покупки были независимы. Лифт показывает, есть ли между X и Y какая-то "особая"связь, а не просто случайное совпадение или популярность одного из товаров.

Задача обучения ассоциативным правилам: Задача состоит из двух основных этапов:

1. **Нахождение всех частых наборов элементов (Frequent Itemsets):** Найти все наборы элементов, поддержка которых не меньше заданного минимального порога поддержки (*minsupp*).

2. **Генерация ассоциативных правил:** Из найденных частых наборов элементов сформировать правила, достоверность которых не меньше заданного минимального порога достоверности (*minconf*).

Первый этап является вычислительно наиболее сложным.

2. Алгоритм Apriori

Алгоритм Apriori — это классический и один из наиболее известных алгоритмов для нахождения частых наборов элементов. Он основан на "антимонотонном"свойстве поддержки:

Свойство Apriori (Антимонотонность поддержки): Если набор элементов является частым, то все его подмножества также должны быть частыми. И наоборот, если какой-либо поднабор элемента не является частым, то и сам набор элементов не может быть частым.

Если набор товаров часто покупают вместе (например, {Хлеб, Молоко, Яйца} — популярная комбинация), то любая его часть (например, {Хлеб, Молоко} или просто {Яйца}) тоже должна быть популярной, как минимум не меньше, а то и больше. Логично, правда? Если тройка товаров часто встречается, то двойка из них точно будет встречаться не реже.

И наоборот (это самое важное для алгоритма Apriori!): если какой-то маленький набор товаров *не популярен* (например, {Экзотический фрукт, Селедка} почти никто не покупает вместе), то и любой **больший** набор, содержащий эту непопулярную пару (например, {Экзотический фрукт, Селедка, Сливки}), тоже *не будет популярным*.

Зачем нам проверять сложную комбинацию, если мы уже знаем, что её часть — провальная? Именно этот принцип позволяет Apriori отсекать множество бесполезных кандидатов ещё до того, как мы начнём считать их поддержку.

Принцип работы алгоритма Apriori:

Алгоритм работает итеративно, уровень за уровнем:

1. **Шаг 1 (k=1):**
   - Сканировать базу данных транзакций для подсчета поддержки каждого отдельного элемента (наборов из 1 элемента, 1-itemsets).
   - Отобрать те элементы, поддержка которых $\geq$ *minsupp*. Это множество частых 1-наборов, обозначаемое L_1.

2. **Шаг k (k > 1):** (Генерация частых k-наборов L_k из частых (k-1)-наборов L_{k-1})

   (a) **Генерация кандидатов (Candidate Generation):** Сгенерировать множество кандидатов в частые k-наборы C_k из L_{k-1}. Это делается путем объединения двух (k-1)-наборов из L_{k-1}, которые имеют (k-2) общих начальных элементов. Например, если $L_2 = \{\{A, B\}, \{A, C\}, \{B, C\}\}$, то кандидатом в C_3 из $\{A, B\}$ и $\{A, C\}$ будет $\{A, B, C\}$.

   (b) **Отсечение кандидатов (Candidate Pruning):** Для каждого кандидата из C_k проверить, все ли его (k-1)-поднаборы содержатся в L_{k-1}. Если хотя бы один (k-1)-поднабор не является частым (не в L_{k-1}), то кандидат удаляется из C_k (согласно свойству Apriori).

   (c) **Подсчет поддержки кандидатов:** Просканировать базу данных транзакций для подсчета поддержки каждого оставшегося кандидата в C_k.

   (d) **Формирование L_k:** Отобрать те кандидаты из C_k, поддержка которых $\geq$ *minsupp*. Это будет множество частых k-наборов L_k.

3. **Завершение:** Алгоритм останавливается, когда на некотором шаге k множество L_k становится пустым. Результатом является объединение всех найденных L_i ($i = 1, \ldots, k-1$).

Генерация правил из частых наборов: После нахождения всех частых наборов элементов F (например, $F = X \cup Y$), для каждого такого набора генерируются все возможные непустые собственные поднаборы $X \subset F$. Затем для каждого такого X формируется правило $X \Rightarrow (F \setminus X)$. Достоверность этого правила вычисляется как $\text{conf}(X \Rightarrow (F \setminus X)) = \frac{\text{supp}(F)}{\text{supp}(X)}$. Если достоверность $\geq$ *minconf*, правило считается значимым. (Заметим, что $\text{supp}(F)$ и $\text{supp}(X)$ уже были вычислены на этапе поиска частых наборов).

Преимущества Apriori:

- Использует свойство антимонотонности для эффективного сокращения пространства поиска кандидатов.
- Относительно прост для понимания и реализации.

Недостатки Apriori:

- Многократное сканирование базы данных (столько раз, какова максимальная длина частого набора). Это может быть очень затратно для больших баз данных.
- Генерация большого числа кандидатов, особенно на ранних этапах для плотных данных или низких *minsupp*.

3. Алгоритм FP-Growth (Frequent Pattern Growth)

FP-Growth делает следующее:

Сжимает всю информацию о частых элементах и их совместной встречаемости в специальную древовидную структуру – FP-дерево. Это делается всего за два прохода по базе данных. Разделяет задачу поиска частых наборов на более мелкие подзадачи, решая каждую из них на основе этого компактного FP-дерева, без обращения к исходной базе данных.

Алгоритм FP-Growth был предложен как альтернатива Apriori для решения проблемы многократного сканирования базы данных и генерации большого числа кандидатов. Он использует специальную древовидную структуру — FP-дерево (Frequent Pattern tree) — для компактного представления информации о частых наборах.

Принцип работы алгоритма FP-Growth:

1. **Первое сканирование базы данных:**
   - Подсчитать поддержку каждого отдельного элемента.
   - Отобрать частые 1-элементы (те, чья поддержка $\geq$ *minsupp*).
   - Отсортировать частые 1-элементы в порядке убывания их поддержки. Этот порядок (F-list) будет использоваться при построении FP-дерева.

2. **Построение FP-дерева (FP-tree Construction):**
   - **Второе сканирование базы данных:** Для каждой транзакции:
     (a) Удалить из транзакции нечастые элементы.
     (b) Отсортировать оставшиеся (частые) элементы в транзакции согласно порядку F-list.
     (c) Вставить отсортированную транзакцию в FP-дерево.
   - **Структура FP-дерева:**
     – Каждый узел дерева соответствует элементу и хранит счетчик, показывающий, сколько транзакций проходят через этот узел.
     – Корень дерева — специальный узел (null).
     – Пути от корня к узлам представляют собой префиксы частых наборов. Общие префиксы транзакций используют общие пути в дереве, что обеспечивает сжатие.
     – Также поддерживается заголовочная таблица (header table), где для каждого частого элемента хранится указатель на первый узел с этим элементом в FP-дереве. Узлы с одинаковыми элементами связаны в списки через специальные указатели (node-links).

3. **Извлечение частых наборов из FP-дерева (Mining FP-tree):** Этот этап выполняется рекурсивно:
   - Для каждого частого элемента в заголовочной таблице (начиная с наименее частого в F-list):
     (a) Сформировать **условный префиксный путь (conditional prefix path)** для этого элемента. Это пути в FP-дереве, заканчивающиеся узлом с данным элементом.
     (b) Создать **условную базу шаблонов (conditional pattern base)** из этих префиксных путей, где каждый путь имеет вес, равный счетчику конечного узла элемента.
     (c) Построить **условное FP-дерево (conditional FP-tree)** из этой условной базы шаблонов (аналогично шагу 2, но для меньших данных).
     (d) Рекурсивно вызвать процедуру извлечения для условного FP-дерева.

(e) Частые наборы, заканчивающиеся текущим элементом, формируются путем объединения этого элемента со всеми частыми наборами, найденными в его условном FP-дереве.

- Базовый случай рекурсии: если FP-дерево содержит только один путь, то все комбинации элементов на этом пути являются частыми наборами.

Генерация правил из частых наборов: Аналогична той, что используется после Apriori.

Преимущества FP-Growth:

- Требует всего два сканирования базы данных (в основной части).
- Не генерирует кандидатов явно. Поиск частых наборов происходит путем построения и анализа меньших условных FP-деревьев.
- Обычно значительно быстрее Apriori, особенно для плотных данных и низких значений *minsupp*.
- Компактное представление данных в FP-дереве.

Недостатки FP-Growth:

- Построение FP-дерева может быть затратным по памяти, если дерево не умещается в оперативной памяти (хотя оно обычно компактнее исходной базы).
- Рекурсивный характер извлечения может быть сложен для понимания и реализации.

18 Машинное обучение: обучение с учителем. Задача классификации, способы оценки качества решений. Классификация текстов: применяемые при обучении признаки текстов, методы обучения классификаторов (на примере двух методов).

Определение: Обучение с учителем — это тип машинного обучения, при котором алгоритм обучается на размеченном наборе данных. Размеченный набор данных (обучающая выборка) состоит из пар "объект-ответ где "объект"— это набор признаков, описывающих экземпляр, а "ответ"— это известное целевое значение (метка класса, числовое значение и т.д.), которое алгоритм должен научиться предсказывать.

Основные задачи обучения с учителем:

- **Задача классификации (Classification):** Целевая переменная является категориальной, т.е. предсказывается метка класса из конечного набора (например, "спам"/"не спам "кошка"/"собака тип заболевания).
- **Задача регрессии (Regression):** Целевая переменная является непрерывной (числовой) (например, предсказание цены дома, температуры, спроса на товар).

Процесс обучения с учителем:

1. **Сбор данных:** Формирование обучающей выборки $D = \{(x_i, y_i)\}_{i=1}^N$, где x_i — вектор признаков i-го объекта, y_i — его известная метка (ответ).

2. **Выбор модели:** Выбор параметрического семейства функций (модели) $a(x, w)$, где w — вектор параметров модели.

3. **Обучение модели (Training):** Подбор таких параметров w, чтобы модель наилучшим образом описывала зависимость между объектами x_i и ответами y_i на обучающей выборке. Это часто сводится к минимизации некоторой функции потерь (loss function).

4. **Оценка качества:** Оценка производительности обученной модели на новых, ранее не виденных данных (тестовая выборка).

5. **Применение модели (Inference/Prediction):** Использование обученной модели для предсказания ответов для новых объектов.

2. Задача классификации

Определение: Задача классификации заключается в построении алгоритма (классификатора), способного отнести новый объект к одному из заранее известных классов на основе его признакового описания.

Формально: Дано:

- Множество объектов X.
- Множество меток классов $Y = \{c_1, c_2, \ldots, c_K\}$.
- Обучающая выборка $D = \{(x_i, y_i)\}_{i=1}^N$, где $x_i \in X$ — вектор признаков, $y_i \in Y$ — метка класса.

Требуется построить классификатор $a : X \to Y$, который для любого нового объекта x предсказывает его класс $y = a(x)$.

Типы классификации:

- **Бинарная классификация:** $K = 2$ (например, да/нет, спам/не спам).
- **Многоклассовая классификация:** $K > 2$.

3. Способы оценки качества решений в задаче классификации

Качество классификатора оценивается на отложенной (тестовой) выборке, которая не использовалась при обучении. Основные метрики:

- **Точность (Accuracy):** Доля правильно классифицированных объектов.
  $\text{Accuracy} = \frac{\text{Количество правильно классифицированных объектов}}{\text{Общее количество объектов}} = \frac{TP+TN}{TP+TN+FP+FN}$ (Для бинарной классификации: TP — истинно положительные, TN — истинно отрицательные, FP — ложно положительные, FN — ложно отрицательные). Не всегда информативна при несбалансированных классах.
- **Матрица ошибок (Confusion Matrix):** Таблица, показывающая количество правильных и неправильных предсказаний для каждого класса. Для бинарной классификации:

		Предсказанный класс	
		Положительный	Отрицательный
Истинный класс	Положительный	TP	FN
	Отрицательный	FP	TN

- **Точность (Precision) для класса k:** Доля объектов, правильно отнесенных к классу k, среди всех объектов, которые классификатор отнес к классу k. $\text{Precision}_k = \frac{TP_k}{TP_k+FP_k}$. Показывает, насколько можно доверять предсказанию класса k.
- **Полнота (Recall) для класса k:** Доля объектов класса k, которые были правильно классифицированы. $\text{Recall}_k = \frac{TP_k}{TP_k+FN_k}$. Показывает, насколько хорошо классификатор находит объекты класса k.
- **F-мера (F1-score) для класса k:** Гармоническое среднее между точностью и полнотой. $F_{1k} = 2 \cdot \frac{\text{Precision}_k \cdot \text{Recall}_k}{\text{Precision}_k + \text{Recall}_k}$. Используется для баланса между точностью и полнотой.
- **Макро- и микро-усреднение (Macro/Micro Averaging):** Способы усреднения метрик (Precision, Recall, F1) по всем классам в многоклассовой задаче.
  - **Макро-усреднение:** Метрика вычисляется для каждого класса, а затем усредняется (дает равный вес каждому классу).
  - **Микро-усреднение:** Метрика вычисляется по суммарным значениям TP, FP, FN по всем классам (дает больший вес более крупным классам).

4. Классификация текстов

Определение: Задача автоматического отнесения текстового документа к одной или нескольким предопределенным категориям (классам). Примеры: определение тематики новости, фильтрация спама, анализ тональности отзывов.

Применяемые при обучении признаки текстов

Тексты необходимо преобразовать в численное векторное представление (признаки), понятное для моделей машинного обучения.

- **Модель "Мешок слов"(Bag-of-Words, BoW):** Текст представляется как неупорядоченное множество слов (или n-грамм). Порядок слов игнорируется.
  - **Частота слова (Term Frequency, TF):** Количество вхождений слова в документ.
  - **Бинарное представление:** 1, если слово есть в документе, 0 — иначе.
  - **TF-IDF (Term Frequency - Inverse Document Frequency):** Мера, учитывающая как частоту слова в документе, так и его "редкость"во всей коллекции документов. $\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \cdot \text{IDF}(t, D)$, где $\text{IDF}(t, D) = \log \frac{|D|}{|\{d' \in D : t \in d'\}|}$ ($|D|$ — общее число документов, знаменатель — число документов, содержащих термин t). Слова, часто встречающиеся во многих документах (например, предлоги), получают низкий вес IDF.

  Каждый документ представляется вектором, где каждая компонента соответствует слову из общего словаря, а значение компоненты — это TF, TF-IDF или бинарный флаг. Размерность вектора равна размеру словаря.
- **N-граммы (N-grams):** Последовательности из N слов (или символов). Позволяют частично учесть локальный контекст.
- **Стемминг и лемматизация:** Приведение слов к их основной форме (стем — "обуч лемма — "обучение") для уменьшения размерности признакового пространства и группировки семантически близких слов.
- **Исключение стоп-слов (Stop-words removal):** Удаление часто встречающихся и малоинформативных слов (предлоги, союзы).
- **Векторные представления слов (Word Embeddings):** Плотные векторные представления слов (например, Word2Vec, GloVe, FastText), которые улавливают семантические отношения между словами. Документ может быть представлен как усредненный вектор его слов или с помощью более сложных архитектур (например, рекуррентных или сверточных сетей).
- **Структурные признаки:** Длина текста, количество предложений, использование заглавных букв, наличие URL, пунктуация и т.д.

Методы обучения классификаторов (на примере двух методов)

а) Наивный байесовский классификатор (Naive Bayes Classifier) **Идея:** Основан на теореме Байеса с "наивным"предположением о независимости признаков (слов) при условии класса. $P(y_k|x) = \frac{P(x|y_k)P(y_k)}{P(x)}$ Классификатор выбирает класс y_k, максимизирующий апостериорную вероятность $P(y_k|x)$: $a(x) = \arg\max_{y_k \in Y} P(y_k|x) = \arg\max_{y_k \in Y} P(x|y_k)P(y_k)$ (знаменатель $P(x)$ одинаков для всех классов и не влияет на argmax).

Наивное предположение о независимости признаков: Если $x = (x_1, x_2, \ldots, x_d)$ — вектор признаков (например, наличие/отсутствие слов или их частоты), то: $P(x|y_k) = P(x_1, x_2, \ldots, x_d|y_k) \approx \prod_{j=1}^{d} P(x_j|y_k)$

Обучение: Заключается в оценке вероятностей на основе обучающей выборки:

- **Априорные вероятности классов $P(y_k)$:** Оцениваются как доля документов класса y_k в обучающей выборке. $P(y_k) = \frac{\text{Количество документов класса } y_k}{\text{Общее количество документов}}$
- **Условные вероятности признаков $P(x_j|y_k)$:** Зависят от типа признаков.
  - **Мультиномиальная модель (Multinomial Naive Bayes):** Часто используется для текстов с TF-признаками. $P(x_j|y_k)$ — вероятность того, что j-е слово словаря встретится в документе класса y_k. Оценивается как: $P(x_j|y_k) = \frac{N_{kj}+\alpha}{N_k+\alpha\cdot|V|}$, где N_{kj} — число вхождений слова j в документы класса y_k, N_k — общее число слов во всех документах класса y_k, $|V|$ — размер словаря, α — параметр сглаживания (например, Лапласовское сглаживание, $\alpha = 1$). Сглаживание необходимо для обработки слов, не встречавшихся в обучающей выборке для данного класса.
  - **Бернуллиевская модель (Bernoulli Naive Bayes):** Используется для бинарных признаков (слово есть/нет). $P(x_j|y_k)$ — вероятность наличия j-го слова в документе класса y_k.

Классификация нового документа x: Вычисляется $\log P(y_k) + \sum_{j=1}^{d} \log P(x_j|y_k)$ для каждого класса y_k и выбирается класс с максимальным значением (логарифмирование используется для численной стабильности и замены произведений суммами).

Преимущества:

- Простота и скорость обучения и классификации.
- Хорошо работает на текстах, даже при нарушении предположения о независимости.
- Требует относительно мало данных для обучения.

Недостатки:

- "Наивное"предположение о независимости признаков редко выполняется на практике.
- Не учитывает взаимодействия между словами.

б) Логистическая регрессия (Logistic Regression) **Идея:** Моделирует вероятность принадлежности объекта к классу с помощью логистической функции (сигмоиды). Это линейный классификатор.

Для бинарной классификации ($y \in \{0, 1\}$): Вероятность того, что объект x принадлежит классу 1, моделируется как: $P(y = 1|x, w) = \sigma(w^T x + w_0) = \frac{1}{1+e^{-(w^T x+w_0)}}$ где w — вектор весов признаков, w_0 — свободный член (смещение), $\sigma(z) = \frac{1}{1+e^{-z}}$ — логистическая функция. $w^T x + w_0$ — это линейная комбинация признаков. Соответственно, $P(y = 0|x, w) = 1 - P(y = 1|x, w)$. Решение о классе принимается на основе порога (обычно 0.5): если $P(y = 1|x) > 0.5$, то класс 1, иначе класс 0.

Обучение: Параметры w и w_0 находятся путем максимизации логарифма правдоподобия на обучающей выборке или, что эквивалентно, минимизации функции потерь — кросс-энтропии (cross-entropy loss) или логистической функции потерь: $L(w, w_0) = -\sum_{i=1}^{N}[y_i \log P(y_i = 1|x_i) + (1 - y_i) \log P(y_i = 0|x_i)]$ Минимизация обычно выполняется численными методами оптимизации (например, градиентный спуск, L-BFGS). Для предотвращения переобучения часто используется регуляризация (L1 или L2).

Многоклассовая логистическая регрессия (Multinomial Logistic Regression / Softmax Regression): Для K классов, вероятность принадлежности к классу k моделируется с помощью функции softmax: $P(y = k|x, W) = \frac{e^{w_k^T x+w_{0k}}}{\sum_{j=1}^{K} e^{w_j^T x+w_{0j}}}$ где w_k — вектор весов для класса k. Обучение также сводится к минимизации кросс-энтропийной функции потерь.

Преимущества:

- Выдает вероятностные оценки принадлежности к классу.
- Относительно проста в интерпретации (веса признаков показывают их вклад).
- Эффективна и хорошо работает на многих задачах, включая классификацию текстов.
- Устойчива к переобучению при использовании регуляризации.

Недостатки:

- Является линейным классификатором, поэтому может плохо работать, если разделяющая поверхность между классами нелинейна (хотя можно использовать полиномиальные признаки или ядерные методы для обхода этого).
- Предполагает линейную зависимость логарифма отношения шансов от признаков.

19 Машинное обучение: обучение без учителя. Задача кластеризации, способы оценки качества кластеризации. Метод К-средних, иерархическая кластеризация. Кластеризация текстовой коллекции, применяемые признаки текстов.

Определение: Обучение без учителя — это тип машинного обучения, при котором алгоритм работает с неразмеченными данными, то есть входные данные не имеют заранее известных "правильных ответов"или меток классов. Цель алгоритма — самостоятельно выявить внутреннюю структуру, закономерности или скрытые зависимости в данных.

Основные задачи обучения без учителя:

- **Кластеризация (Clustering):** Группировка схожих объектов в кластеры так, чтобы объекты в одном кластере были максимально похожи друг на друга, а объекты из разных кластеров — максимально различны.
- **Уменьшение размерности (Dimensionality Reduction):** Сокращение числа признаков (переменных) в данных при сохранении наиболее важной информации. Это помогает упростить данные, устранить шум и улучшить производительность других ML-алгоритмов. Примеры: метод главных компонент (PCA), t-SNE.
- **Поиск ассоциативных правил (Association Rule Mining):** Выявление закономерностей вида "если A, то B"в больших наборах данных (например, анализ рыночных корзин).
- **Оценка плотности распределения (Density Estimation):** Построение модели распределения вероятностей для данных.
- **Обнаружение аномалий (Anomaly Detection):** Выявление объектов, которые значительно отличаются от остальных данных.

2. Задача кластеризации (Clustering)

Определение: Кластеризация — это задача разделения множества объектов (наблюдений, точек данных) на группы, называемые кластерами. Внутри каждого кластера объекты должны быть "похожи"друг на друга, а объекты из разных кластеров должны быть "непохожи". Степень схожести (или различия) определяется на основе выбранной метрики расстояния или схожести между объектами.

Цели кластеризации:

- Исследовательский анализ данных: выявление естественных группировок.
- Сегментация: например, сегментация клиентов, изображений.
- Обнаружение выбросов: объекты, не принадлежащие ни одному кластеру или образующие очень маленькие кластеры.
- Предварительная обработка данных: использование кластеров как признаков для других моделей.

Способы оценки качества кластеризации

Оценка качества кластеризации сложна, так как "правильных"меток нет. Существуют различные подходы:

1. **Внутренние меры (Internal Measures):** Оценивают качество кластеризации, используя только информацию из самих данных (без внешних меток). Они обычно измеряют компактность кластеров и их разделенность.
    - **Индекс Данна (Dunn Index):** Отношение минимального межкластерного расстояния к максимальному внутрикластерному расстоянию. Чем выше значение, тем лучше кластеризация.
    - **Индекс Дэвиса-Болдина (Davies-Bouldin Index):** Среднее значение схожести между каждым кластером и его наиболее похожим кластером. Чем меньше значение, тем лучше (кластеры более компактны и хорошо разделены).
    - **Коэффициент силуэта (Silhouette Coefficient):** Для каждого объекта измеряет, насколько он похож на объекты своего кластера по сравнению с объектами других (соседних) кластеров. Значение от -1 до 1. Близко к 1 — объект хорошо кластеризован; близко к 0 — на границе кластеров; близко к -1 — вероятно, в неправильном кластере. Средний коэффициент силуэта по всем объектам используется как общая мера.

2. **Внешние меры (External Measures):** Используются, когда доступны истинные метки классов для данных (например, для оценки алгоритма кластеризации на тестовом наборе или если кластеризация используется для задачи, где есть "ground truth"). Сравнивают результат кластеризации с этими истинными метками.

- **Индекс Рэнда (Rand Index, RI) и Скорректированный Индекс Рэнда (Adjusted Rand Index, ARI):** Измеряет сходство между двумя разбиениями (результатом кластеризации и истинными метками). ARI скорректирован на случайность и принимает значения от -1 до 1 (1 — идеальное совпадение).
- **Чистота (Purity):** Для каждого кластера определяется доминирующий истинный класс, и чистота вычисляется как доля объектов этого класса в кластере. Общая чистота — средневзвешенная по кластерам.
- **F-мера (F-measure):** Основана на точности (precision) и полноте (recall), адаптированных для задачи кластеризации.

3. Метод К-средних (K-Means)

Определение: K-means — это итеративный алгоритм кластеризации, который разделяет набор из n наблюдений на k заранее заданных непересекающихся кластеров. Каждый объект относится к кластеру с ближайшим средним значением (центроидом кластера).

Алгоритм:

1. **Инициализация:** Случайным образом выбрать k начальных центроидов кластеров из набора данных или сгенерировать их.

2. **Итерационный процесс (пока центроиды не перестанут изменяться или не будет достигнуто максимальное число итераций):**

   (a) **Шаг присвоения (Assignment step):** Каждый объект данных присваивается к кластеру, центроид которого ему ближайший (обычно используется евклидово расстояние).

   (b) **Шаг обновления (Update step):** Центроиды каждого кластера пересчитываются как среднее арифметическое (центр масс) всех объектов, отнесенных к этому кластеру на предыдущем шаге.

Особенности K-means:

- **Параметр k:** Необходимо заранее задать число кластеров k. Выбор k является отдельной задачей (например, метод "локтя"(elbow method), коэффициент силуэта).
- **Чувствительность к начальным центроидам:** Результат может зависеть от начального выбора центроидов. Для решения этой проблемы алгоритм часто запускают несколько раз с разными начальными положениями.
- **Форма кластеров:** K-means хорошо работает, когда кластеры имеют сферическую (или гиперсферическую) форму, примерно одинаковый размер и плотность. Плохо справляется с кластерами вытянутой формы, сложной геометрии или разного размера.
- **Сходимость:** Алгоритм гарантированно сходится к локальному минимуму суммы квадратов расстояний от объектов до центроидов их кластеров (WCSS).
- **Вычислительная эффективность:** Относительно быстрый, особенно для больших наборов данных. Сложность одной итерации $\mathcal{O}(n \cdot k \cdot d \cdot i)$, где n — число объектов, k — число кластеров, d — размерность данных, i — число итераций.

4. Иерархическая кластеризация (Hierarchical Clustering)

Определение: Иерархическая кластеризация строит иерархию (дерево) кластеров, называемую **дендрограммой**. Не требует заранее задавать число кластеров.

Типы иерархической кластеризации:

1. **Агломеративная (Agglomerative / Bottom-up):**
   - Начинает с того, что каждый объект является отдельным кластером.
   - На каждом шаге два наиболее близких кластера объединяются в один.
   - Процесс продолжается до тех пор, пока все объекты не окажутся в одном большом кластере.

2. **Дивизивная (Divisive / Top-down):**
   - Начинает с того, что все объекты находятся в одном кластере.
   - На каждом шаге один из существующих кластеров разделяется на два наименее похожих подкластера.

- Процесс продолжается до тех пор, пока каждый объект не станет отдельным кластером или не будет достигнуто заданное число кластеров. Дивизивная кластеризация вычислительно более сложна.

Меры расстояния между кластерами (Linkage Criteria) в агломеративной кластеризации: Определяют, как измеряется расстояние между двумя кластерами C_i и C_j:

- **Одиночная связь (Single Linkage / Nearest Neighbor):** Расстояние между ближайшими объектами из C_i и C_j. Может приводить к "эффекту цепочки"(длинные, вытянутые кластеры). $d(C_i, C_j) = \min_{x \in C_i, y \in C_j} \text{dist}(x, y)$.
- **Полная связь (Complete Linkage / Farthest Neighbor):** Расстояние между самыми удаленными объектами из C_i и C_j. Формирует более компактные, сферические кластеры. $d(C_i, C_j) = \max_{x \in C_i, y \in C_j} \text{dist}(x, y)$.
- **Метод Уорда (Ward's Method):** Объединяет те два кластера, слияние которых приводит к минимальному увеличению суммы квадратов расстояний внутри всех кластеров (WCSS). Стремится создавать компактные кластеры примерно одинакового размера.

Дендрограмма: Результат иерархической кластеризации — дендрограмма — это древовидная диаграмма, которая показывает последовательность слияний (или разделений) кластеров. Высота, на которой два кластера объединяются, обычно соответствует расстоянию между ними. "Разрезая"дендрограмму на определенной высоте, можно получить желаемое количество кластеров.

Особенности иерархической кластеризации:

- **Нет необходимости задавать k заранее:** Число кластеров определяется после построения дендрограммы.
- **Визуализация:** Дендрограмма дает наглядное представление о структуре данных.
- **Вычислительная сложность:** Обычно выше, чем у K-means. Для агломеративных методов не менее $\mathcal{O}(n^2 \log n)$ или $\mathcal{O}(n^2)$ для эффективных реализаций с некоторыми критериями связи, что делает их медленными для очень больших наборов данных.
- **Необратимость решений:** Однажды принятое решение об объединении (или разделении) кластеров не может быть отменено на последующих шагах.

5. Кластеризация текстовой коллекции

Кластеризация текстов — это задача группировки текстовых документов таким образом, чтобы документы в одном кластере были схожи по тематике или содержанию, а документы из разных кластеров — различны.

Применяемые признаки текстов (Представление текстов): Перед кластеризацией тексты необходимо преобразовать в числовое векторное представление.

1. **Предобработка текста:**
   - Токенизация (разбиение на слова или n-граммы).
   - Приведение к нижнему регистру.
   - Удаление стоп-слов (часто встречающихся слов, не несущих смысловой нагрузки, например, "и "в "на").
   - Стемминг или Лемматизация: приведение слов к их основной форме (стемминг — отсечение окончаний, лемматизация — приведение к словарной форме).

2. **Векторизация (Feature Extraction):**
   - **Модель "Мешок слов"(Bag-of-Words, BoW):** Текст представляется как неупорядоченный набор слов. Каждое уникальное слово в коллекции становится признаком.
     - **Частота слова (Term Frequency, TF):** Количество вхождений слова в документ.
     - **TF-IDF (Term Frequency-Inverse Document Frequency):** Взвешивание слов. $TF{-}IDF(t, d, D) = TF(t, d) \times IDF(t, D)$ $IDF(t, D) = \log \frac{|D|}{|\{d' \in D : t \in d'\}|}$ где $TF(t, d)$ — частота термина t в документе d, $IDF(t, D)$ — обратная документная частота термина t в коллекции D. TF-IDF присваивает больший вес словам, которые часто встречаются в данном документе, но редко в других документах коллекции.
   - **N-граммы (N-grams):** Последовательности из N слов или символов. Позволяют учитывать некоторый контекст (например, "машинное обучение"как биграмма).
   - **Векторные представления слов/документов (Word/Document Embeddings):** Современные методы, основанные на нейронных сетях (например, Word2Vec, GloVe, FastText, Doc2Vec, BERT-подобные модели). Они отображают слова или целые документы в плотные векторы низкой размерности, захватывая семантические отношения. Эти векторы затем могут использоваться для кластеризации.

Применение алгоритмов кластеризации к текстам:

- После векторизации текстов к ним могут быть применены стандартные алгоритмы кластеризации, такие как K-means или иерархическая кластеризация.
- **Метрика расстояния/схожести:** Для текстовых векторов часто используется **косинусное расстояние** (или косинусная схожесть), так как оно хорошо работает с высокоразмерными и разреженными данными (какими часто являются TF-IDF векторы) и измеряет схожесть направлений векторов, а не их магнитуду. $cosine_similarity(\vec{a}, \vec{b}) = \frac{\vec{a} \cdot \vec{b}}{||\vec{a}|| \cdot ||\vec{b}||}$ Косинусное расстояние = $1 - cosine_similarity$. Также может использоваться евклидово расстояние, особенно для плотных эмбеддингов.

Меры расстояний

Похожесть объектов определяется следующим образом: для начала нужно составить вектор характеристик для каждого объекта (набор числовых значений). Провести нормализацию, чтобы все компоненты давали одинаковый вклад. Для каждой пары объектов измеряется «расстояние» между ними — степень похожести. Существует множество метрик, основные из них:

– **Евклидово расстояние:**

$$\rho(x, x') = \sqrt{\sum_{i=1}^{n} (x_i - x'_i)^2}$$

– **Квадрат евклидова расстояния:**

$$\rho(x, x')^2 = \sum_{i=1}^{n} (x_i - x'_i)^2$$

– **Манхэттенское расстояние (расстояние города по кварталам):**

$$\rho(x, x') = \sum_{i=1}^{n} |x_i - x'_i|$$

– **Косинусное расстояние:**

$$\rho_{\cos}(x, x') = 1 - \frac{x \cdot x'}{\|x\| \cdot \|x'\|} = 1 - \frac{\sum_{i=1}^{n} x_i x'_i}{\sqrt{\sum_{i=1}^{n} x_i^2} \cdot \sqrt{\sum_{i=1}^{n} x_i'^2}}$$

20 Уровни естественного языка и основные этапы/модули многоуровневого анализа текста. Подходы к решению задач обработки текста: инженерный (на правилах) и основанный на машинном обучении. Лингвистические ресурсы, их виды и назначение.

1. Уровни естественного языка

Понимание и обработка естественного языка — сложный процесс, который обычно рассматривается на нескольких взаимосвязанных уровнях:

- **Фонология и Фонетика (Phonology & Phonetics):**
  - **Фонетика:** Изучает физические свойства звуков речи, их артикуляцию и восприятие.
  - **Фонология:** Изучает звуковую систему языка, то есть то, как звуки (фонемы) функционируют и организуются в языке для различения смысла.
  - В NLP это важно для систем распознавания и синтеза речи.
- **Морфология (Morphology):**
  - Изучает внутреннюю структуру слов и способы их образования (словообразование и словоизменение).
  - **Морфема:** Минимальная значащая единица языка (корень, приставка, суффикс, окончание).
  - Задачи: лемматизация (приведение слова к начальной форме), стемминг (отсечение окончаний/суффиксов), морфологический анализ (определение грамматических характеристик слова: часть речи, род, число, падеж и т.д.).
- **Лексикология (Lexicology) и Лексическая семантика (Lexical Semantics):**
  - Изучает словарный состав языка (лексику) и значение отдельных слов.
  - Включает такие явления, как синонимия, антонимия, омонимия, полисемия.
  - Задачи: разрешение лексической многозначности (Word Sense Disambiguation, WSD), идентификация именованных сущностей (Named Entity Recognition, NER).
- **Синтаксис (Syntax):**
  - Изучает правила построения фраз и предложений из слов. Описывает грамматическую структуру предложений.
  - Задачи: синтаксический анализ (парсинг) — построение дерева зависимостей или дерева составляющих, определение грамматических ролей слов в предложении (подлежащее, сказуемое, дополнение и т.д.).
- **Семантика (Semantics):**
  - Изучает значение языковых выражений (слов, фраз, предложений, текстов).
  - **Формальная семантика:** Пытается представить значение в виде формальных логических структур.
  - Задачи: извлечение смысла, семантический анализ ролей, построение семантических представлений текста.
- **Прагматика (Pragmatics):**
  - Изучает, как контекст (ситуация общения, знания говорящего и слушающего, цели коммуникации) влияет на интерпретацию языковых выражений.
  - Включает анализ речевых актов, импликатур, разрешение анафоры (связывание местоимений с их референтами), анализ дискурса (структуры текста большего, чем предложение).
- **Дискурс (Discourse):**
  - Изучает организацию и связность текста на уровне выше предложения (абзацы, главы, весь текст).
  - Включает анализ когерентности (смысловой связности) и когезии (формальной связности), структуры аргументации.

2. Основные этапы/модули многоуровневого анализа текста (Pipeline)

Типичный конвейер (pipeline) обработки текста включает последовательность модулей, каждый из которых решает задачи на определенном уровне языка. Результаты работы одного модуля часто служат входом для следующего.

1. **Сегментация текста (Text Segmentation):** Разделение текста на структурные единицы: предложения, слова (токены).
   - **Токенизация (Tokenization):** Разбиение текста на отдельные слова, знаки препинания и другие символы (токены).

- **Сегментация предложений (Sentence Splitting):** Определение границ предложений.

2. **Морфологический анализ (Morphological Analysis):** Анализ каждого токена для определения его базовой формы (леммы) и грамматических характеристик.
   - **Лемматизация:** Приведение слова к его словарной (начальной) форме (например, "бежал"→ "бежать").
   - **Стемминг:** Упрощенный вариант, отсечение суффиксов и окончаний для получения основы слова (например, "бежал"→ "беж"). Менее точен, чем лемматизация.
   - **Частеречная разметка (Part-of-Speech Tagging, POS-tagging):** Присвоение каждому слову его части речи (существительное, глагол, прилагательное и т.д.) с учетом контекста.

3. **Синтаксический анализ (Syntactic Analysis / Parsing):** Определение грамматической структуры предложений.
   - **Парсинг составляющих (Constituency Parsing):** Построение дерева, показывающего иерархическую структуру фразовых групп (например, именная группа, глагольная группа).
   - **Парсинг зависимостей (Dependency Parsing):** Построение дерева, показывающего синтаксические зависимости (отношения) между словами в предложении (например, определение-существительное, глагол-дополнение).

4. **Извлечение именованных сущностей (Named Entity Recognition, NER):** Идентификация и классификация именованных сущностей в тексте (например, имена людей, названия организаций, географические названия, даты, денежные суммы).

5. **Семантический анализ (Semantic Analysis):** Извлечение смысла из текста.
   - **Разрешение лексической многозначности (Word Sense Disambiguation, WSD):** Определение конкретного значения многозначного слова в данном контексте.
   - **Анализ семантических ролей (Semantic Role Labeling, SRL):** Определение семантических ролей участников ситуации, описываемой глаголом (кто что сделал, с кем, где, когда и т.д.).
   - **Построение семантических представлений:** Например, в виде графов знаний, логических форм.

6. **Анализ дискурса и прагматический анализ:**
   - **Разрешение анафоры и кореферентности (Anaphora/Coreference Resolution):** Определение, какие языковые выражения (например, местоимения, именные группы) ссылаются на один и тот же объект или сущность в тексте.
   - **Анализ тональности (Sentiment Analysis):** Определение эмоциональной окраски текста (позитивная, негативная, нейтральная).
   - **Извлечение отношений (Relation Extraction):** Идентификация семантических отношений между сущностями в тексте.

3. Подходы к решению задач обработки текста

а) Инженерный подход (на правилах, Rule-Based Approach)

- **Принцип:** Основан на создании наборов явных правил, вручную разработанных лингвистами или экспертами в предметной области. Эти правила описывают закономерности языка и способы их обработки.
- **Примеры правил:**
  - Регулярные выражения для поиска шаблонов в тексте.
  - Контекстно-свободные грамматики для синтаксического анализа.
  - Правила вида "если ... то ..."для классификации или извлечения информации.
- **Преимущества:**
  - Интерпретируемость: Правила понятны человеку, легко отлаживать и модифицировать.
  - Точность: Могут быть очень точными для узких областей или специфических задач, где закономерности хорошо известны.
  - Не требуют больших объемов размеченных данных для обучения.
- **Недостатки:**
  - Трудоемкость создания и поддержки: Разработка исчерпывающего набора правил для сложного языка требует значительных усилий.

- Хрупкость: Плохо справляются с вариативностью языка, ошибками, новыми конструкциями, которые не были учтены в правилах.
- Низкая масштабируемость на новые домены или языки.

б) Подход, основанный на машинном обучении (Machine Learning-Based Approach)

- **Принцип:** Модели NLP обучаются автоматически на больших объемах текстовых данных (корпусах). Алгоритмы машинного обучения выявляют статистические закономерности и шаблоны в данных и строят на их основе предсказательные модели.
- **Типы машинного обучения в NLP:**
  - **Обучение с учителем (Supervised Learning):** Требуются размеченные данные, где для каждого примера указан правильный ответ (например, POS-теги для слов, метки тональности для текстов). Алгоритмы: Naive Bayes, Logistic Regression, Support Vector Machines (SVM), нейронные сети.
  - **Обучение без учителя (Unsupervised Learning):** Работает с неразмеченными данными. Задачи: кластеризация текстов, тематическое моделирование (например, LDA), изучение векторных представлений слов (word embeddings, например, Word2Vec, GloVe, FastText).
  - **Обучение с частичным привлечением учителя (Semi-Supervised Learning):** Использует небольшое количество размеченных данных и большое количество неразмеченных.
  - **Обучение с подкреплением (Reinforcement Learning):** Используется в диалоговых системах, генерации текста.
- **Современные тенденции:** Глубокое обучение (Deep Learning) с использованием различных архитектур нейронных сетей (рекуррентные нейронные сети RNN/LSTM/GRU, сверточные нейронные сети CNN, трансформеры) достигло значительных успехов во многих задачах NLP.
- **Преимущества:**
  - Адаптивность и обобщающая способность: Модели могут хорошо работать с новыми, ранее не встречавшимися данными, если они похожи на обучающие.
  - Меньшие затраты на ручную разработку правил (но требуются усилия на сбор и разметку данных).
  - Способность улавливать сложные, неявные закономерности.
  - Часто показывают лучшую производительность на стандартных задачах по сравнению с системами на правилах.
- **Недостатки:**
  - Требовательность к данным: Нужны большие объемы качественных (часто размеченных) данных.
  - "Черный ящик": Некоторые модели (особенно глубокие нейронные сети) сложны для интерпретации, трудно понять, почему модель приняла то или иное решение.
  - Вычислительная сложность обучения: Обучение сложных моделей может требовать значительных вычислительных ресурсов.
  - Чувствительность к качеству данных: "Мусор на входе — мусор на выходе".

4. Лингвистические ресурсы, их виды и назначение

Лингвистические ресурсы — это наборы данных и инструментов, используемые для разработки и оценки NLP-систем.

Виды лингвистических ресурсов:

1. **Корпусы текстов (Corpora):** Большие коллекции текстов, собранные по определенным принципам. Могут быть:
   - **Неразмеченные (Raw Corpora):** Просто наборы текстов. Используются для обучения без учителя (например, для языковых моделей, word embeddings), статистического анализа языка.
   - **Размеченные (Annotated Corpora):** Тексты, в которых вручную или автоматически добавлена лингвистическая информация (аннотации).
     - *POS-теговые корпусы:* Слова размечены частями речи (например, Penn Treebank, Brown Corpus).
     - *Синтаксически размеченные корпусы (Treebanks):* Предложения размечены синтаксическими структурами (например, Penn Treebank, Universal Dependencies).
     - *Семантически размеченные корпусы:* Аннотации семантических ролей, значений слов (например, FrameNet, PropBank).

- *Корпусы с разметкой именованных сущностей.*
- *Параллельные корпусы:* Наборы текстов и их переводов на другие языки. Используются для машинного перевода.
- *Диалоговые корпусы, корпусы тональности* и т.д.

Назначение: Обучение NLP-моделей, тестирование и оценка их производительности, лингвистические исследования.

2. **Лексические ресурсы (Словари, Тезаурусы, Онтологии):**
   - **Электронные словари:** Содержат информацию о словах (значения, грамматические характеристики, примеры употребления).
   - **Тезаурусы:** Словари, в которых слова сгруппированы по смыслу, с указанием семантических отношений (синонимы, антонимы, гипонимы, гиперонимы и т.д.). Пример: WordNet.
   - **Онтологии:** Формальные спецификации концептуализации предметной области. Определяют классы (концепты), их свойства и отношения между ними. Примеры: Cyc, SUMO, доменные онтологии.
   - **Списки именованных сущностей (Gazetteers):** Списки известных имен, географических названий, организаций.

   Назначение: Разрешение лексической многозначности, извлечение информации, построение баз знаний, улучшение качества поиска.

3. **Грамматики и наборы правил:**
   - Формальные грамматики (например, контекстно-свободные грамматики, HPSG, LFG), описывающие синтаксис языка.
   - Наборы правил для морфологического анализа, разрешения анафоры и т.д.

   Назначение: Используются в системах, основанных на правилах, для парсинга, генерации текста.

21 Понятие языковой модели, виды моделей. Статистическая языковая модель: униграммные и N-граммные модели, применение в задачах обработки текстов. Сглаживание модели. Перплексия и способ её подсчета.

Определение: Языковая модель — это вероятностное распределение на последовательностях слов (или других языковых единиц, таких как символы). Для любой заданной последовательности слов $W = (w_1, w_2, \ldots, w_m)$ языковая модель вычисляет вероятность $P(W) = P(w_1, w_2, \ldots, w_m)$.

Основная задача: Предсказать следующее слово в последовательности, имея предыдущие слова. То есть, оценить $P(w_k|w_1, \ldots, w_{k-1})$. Вероятность всей последовательности можно разложить с использованием цепного правила вероятности: $P(w_1, w_2, \ldots, w_m) = P(w_1) \cdot P(w_2|w_1) \cdot P(w_3|w_1, w_2) \cdot \cdots \cdot P(w_m|w_1, \ldots, w_{m-1})$ $P(W) = \prod_{k=1}^{m} P(w_k|w_1, \ldots, w_{k-1})$

Виды языковых моделей:

1. **Статистические языковые модели (Statistical Language Models, SLM):** Основаны на подсчете частот встречаемости слов и последовательностей слов (N-грамм) в больших текстовых корпусах.
   - N-граммные модели (Unigram, Bigram, Trigram, etc.)
2. **Нейросетевые языковые модели (Neural Language Models, NLM):** Используют нейронные сети для изучения распределения вероятностей слов. Они способны улавливать более сложные и дальнодействующие зависимости в тексте.
   - Модели на основе рекуррентных нейронных сетей (RNN, LSTM, GRU).
   - Модели на основе архитектуры Трансформер (Transformer), например, BERT, GPT, XLNet. Эти модели часто являются предобученными на огромных корпусах и затем дообучаются (fine-tuning) для конкретных задач.

2. Статистическая языковая модель: униграммные и N-граммные модели

N-граммные модели (N-gram Models)

N-грамма — это последовательность из N слов. Предположение: вероятность слова w_k зависит только от $N-1$ непосредственно предшествующих ему слов: $P(w_k|w_1, \ldots, w_{k-1}) \approx P(w_k|w_{k-N+1}, \ldots, w_{k-1})$

Расчет вероятностей N-грамм (метод максимального правдоподобия): Вероятность N-граммы $P(w_k|w_{k-N+1}$ оценивается как относительная частота встречаемости этой N-граммы по отношению к частоте встречаемости ее префикса (контекста) из $N-1$ слов в обучающем корпусе: $P(w_k|w_{k-N+1}, \ldots, w_{k-1}) = \frac{\text{Count}(w_{k-N+1}, \ldots, w_{k-1}, w_k)}{\text{Count}(w_{k-N+1}, \ldots, w_{k-1})}$ где $\text{Count}(\cdot)$ — количество раз, которое данная последовательность слов встретилась в корпусе.

а) Униграммная модель (Unigram Model, N=1)

Предполагает, что каждое слово в последовательности независимо от других слов. Вероятность слова зависит только от самого слова. $P(w_k|w_1, \ldots, w_{k-1}) \approx P(w_k)$ $P(W) = \prod_{k=1}^{m} P(w_k)$ Вероятность $P(w_k)$ оценивается как: $P(w_k) = \frac{\text{Count}(w_k)}{\sum_{w'} \text{Count}(w')} = \frac{\text{Count}(w_k)}{N_{\text{total_words}}}$ Эта модель не учитывает контекст и порядок слов, поэтому ее предсказательная сила низка. Используется как базовая линия или в задачах, где порядок не важен (например, "мешок слов").

б) Биграммная модель (Bigram Model, N=2)

Предполагает, что вероятность слова w_k зависит только от одного предыдущего слова w_{k-1}. $P(w_k|w_1, \ldots, w_{k-1}) \approx P(w_k|w_{k-1})$ $P(W) = P(w_1) \cdot \prod_{k=2}^{m} P(w_k|w_{k-1})$ (Часто добавляют специальные символы начала <s> и конца </s> предложения: $P(W) = \prod_{k=1}^{m+1} P(w_k|w_{k-1})$, где $w_0 = $ <s>, $w_{m+1} = $ </s>) Вероятность $P(w_k|w_{k-1})$ оценивается как: $P(w_k|w_{k-1}) = \frac{\text{Count}(w_{k-1}, w_k)}{\text{Count}(w_{k-1})}$

в) Триграммная модель (Trigram Model, N=3)

Предполагает, что вероятность слова w_k зависит от двух предыдущих слов w_{k-2}, w_{k-1}. $P(w_k|w_1, \ldots, w_{k-1}) \approx P(w_k|w_{k-2}, w_{k-1})$ $P(W) = P(w_1)P(w_2|w_1) \prod_{k=3}^{m} P(w_k|w_{k-2}, w_{k-1})$ (С использованием символов начала предложения: $P(W) = \prod_{k=1}^{m+1} P(w_k|w_{k-2}, w_{k-1})$, где $w_{-1} = w_0 = $ <s>, $w_{m+1} = $ </s>) Вероятность $P(w_k|w_{k-2}, w_{k-1})$ оценивается как: $P(w_k|w_{k-2}, w_{k-1}) = \frac{\text{Count}(w_{k-2}, w_{k-1}, w_k)}{\text{Count}(w_{k-2}, w_{k-1})}$

Увеличение N позволяет учитывать более длинный контекст, что улучшает качество модели, но приводит к проблемам:

- **Разреженность данных (Data Sparsity):** Длинные N-граммы встречаются в корпусе редко или вообще не встречаются. Это приводит к нулевым вероятностям для многих допустимых последовательностей.
- **Рост размера модели:** Количество возможных N-грамм растет экспоненциально с N.

Применение N-граммных моделей в задачах обработки текстов

- **Распознавание речи:** Выбор наиболее вероятной последовательности слов из нескольких акустически похожих вариантов.
- **Машинный перевод:** Оценка "гладкости"и грамматической правильности переведенного текста.
- **Проверка орфографии и грамматики (Spell Checking / Grammar Checking):** Определение вероятности слова или последовательности слов в данном контексте.
- **Автодополнение (Text Prediction / Autocompletion):** Предсказание следующего слова или фразы.
- **Классификация текстов, информационный поиск:** В качестве признаков или для оценки релевантности.

3. Сглаживание модели (Smoothing)

Проблема нулевых вероятностей (когда N-грамма не встретилась в обучающем корпусе, но может встретиться в тестовом) является критической для N-граммных моделей. Сглаживание — это набор техник для перераспределения вероятностной массы от часто встречающихся N-грамм к редко встречающимся или не встречавшимся N-граммам, чтобы избежать нулевых вероятностей.

Основные методы сглаживания:

1. **Сглаживание Лапласа (Add-one Smoothing):** К счетчику каждой N-граммы добавляется 1 (или небольшое число δ, Add-δ Smoothing). $P_{\text{Laplace}}(w_k|w_{k-N+1},\ldots,w_{k-1}) = \frac{\text{Count}(w_{k-N+1},\ldots,w_{k-1},w_k)+1}{\text{Count}(w_{k-N+1},\ldots,w_{k-1})+V}$ где V — размер словаря (количество уникальных слов). *Недостаток:* Сильно переоценивает вероятности редких N-грамм, особенно для больших словарей.

2. **Интерполяция (Interpolation):** Вероятность N-граммы вычисляется как взвешенная сумма вероятностей, полученных из моделей разного порядка (например, триграммной, биграммной и униграммной). $P_{\text{interp}}(w_k|w_{k-1},w_{k-2}) = \lambda_3 P(w_k|w_{k-1},w_{k-2}) + \lambda_2 P(w_k|w_{k-1}) + \lambda_1 P(w_k)$ где $\lambda_1+\lambda_2+\lambda_3=1$, $\lambda_i \geq 0$. Веса λ_i могут быть фиксированными или обучаться на отложенной выборке (held-out data).

3. **Отступление (Backoff):** Если N-грамма высокого порядка имеет нулевой или очень низкий счетчик, модель "отступает"к N-грамме меньшего порядка. Например, если $P(w_k|w_{k-1},w_{k-2})$ не может быть надежно оценена, используется $P(w_k|w_{k-1})$ с некоторым весом отступления.

4. Перплексия и способ её подсчета

Определение перплексии (Perplexity, PP): Перплексия — это стандартная метрика для оценки качества языковой модели. Она измеряет, насколько хорошо модель предсказывает тестовую выборку. Чем ниже перплексия, тем лучше модель. Интуитивно, перплексия — это средний "фактор ветвления"(число возможных следующих слов), который модель рассматривает на каждом шаге предсказания.

Формула для расчета перплексии: Для тестовой последовательности слов $W=(w_1,w_2,\ldots,w_M)$ перплексия вычисляется как: $\text{PP}(W) = P(w_1,w_2,\ldots,w_M)^{-\frac{1}{M}}$ $PP(W) = \sqrt[M]{\frac{1}{P(w_1,w_2,\ldots,w_M)}} = \left(\prod_{i=1}^{M} P(w_i|w_1,\ldots,w_{i-1})\right)^{-\frac{1}{M}}$

Часто используется логарифмическая форма для удобства вычислений и избежания проблем с исчезающе малыми вероятностями: $\log_2 \text{PP}(W) = -\frac{1}{M}\sum_{i=1}^{M}\log_2 P(w_i|w_1,\ldots,w_{i-1})$ $\text{PP}(W) = 2^{-\frac{1}{M}\sum_{i=1}^{M}\log_2 P(w_i|w_1,\ldots,w_{i-1})}$ Сумма $\sum_{i=1}^{M}\log_2 P(w_i|w_1,\ldots,w_{i-1})$ — это логарифм правдоподобия тестового набора, а $-\frac{1}{M}\sum\ldots$ — это средняя отрицательная логарифмическая вероятность на слово, или кросс-энтропия между эмпирическим распределением тестовых данных и предсказаниями модели.

Способ подсчета:

1. Взять тестовый корпус, который не использовался для обучения модели.

2. Для каждой последовательности (предложения) W_j в тестовом корпусе вычислить ее вероятность $P(W_j)$ с помощью языковой модели.

3. Объединить все предложения тестового корпуса в одну длинную последовательность слов W_{test} длины M_{total}.

4. Рассчитать общую вероятность $P(W_{test}) = \prod_j P(W_j)$.

5. Вычислить перплексию по формуле: PP$(W_{test}) = P(W_{test})^{-\frac{1}{M_{total}}}$.

Интерпретация перплексии:

- Если перплексия равна k, это означает, что модель на каждом шаге так же неопределенна в выборе следующего слова, как если бы ей приходилось выбирать из k равновероятных слов.
- Например, если словарь состоит из 100000 слов, и модель имеет перплексию 100, это значительно лучше, чем случайное угадывание (перплексия 100000).
- Перплексия зависит от задачи и сложности языка/домена. Сравнивать перплексии имеет смысл только для моделей, обученных и протестированных на одинаковых данных и с одинаковым словарем.

Перплексия является внутренней (intrinsic) метрикой оценки языковой модели. Для оценки реальной полезности модели также важны внешние (extrinsic) метрики, показывающие, как модель справляется с конкретной NLP-задачей (например, точность перевода, процент ошибок в распознавании речи).

22 Дистрибутивная семантика в автоматической обработке текстов: понятие эмбеддинга (вектора слова). Нейросетевые языковые модели Word2Vec и FastText: способы обучения, семантические пространства слов.

Дистрибутивная семантика — это подход к пониманию и представлению значения слов, основанный на идее, что значение слова определяется его окружением или контекстом, в котором оно встречается.

1. Дистрибутивная семантика и понятие эмбеддинга (вектора слова)

Дистрибутивная гипотеза: Слова, которые встречаются в схожих контекстах, имеют схожие значения. Например, слова "кошка"и "собака"часто встречаются в контекстах, связанных с "домашними животными "кормом "игрой "лаем/мяуканьем поэтому их значения близки.

Эмбеддинг (Embedding) / Вектор слова (Word Vector): Значение слова представляется в виде плотного числового вектора в многомерном пространстве. Этот вектор называется **эмбеддингом** или **вектором слова**.

- **Плотное представление (Dense Representation):** Векторы имеют относительно небольшую размерность (например, 50-1000) по сравнению с разреженными представлениями (например, one-hot encoding, где размерность равна размеру словаря). Каждая компонента вектора — это действительное число.
- **Распределенное представление (Distributed Representation):** Значение слова "распределено"по многим компонентам вектора, и каждая компонента может участвовать в представлении многих слов.
- **Обучение на данных:** Эмбеддинги слов обычно обучаются на больших текстовых корпусах с использованием методов машинного обучения, в частности, нейронных сетей.

Свойства векторных представлений слов:

- **Семантическая близость:** Слова с похожими значениями имеют близкие векторы в пространстве (например, по косинусному расстоянию или евклидову расстоянию).
- **Линейные аналогии:** Векторные пространства, полученные некоторыми методами (например, Word2Vec), могут улавливать семантические и синтаксические аналогии. Классический пример: vector("король) − vector("мужчина) + vector("женщина) ≈ vector("королева).
- **Использование в качестве признаков:** Эмбеддинги слов служат эффективными признаками для различных задач NLP (классификация текстов, анализ тональности, машинный перевод и т.д.), так как они несут в себе семантическую информацию.

Отличие от традиционных подходов: В отличие от символьных подходов, где слова рассматриваются как дискретные единицы, дистрибутивная семантика позволяет работать со словами в непрерывном векторном пространстве, что облегчает измерение семантической схожести и обобщение на новые контексты.

2. Нейросетевые языковые модели Word2Vec и FastText

Word2Vec и FastText — это популярные модели для обучения векторных представлений слов (эмбеддингов) на основе нейронных сетей. Они относятся к классу моделей "предсказательного"типа (predictive models), где задача обучения заключается в предсказании слова по его контексту или контекста по слову.

Word2Vec

Word2Vec включает два основных архитектурных подхода: CBOW и Skip-gram.

a) CBOW (Continuous Bag-of-Words / Непрерывный мешок слов)

- **Задача:** Предсказать текущее (центральное) слово на основе его окружающих слов (контекста).
- **Архитектура:**
  1. **Входной слой:** Контекстные слова (например, k слов до и k слов после центрального слова) представляются в виде one-hot векторов.
  2. **Слой проекции (Embedding Layer):** Каждый one-hot вектор контекстного слова умножается на общую матрицу весов W (матрица эмбеддингов). Получаются векторы эмбеддингов для каждого контекстного слова.
  3. Векторы эмбеддингов контекстных слов усредняются (или суммируются) для получения единого вектора контекста.
  4. **Выходной слой:** Полученный вектор контекста умножается на другую матрицу весов W' (выходная матрица эмбеддингов). Результат проходит через функцию softmax для получения распределения вероятностей по всему словарю для центрального слова.

- **Обучение:** Модель обучается минимизировать ошибку предсказания (например, кросс-энтропию) между предсказанным распределением и реальным центральным словом (представленным как one-hot вектор). В процессе обучения настраиваются веса матриц W и W'. Матрица W и есть искомая матрица эмбеддингов слов.
- **Особенности:** CBOW обычно быстрее обучается и лучше работает для часто встречающихся слов.

б) Skip-gram (Пропуск грамм)

- **Задача:** Предсказать окружающие слова (контекст) на основе текущего (центрального) слова.
- **Архитектура:**
  1. **Входной слой:** Центральное слово представляется в виде one-hot вектора.
  2. **Слой проекции (Embedding Layer):** One-hot вектор центрального слова умножается на матрицу весов W, получая его эмбеддинг.
  3. **Выходной слой:** Эмбеддинг центрального слова умножается на матрицу W'. Для каждого слова из контекстного окна применяется softmax для получения распределения вероятностей.
- **Обучение:** Модель обучается максимизировать вероятность правильных контекстных слов. Для каждого центрального слова модель пытается предсказать несколько слов из его контекста.
- **Особенности:** Skip-gram обычно медленнее обучается, но лучше работает для редких слов и для больших корпусов. Часто дает более качественные эмбеддинги.

Техники оптимизации для Word2Vec: Поскольку вычисление softmax по всему словарю (который может быть очень большим) на каждом шаге очень затратно, используются техники оптимизации:

- **Иерархический Softmax (Hierarchical Softmax):** Вместо плоского softmax используется дерево (обычно бинарное дерево Хаффмана), где слова являются листьями. Вероятность слова вычисляется как произведение вероятностей на пути от корня к листу.
- **Негативное сэмплирование (Negative Sampling):** Для каждого обучающего примера (например, пара "центральное слово - контекстное слово") выбирается несколько "негативных"(неправильных) контекстных слов. Модель обучается отличать правильную пару от негативных пар (задача бинарной классификации). Это значительно эффективнее, чем полный softmax.

FastText

FastText является расширением модели Word2Vec (в частности, Skip-gram).

Ключевая идея: Представление слова как "мешка"N-грамм символов (character n-grams).

- Каждое слово разбивается на N-граммы символов. Например, для слова "apple"и N=3, N-граммы могут быть: «ap "app "ppl "ple "le>"(специальные символы начала < и конца > слова добавляются).
- Также само слово целиком рассматривается как одна из его N-грамм.
- Вектор слова (эмбеддинг) получается как сумма векторов его N-грамм символов.

Преимущества FastText:

1. **Работа с редкими словами и словами вне словаря (Out-of-Vocabulary, OOV):** FastText может генерировать эмбеддинги для слов, которые не встречались в обучающем корпусе, на основе эмбеддингов их N-грамм символов. Word2Vec присваивает таким словам случайный вектор или специальный OOV-вектор.

2. **Учет морфологии:** Так как модель работает с N-граммами символов, она способна улавливать морфологическую структуру слов. Например, слова с одинаковыми приставками или суффиксами будут иметь схожие компоненты в своих эмбеддингах, что отражает их морфологическую близость. Это особенно полезно для языков с богатой морфологией.

3. **Скорость обучения:** FastText часто обучается быстрее, чем Word2Vec (особенно Skip-gram без оптимизаций) при сопоставимом качестве на задачах семантической схожести.

Способы обучения FastText: FastText может обучаться аналогично Word2Vec (CBOW или Skip-gram) с использованием иерархического softmax или негативного сэмплирования. Основное отличие — на вход подаются не целые слова, а их N-граммы символов. Также FastText известен как эффективный инструмент для классификации текстов, где весь текст представляется как усредненный вектор эмбеддингов его слов (или N-грамм слов), и затем этот вектор подается на вход линейного классификатора.

3. Семантические пространства слов

Обученные эмбеддинги (Word2Vec, FastText или другие) формируют **семантическое пространство**, где каждое слово представлено точкой (вектором).

- **Структура пространства:** Расположение точек в этом пространстве отражает семантические отношения между словами.
  - **Близость:** Слова с похожими значениями находятся близко друг к другу.
  - **Направления:** Определенные направления в пространстве могут соответствовать семантическим отношениям. Например, вектор от "мужчина"к "женщина"может быть похож на вектор от "король"к "королева"или от "дядя"к "тетя".
- **Визуализация:** Для визуализации многомерных семантических пространств используются методы понижения размерности, такие как t-SNE или PCA, которые проецируют векторы на 2D или 3D плоскость.
- **Арифметические операции с векторами:** Как упоминалось ранее, некоторые модели (особенно Skip-gram) позволяют выполнять осмысленные арифметические операции с векторами слов для решения задач на аналогии.
- **Контекстуализированные эмбеддинги:** Модели типа Word2Vec и FastText генерируют статические эмбеддинги — одно векторное представление для каждого слова, независимо от контекста. Более современные модели (ELMo, BERT, GPT) генерируют **контекстуализированные эмбеддинги**, где вектор слова зависит от окружающего его контекста, что позволяет лучше справляться с многозначностью слов.

23 Современные нейросетевые языковые модели: особенности архитектуры и обучения, области применения. Понятие переноса обучения (Transfer Learning) и тонкой настройки (Fine-Tuning) на примере модели BERT.

В отличие от традиционных статистических N-граммных моделей, которые ограничены локальным контекстом, современные NLM, особенно те, что основаны на архитектуре Трансформер, могут учитывать дальние зависимости в тексте и генерировать контекстуализированные представления слов.

Особенности архитектуры

Ключевые компоненты Трансформера:

- **Механизм внимания (Attention Mechanism):** Позволяет модели взвешивать важность различных частей входной последовательности при обработке каждого элемента.
  - **Само-внимание (Self-Attention):** Позволяет каждому слову в последовательности "обращать внимание"на другие слова в той же последовательности, вычисляя контекстуализированное представление для каждого слова. Модель учится определять, какие слова наиболее релевантны для понимания текущего слова.
  - **Многоголовое внимание (Multi-Head Attention):** Выполняет механизм внимания параллельно несколько раз с разными наборами весов ("головами"), а затем объединяет результаты. Это позволяет модели одновременно фокусироваться на разных аспектах зависимостей.
- **Энкодер-декодерная структура (Encoder-Decoder Architecture):** Изначально Трансформер был предложен для задач последовательность-в-последовательность (seq2seq), таких как машинный перевод.
  - **Энкодер:** Обрабатывает входную последовательность и создает ее скрытое представление. Состоит из стека одинаковых слоев, каждый из которых содержит слой само-внимания и полносвязную нейронную сеть (feed-forward network).
  - **Декодер:** Генерирует выходную последовательность на основе представления, полученного от энкодера. Также состоит из стека слоев, но каждый слой декодера дополнительно включает механизм внимания к выходу энкодера (encoder-decoder attention).
- **Позиционное кодирование (Positional Encoding):** Поскольку Трансформер не имеет встроенных механизмов для учета порядка слов (в отличие от RNN), информация о позиции слов добавляется к их входным эмбеддингам.
- **Остаточные соединения (Residual Connections) и Нормализация слоев (Layer Normalization):** Используются для облегчения обучения глубоких сетей и стабилизации градиентов.

Примеры архитектур на основе Трансформера:

- **Модели только на энкодерах (Encoder-only):** Например, BERT, RoBERTa, ALBERT. Хорошо подходят для задач понимания языка (NLU), таких как классификация текста, извлечение именованных сущностей, ответ на вопросы.
- **Модели только на декодерах (Decoder-only):** Например, GPT (Generative Pre-trained Transformer), XLNet (частично). Оптимизированы для генерации текста (NLG).
- **Энкодер-декодерные модели (Encoder-Decoder):** Например, T5, BART, исходный Трансформер. Используются для задач seq2seq, таких как машинный перевод, суммаризация текста.

Особенности обучения

Современные NLM обычно обучаются в два этапа:

1. **Предобучение (Pre-training):** Модель обучается на очень больших неразмеченных текстовых корпусах (например, Википедия, Common Crawl) на задачах самообучения (self-supervised learning). Цель — научить модель общим языковым закономерностям.
   - **Задачи предобучения для энкодерных моделей (например, BERT):**
     - **Masked Language Modeling (MLM):** Часть слов во входной последовательности случайным образом маскируется (заменяется на специальный токен [MASK]), и модель должна предсказать исходные замаскированные слова на основе окружающего контекста.
     - **Next Sentence Prediction (NSP) (в BERT, позже признана менее эффективной):** Модели подается пара предложений, и она должна предсказать, является ли второе предложение логическим продолжением первого.
   - **Задачи предобучения для декодерных моделей (например, GPT):**

- **Стандартное языковое моделирование (Causal Language Modeling, CLM):** Модель предсказывает следующее слово в последовательности, имея доступ только к предыдущим словам (авторегрессионный подход).

2. **Дообучение (Fine-tuning) / Тонкая настройка:** Предобученная модель адаптируется для решения конкретной прикладной задачи NLP. На этом этапе модель дообучается на небольшом наборе размеченных данных, специфичных для этой задачи. Веса модели, полученные на этапе предобучения, служат хорошей инициализацией.

Этот двухэтапный подход (предобучение + дообучение) является ключевым для успеха современных NLM и называется **переносом обучения (Transfer Learning)**.

2. Понятие переноса обучения (Transfer Learning)

Определение: Перенос обучения — это методология машинного обучения, при которой знания, полученные при решении одной задачи (исходной, source task), используются для улучшения обучения или производительности на другой, но связанной с ней задаче (целевой, target task).

В контексте NLP:

- **Исходная задача:** Обучение языковой модели на общих задачах (MLM, CLM) на больших неразмеченных корпусах. Модель учит общие представления о синтаксисе, семантике, фактах о мире, содержащихся в текстах.
- **Целевая задача:** Конкретная задача NLP (например, классификация тональности отзывов о фильмах).
- **Переносимые знания:** Веса нейронной сети (параметры модели), полученные на этапе предобучения. Эти веса кодируют "понимание"языка.

Преимущества переноса обучения в NLP:

- **Снижение потребности в больших размеченных датасетах для целевой задачи:** Предобученная модель уже обладает значительными "знаниями"о языке, поэтому для адаптации к новой задаче требуется гораздо меньше специфических размеченных данных.
- **Улучшение качества моделей:** Использование предобученных весов часто приводит к более высокой точности и лучшей обобщающей способности на целевой задаче.
- **Ускорение процесса обучения на целевой задаче:** Модель начинает не "с нуля а с хорошей точки инициализации.
- **Демократизация NLP:** Позволяет исследователям и разработчикам без доступа к огромным вычислительным ресурсам (необходимым для предобучения с нуля) использовать мощные модели, просто дообучая уже существующие предобученные версии.

3. Тонкая настройка (Fine-Tuning) на примере модели BERT

BERT (Bidirectional Encoder Representations from Transformers): BERT — это языковая модель, основанная на архитектуре Трансформер (только энкодеры), предобученная на задачах Masked Language Modeling (MLM) и Next Sentence Prediction (NSP). Ключевая особенность BERT — его способность учитывать контекст с обеих сторон (слева и справа) от слова при построении его представления.

Процесс тонкой настройки BERT для конкретной задачи:

1. **Выбор предобученной модели BERT:** Существуют различные версии BERT (например, BERT-Base, BERT-Large, многоязычные версии).

2. **Модификация архитектуры для целевой задачи:** К выходным представлениям BERT (обычно к эмбеддингу специального токена [CLS] для задач классификации предложений/пар предложений, или к эмбеддингам каждого токена для задач разметки последовательностей) добавляется один или несколько дополнительных слоев, специфичных для задачи.

   - **Для классификации текста/предложений:** Добавляется полносвязный слой с softmax-активацией поверх эмбеддинга [CLS].
   - **Для разметки последовательностей (например, NER, POS-tagging):** Добавляется полносвязный слой с softmax-активацией поверх эмбеддинга каждого токена.
   - **Для ответа на вопросы (например, SQuAD):** Добавляются слои для предсказания начала и конца ответа в тексте.

3. **Подготовка данных для целевой задачи:** Входные данные форматируются в соответствии с требованиями BERT (специальные токены [CLS], [SEP], токенизация WordPiece, создание масок внимания и т.д.).

4. **Обучение (Fine-tuning):** Вся модель (изначальные веса BERT и веса новых добавленных слоев) дообучается на размеченных данных целевой задачи.
   - Используется стандартный алгоритм обратного распространения ошибки.
   - Обычно используются небольшие скорости обучения (learning rate), чтобы не "разрушить"знания, полученные на этапе предобучения.
   - Обучение проводится в течение небольшого числа эпох.

5. **Оценка и применение:** Модель оценивается на тестовом наборе целевой задачи.

Пример тонкой настройки BERT для классификации предложений (например, анализ тональности):

1. Входное предложение: "Этот фильм был великолепен!"

2. Форматирование входа для BERT: [CLS] Этот фильм был великолепен ! [SEP]

3. BERT генерирует контекстуализированные эмбеддинги для каждого токена.

4. Эмбеддинг токена [CLS] (который агрегирует информацию обо всем предложении) подается на вход полносвязного классификационного слоя.

5. Классификационный слой предсказывает метку класса (например, "позитивный "негативный").

6. В процессе тонкой настройки веса BERT и классификационного слоя обновляются для минимизации ошибки классификации на обучающих примерах.

Варианты Fine-tuning:
- **Полное дообучение:** Обновляются все параметры модели.
- **Частичное дообучение / "Заморозка"слоев (Feature Extraction):** Обновляются только параметры добавленных слоев, а веса основной части предобученной модели (например, BERT) остаются неизменными. Это полезно, когда данных для целевой задачи очень мало. BERT используется как экстрактор признаков.

24 Машинный перевод: основные технологии автоматического перевода текста, их ограничения. Статистические и нейросетевые переводчики, принципы их построения.

а) Машинный перевод на основе правил (Rule-Based Machine Translation, RBMT)

- **Принцип:** Основан на явном кодировании лингвистических правил для анализа исходного текста и синтеза целевого текста. Требует большого количества двуязычных словарей и грамматических правил.
- **Подходы внутри RBMT:**
  - **Прямой (Direct) перевод:** Перевод слово в слово с минимальными локальными перестановками. Ранний и самый простой подход.
  - **Трансферный (Transfer-based) перевод:** Включает три этапа:
    1. Анализ исходного текста для получения его лингвистического представления (например, синтаксической структуры).
    2. Трансформация этого представления в эквивалентное представление на целевом языке с использованием двуязычных правил.
    3. Синтез (генерация) текста на целевом языке из его представления.
  - **Интерлингва (Interlingua-based) перевод:** Исходный текст переводится в некоторое универсальное, независимое от языка семантическое представление (интерлингву), а затем из этого представления генерируется текст на целевом языке. Наиболее амбициозный, но сложный в реализации.
- **Ограничения RBMT:**
  - **Трудоемкость создания правил:** Требует огромных усилий лингвистов для разработки и поддержки правил и словарей для каждой языковой пары.
  - **Хрупкость и неполнота покрытия:** Сложно учесть все языковые нюансы, идиомы, исключения и вариативность языка. Системы плохо справляются с неграмматичным или неожиданным вводом.
  - **Низкая адаптивность:** Трудно адаптировать к новым доменам или стилям текста.
  - **Качество перевода:** Часто переводы получаются грамматически корректными, но неестественными или с потерей смысла из-за недостаточной глубины семантического анализа.

б) Статистический машинный перевод (Statistical Machine Translation, SMT)

- **Принцип:** Основан на обучении вероятностных моделей перевода на больших параллельных корпусах (текстах на исходном языке и их переводах на целевой язык). Модель учится находить наиболее вероятный перевод для данного исходного предложения.
- **Фундаментальное уравнение SMT (по теореме Байеса):** Цель — найти такой перевод $\mathbf{e}$ (на целевом языке), который максимизирует вероятность $P(\mathbf{e}|\mathbf{f})$ для данного исходного предложения $\mathbf{f}$. $P(\mathbf{e}|\mathbf{f}) = \frac{P(\mathbf{f}|\mathbf{e}) \cdot P(\mathbf{e})}{P(\mathbf{f})}$ Поскольку $P(\mathbf{f})$ постоянно для данного $\mathbf{f}$, задача сводится к максимизации $P(\mathbf{f}|\mathbf{e}) \cdot P(\mathbf{e})$: $\mathbf{e}^* = \arg\max_{\mathbf{e}} P(\mathbf{f}|\mathbf{e}) \cdot P(\mathbf{e})$
  - $P(\mathbf{f}|\mathbf{e})$ — **модель перевода (Translation Model):** Вероятность того, что $\mathbf{f}$ является переводом $\mathbf{e}$. Оценивает адекватность перевода (насколько хорошо смысл передан).
  - $P(\mathbf{e})$ — **языковая модель (Language Model) целевого языка:** Вероятность того, что $\mathbf{e}$ является "гладким"и грамматически правильным предложением на целевом языке.
- **Подходы внутри SMT:**
  - **Словно-ориентированный (Word-based) SMT:** Модель перевода основана на вероятностях перевода отдельных слов и их выравнивании (alignment).
  - **Фразовый (Phrase-based) SMT (PB-SMT):** Наиболее популярный вариант SMT. Переводит последовательности слов (фразы), а не отдельные слова. Это позволяет лучше справляться с локальными перестановками и идиомами. Включает модели выравнивания фраз, переупорядочивания фраз и лексического взвешивания.
  - **Синтаксически-ориентированный (Syntax-based) SMT:** Использует синтаксические структуры (например, деревья разбора) для улучшения качества перевода, особенно для языковых пар с сильно различающимся порядком слов.
- **Декодирование (Decoding):** Процесс поиска наиболее вероятного перевода $\mathbf{e}^*$. Является сложной задачей поиска, часто решаемой с помощью алгоритмов, подобных лучевому поиску.
- **Ограничения SMT:**
  - **Зависимость от больших параллельных корпусов:** Качество сильно зависит от объема и качества обучающих данных. Для редких языковых пар таких корпусов может не быть.

- **Трудности с дальними зависимостями и сложной структурой:** Фразовые модели ограничены локальным контекстом.
- **Недостаточная "гладкость"и естественность:** Хотя языковая модель помогает, переводы могут быть грамматически неидеальными или стилистически корявыми.
- **Сложность моделей и компонентов:** Требуется настройка множества компонентов (модель выравнивания, модель перевода фраз, модель переупорядочивания, языковая модель, декодер).

в) Нейросетевой машинный перевод (Neural Machine Translation, NMT)

- **Принцип:** Использует одну большую нейронную сеть для прямого моделирования вероятности $P(\mathbf{e}|\mathbf{f})$. Обычно реализуется с помощью архитектуры "кодер-декодер"(Encoder-Decoder).
- **Подходы внутри NMT (основные архитектуры):**
  - **На основе рекуррентных нейронных сетей (RNN-based NMT):** Ранние модели NMT использовали RNN (часто LSTM или GRU) в качестве кодера и декодера. Кодер считывает исходное предложение и сжимает его в вектор фиксированной длины (вектор контекста), а декодер генерирует перевод на основе этого вектора.
  - **С механизмом внимания (Attention Mechanism):** Значительное улучшение RNN-based NMT. Механизм внимания позволяет декодеру "смотреть"на различные части исходного предложения при генерации каждого слова перевода, вместо того чтобы полагаться только на один вектор контекста.
  - **На основе Трансформера (Transformer-based NMT):** Современный стандарт. Использует архитектуру Трансформер, полностью основанную на механизмах внимания, без рекуррентных слоев. Обеспечивает лучшую параллелизацию и способность улавливать дальние зависимости.
- **Ограничения NMT:**
  - **Требовательность к данным и вычислительным ресурсам:** Обучение больших моделей NMT требует огромных корпусов и значительных вычислительных мощностей (GPU/TPU).
  - **Интерпретируемость ("черный ящик"):** Сложно понять, почему модель сделала тот или иной перевод.
  - **Проблемы с редкими словами (Out-of-Vocabulary, OOV):** Хотя техники, такие как Byte Pair Encoding (BPE) или SentencePiece, помогают, модель может испытывать трудности с очень редкими или новыми словами.
  - **Чувствительность к длине предложений:** Качество может падать на очень длинных предложениях.
  - **Иногда генерирует "галлюцинации":** Может производить гладкий, но неверный по смыслу или неадекватный перевод.
  - **Сложность контроля над стилем и спецификой перевода** без дополнительных техник.

г) Гибридный машинный перевод (Hybrid Machine Translation)

- **Принцип:** Комбинирует сильные стороны различных подходов (например, RBMT и SMT, или SMT и NMT).
- **Примеры:**
  - Использование правил для пре- или пост-обработки вывода SMT/NMT.
  - Интеграция статистических моделей фраз в NMT.
- **Ограничения:** Сложность интеграции и настройки различных систем.

2. Статистические и нейросетевые переводчики: принципы построения

Статистические переводчики (на примере Phrase-Based SMT)

Принципы построения PB-SMT:

1. **Сбор данных:** Требуется большой параллельный корпус (например, миллионы пар предложений). Также нужен большой моноязычный корпус для обучения языковой модели целевого языка.

2. **Выравнивание слов (Word Alignment):** С помощью статистических моделей (например, IBM Models 1-5, HMM-based alignment) на параллельном корпусе находятся соответствия между словами в исходных и целевых предложениях.

3. **Извлечение фраз (Phrase Extraction):** На основе выравниваний слов извлекаются пары соответствующих фраз (последовательностей слов).

4. **Оценка вероятностей фраз:** Для каждой пары фраз (f_i, e_j) оцениваются вероятности перевода $P(f_i|e_j)$ и $P(e_j|f_i)$ на основе их частот встречаемости.

5. **Обучение модели переупорядочивания (Reordering Model):** Моделирует, как фразы могут переупорядочиваться при переводе между языками с разным порядком слов.

6. **Обучение языковой модели (Language Model):** Обычно N-граммная модель обучается на большом моноязычном корпусе целевого языка.

7. **Настройка весов (Tuning):** Веса различных компонентов модели (модель перевода фраз, языковая модель, модель переупорядочивания, штраф за длину и т.д.) настраиваются (оптимизируются) на небольшом отладочном (development) параллельном корпусе для максимизации некоторой метрики качества перевода (например, BLEU). Часто используется алгоритм MERT (Minimum Error Rate Training).

8. **Декодирование (Decoding):** Для нового исходного предложения декодер ищет наиболее вероятный перевод, комбинируя информацию от всех моделей. Это сложная задача поиска, обычно решаемая с помощью алгоритмов на основе стека (stack-based decoding) или лучевого поиска.

Нейросетевые переводчики (на примере Transformer-based NMT)

Принципы построения NMT:

1. **Сбор данных:** Требуется большой параллельный корпус. Моноязычные данные также могут использоваться для предобучения или техник, таких как back-translation.

2. **Предобработка данных:**
   - **Токенизация:** Текст разбивается на токены. Часто используются субсловные единицы (subword units), такие как Byte Pair Encoding (BPE) или SentencePiece, чтобы справиться с проблемой OOV и уменьшить размер словаря.
   - **Построение словарей** для исходного и целевого языков (или общего словаря).

3. **Определение архитектуры модели:** Выбирается конкретная архитектура (например, Трансформер) и ее гиперпараметры (количество слоев, голов внимания, размерность эмбеддингов и т.д.).

4. **Обучение модели (End-to-End):**
   - Модель обучается на параллельном корпусе минимизировать функцию потерь (обычно кросс-энтропию) между предсказанным распределением вероятностей слов перевода и реальными словами перевода.
   - Используются методы оптимизации, такие как Adam, и техники регуляризации (например, dropout, label smoothing).

5. **Декодирование (Inference/Translation):** Для нового исходного предложения кодер генерирует его представление. Затем декодер авторегрессионно (слово за словом) генерирует перевод. На каждом шаге декодер предсказывает следующее слово на основе ранее сгенерированных слов и представления исходного предложения.
   - **Лучевой поиск (Beam Search):** Наиболее распространенный метод декодирования. Вместо того чтобы на каждом шаге выбирать только одно наиболее вероятное слово (жадный поиск), лучевой поиск поддерживает k (ширина луча) наиболее вероятных частичных гипотез перевода и расширяет их.

6. **Оценка качества:** Используются автоматические метрики (BLEU, METEOR, TER) и ручная оценка.

25 Извлечение информации из текстов: особенности задачи, виды извлекаемых данных, применяемые подходы. Задача извлечения мнений и анализа тональности текстов.

Извлечение информации (Information Extraction, IE) — это задача автоматического извлечения структурированной информации из неструктурированных или слабоструктурированных текстовых источников. Цель IE — преобразовать неформальные текстовые данные в структурированный формат, пригодный для дальнейшей обработки, анализа или хранения в базах данных.

1. Извлечение информации из текстов: особенности задачи

Особенности задачи IE:

- **Работа с неструктурированными данными:** Исходными данными являются тексты на естественном языке, которые не имеют предопределенной структуры, в отличие от таблиц баз данных.
- **Фокус на конкретной информации:** IE нацелено на извлечение предопределенных типов информации (например, имен, дат, событий, отношений), релевантных для конкретной задачи или домена. Это отличает IE от информационного поиска (Information Retrieval, IR), который находит релевантные *документы*, а не конкретные факты.
- **Многообразие языковых явлений:** Эффективное извлечение требует учета лексической вариативности, синтаксических конструкций, семантических неоднозначностей и прагматического контекста.
- **Доменная зависимость:** Многие IE-системы разрабатываются для конкретных предметных областей (например, финансы, медицина, новости), так как типы извлекаемой информации и используемая терминология специфичны для домена.
- **Оценка качества:** Качество IE-систем оценивается по метрикам точности (precision), полноты (recall) и F-меры (F1-score).

2. Виды извлекаемых данных

IE-системы могут извлекать различные типы структурированной информации:

1. **Именованные сущности (Named Entities):** Фрагменты текста, обозначающие конкретные объекты реального мира. Наиболее распространенные типы:
   - **PER (Person):** Имена людей.
   - **ORG (Organization):** Названия организаций.
   - **LOC (Location):** Географические названия.

   Задача их извлечения называется **Распознавание именованных сущностей (Named Entity Recognition, NER)**.

2. **Отношения (Relations):** Семантические связи между двумя или более именованными сущностями. Примеры:
   - works_for(Человек, Организация): "Сергей Брин работает в Google."→ works_for(Сергей Брин, Google).
   - located_in(Организация, Местоположение): "Штаб-квартира Apple находится в Купертино."→ located_in(Apple, Купертино).
   - married_to(Человек1, Человек2).

   Задача их извлечения называется **Извлечение отношений (Relation Extraction, RE)**.

3. **События (Events):** Происшествия или ситуации, в которых участвуют сущности. Событие обычно характеризуется типом, триггерным словом (слово, указывающее на событие) и аргументами (участниками события и их ролями). Пример: Текст "Компания X объявила о назначении Y на пост CEO."
   - Тип события: Назначение_на_должность.
   - Триггер: "назначении".
   - Аргументы: Компания=X, Сотрудник=Y, Должность=CEO.

   Задача их извлечения называется **Извлечение событий (Event Extraction)**.

4. **Мнения и тональность (Opinions and Sentiments):** Извлечение субъективной информации: кто выражает мнение, о каком объекте/аспекте, какова полярность мнения (позитивное, негативное, нейтральное) и его интенсивность. (Рассматривается подробнее ниже).

5. **Заполнение шаблонов (Template Filling):** Извлечение информации для заполнения предопределенных структур (шаблонов), описывающих стандартные ситуации или объекты (например, шаблон для описания теракта, шаблон для описания вакансии).

3. Применяемые подходы к извлечению информации

1. **Подход на основе правил (Rule-Based IE):**
   - **Принцип:** Используются наборы правил, созданные вручную лингвистами или экспертами. Правила могут быть основаны на:
     - **Регулярных выражениях:** Для поиска простых шаблонов (например, форматов дат, телефонных номеров).
     - **Лексико-синтаксических шаблонах (Gazetteers and Patterns):** Списки известных сущностей (газеттиры) и шаблоны, описывающие контексты, в которых обычно появляются искомые сущности или отношения. Часто используются совместно с результатами POS-теггинга и синтаксического анализа.
     - **Контекстно-свободных грамматиках (CFG):** Для описания структуры извлекаемой информации.
   - **Преимущества:** Высокая точность для хорошо определенных задач, интерпретируемость.
   - **Недостатки:** Трудоемкость создания и поддержки, низкая адаптивность, хрупкость к вариациям языка.

2. **Подход на основе машинного обучения (Machine Learning-Based IE):**
   - **Принцип:** Модели обучаются на размеченных текстовых корпусах, где целевая информация (например, именованные сущности, отношения) уже аннотирована.
   - **NER как задача разметки последовательности (Sequence Labeling):** Каждому токену в предложении присваивается метка (например, B-PER, I-PER, O — начало сущности "Человек продолжение сущности "Человек не сущность). Используемые модели:
     - Скрытые Марковские Модели (HMM).
     - Рекуррентные Нейронные Сети (RNN, LSTM, BiLSTM) часто в комбинации с CRF-слоем на выходе (BiLSTM-CRF).
     - Трансформеры (например, BERT) — современные state-of-the-art модели.
   - **RE как задача классификации:**
     - **Классификация пар сущностей:** Для каждой пары сущностей в предложении модель предсказывает тип отношения между ними (или отсутствие отношения). Признаками могут быть сами сущности, их контекст, расстояние между ними, синтаксический путь.
     - **Ядерные методы (Kernel Methods):** Используют функции ядра для сравнения структур (например, синтаксических деревьев), содержащих пары сущностей.
     - **Нейросетевые подходы:** CNN, RNN, Трансформеры для кодирования контекста и предсказания отношений.
   - **Преимущества:** Лучшая адаптивность и обобщающая способность, способность улавливать сложные закономерности, меньшие затраты на ручную разработку правил (но требуются размеченные данные).
   - **Недостатки:** Требуются большие размеченные корпусы, модели могут быть "черным ящиком вычислительная сложность обучения.

3. **Гибридные подходы:** Комбинируют правила и машинное обучение (например, использование газеттиров как признаков для ML-моделей, или использование ML для генерации кандидатов, которые затем фильтруются правилами).

4. **Слабо контролируемое обучение (Weakly Supervised / Distant Supervision):** Для RE: если две сущности связаны известным отношением в базе знаний (например, Freebase), то все предложения в большом корпусе, где эти две сущности встречаются вместе, рассматриваются как положительные примеры для этого отношения. Это позволяет автоматически генерировать большие объемы "шумных"обучающих данных.

5. **Открытое извлечение информации (Open Information Extraction, OpenIE):** Цель — извлечение отношений из текста без предопределенного списка типов отношений. Системы OpenIE обычно извлекают тройки (аргумент1, отношение-фраза, аргумент2).

4. Задача извлечения мнений и анализа тональности текстов (Opinion Mining / Sentiment Analysis)

Определение: Анализ тональности — это область NLP, которая занимается выявлением, извлечением и количественной оценкой субъективной информации, мнений, эмоций и оценок, выраженных в тексте.

Уровни анализа тональности:

- **Уровень документа (Document-Level):** Определение общей тональности всего документа (например, отзыва о продукте) как позитивной, негативной или нейтральной.
- **Уровень предложения (Sentence-Level):** Определение тональности каждого отдельного предложения. Предложение может быть объективным (без тональности) или субъективным (выражающим мнение).
- **Аспектный анализ тональности (Aspect-Based Sentiment Analysis, ABSA):** Наиболее детальный уровень. Цель — выявить:
  - **Аспекты (Aspects/Features):** Конкретные атрибуты или компоненты объекта, о которых высказывается мнение (например, для телефона: "камера "батарея "дизайн").
  - **Мнения/Оценки (Opinions/Sentiments):** Тональность высказывания относительно каждого аспекта.
  - **Держателя мнения (Opinion Holder):** Кто выражает мнение.
  - **Объект мнения (Opinion Target):** О чем высказывается мнение.

  Пример: "Камера у этого телефона отличная, но батарея держит слабо."
  - Аспект "камера тональность "позитивная".
  - Аспект "батарея тональность "негативная".

Задачи в рамках анализа тональности:

- **Классификация тональности:** Присвоение тексту или его части метки тональности (позитивная, негативная, нейтральная; иногда используется шкала, например, от 1 до 5 звезд).
- **Извлечение аспектов (Aspect Extraction):** Идентификация аспектов, обсуждаемых в тексте.
- **Идентификация слов-мнений (Opinion Word Identification):** Нахождение слов и фраз, выражающих тональность (например, "хороший "ужасный "не понравился").
- **Обнаружение сарказма и иронии:** Сложная задача, так как буквальный смысл противоречит подразумеваемому.

Применяемые подходы к анализу тональности:

1. **Подход на основе словарей тональности (Lexicon-Based Approach):**
   - Используются словари (лексиконы) слов и фраз с заранее присвоенной им тональностью.
   - Тональность текста определяется путем подсчета количества позитивных и негативных слов, с учетом их интенсивности, отрицаний, усилителей/ослабителей.
   - **Преимущества:** Не требует обучающих данных, прост в реализации.
   - **Недостатки:** Не учитывает контекст, проблемы с многозначностью, сарказмом, доменно-специфичной лексикой.

2. **Подход на основе машинного обучения (Machine Learning-Based Approach):** Рассматривается как задача классификации текста.
   - **Признаки:** "Мешок слов"(Bag-of-Words), N-граммы слов, POS-теги, наличие слов из словарей тональности, векторные представления слов (эмбеддинги).
   - **Алгоритмы:** Naive Bayes, Logistic Regression, SVM, нейронные сети (CNN, RNN, LSTM, Трансформеры).
   - **Преимущества:** Лучше учитывает контекст, выше точность при наличии достаточного количества размеченных данных.
   - **Недостатки:** Требуются размеченные данные для обучения.

26 Процессная модель управления проектом: основные процессы и их взаимодействие. Особенности управления программным проектом.

Управление проектами — это применение знаний, навыков, инструментов и методов к проектной деятельности для удовлетворения требований проекта.

1. Процессная модель управления проектом

Определение: Процессная модель управления проектом рассматривает управление проектом как совокупность взаимосвязанных процессов, которые выполняются на протяжении всего жизненного цикла проекта для достижения его целей. Каждый процесс имеет свои входы, инструменты и методы, а также выходы.

Стандарты управления проектами, такие как PMBOK (Project Management Body of Knowledge) от PMI (Project Management Institute), выделяют группы процессов управления проектом. Традиционно их пять:

1. **Группа процессов инициации:**
   - **Цель:** Определение и авторизация проекта или его фазы. Формальное признание того, что новый проект существует или что существующий проект должен перейти в следующую фазу.
   - **Основные процессы:**
     - *Разработка Устава проекта (Develop Project Charter):* Документ, который формально авторизует проект и наделяет менеджера проекта полномочиями. Содержит высокоуровневую информацию о проекте (цели, требования, риски, ключевые заинтересованные стороны, бюджет, сроки).
     - *Определение заинтересованных сторон (Identify Stakeholders):* Выявление всех лиц, групп или организаций, которые могут влиять на проект или на которых может повлиять проект, а также анализ их интересов, вовлеченности и влияния.
   - **Взаимодействие:** Получает на вход бизнес-обоснование, соглашения. Выходом является Устав проекта и реестр заинтересованных сторон, которые служат основой для дальнейшего планирования.

2. **Группа процессов планирования (Planning Process Group):**
   - **Цель:** Определение и уточнение целей, а также планирование курса действий, необходимых для достижения этих целей и ожидаемых результатов проекта. Это итеративный процесс.
   - **Основные процессы (некоторые из многих):**
     - *Разработка плана управления проектом (Develop Project Management Plan):* Создание всеобъемлющего документа, который определяет, как проект будет исполняться, контролироваться и закрываться.
     - *Сбор требований (Collect Requirements):* Определение, документирование и управление потребностями и ожиданиями заинтересованных сторон.
     - *Определение содержания (Define Scope):* Детальное описание границ проекта, его результатов и критериев приемки.
     - *Создание иерархической структуры работ (Create WBS - Work Breakdown Structure):* Декомпозиция результатов проекта и работ по проекту на более мелкие, управляемые компоненты.
     - *Определение последовательности операций (Sequence Activities):* Идентификация и документирование зависимостей между операциями проекта.
     - *Оценка длительности операций (Estimate Activity Durations):* Оценка количества рабочих периодов, необходимых для завершения отдельных операций.
     - *Разработка расписания (Develop Schedule):* Анализ последовательностей операций, их длительностей, требований к ресурсам и ограничений расписания для создания модели расписания проекта.
     - *Оценка стоимости (Estimate Costs):* Разработка приближенной оценки денежных ресурсов, необходимых для выполнения операций проекта.
     - *Определение бюджета (Determine Budget):* Агрегирование оценочных стоимостей отдельных операций или пакетов работ для установления авторизованного базового плана по стоимости.
     - *Планирование управления качеством (Plan Quality Management).*
     - *Планирование управления ресурсами (Plan Resource Management).*
     - *Планирование управления коммуникациями (Plan Communications Management).*

- *Планирование управления рисками (Plan Risk Management):* Определение того, как будут осуществляться действия по управлению рисками проекта (включая идентификацию, анализ, планирование реагирования).
- *Планирование управления закупками (Plan Procurement Management).*
- *Планирование вовлечения заинтересованных сторон (Plan Stakeholder Engagement).*

- **Взаимодействие:** Основывается на Уставе проекта и реестре заинтересованных сторон. Результатом является План управления проектом и множество вспомогательных планов, которые направляют исполнение и контроль.

3. **Группа процессов исполнения (Executing Process Group):**
   - **Цель:** Выполнение работ, определенных в плане управления проектом, для удовлетворения требований проекта. Потребляет большую часть бюджета и времени проекта.
   - **Основные процессы:**
     - *Руководство и управление работами проекта (Direct and Manage Project Work):* Исполнение запланированных работ по проекту для достижения целей проекта.
     - *Управление знаниями проекта (Manage Project Knowledge):* Использование существующих знаний и создание новых знаний для достижения целей проекта и содействия обучению организации.
     - *Управление качеством (Manage Quality):* Преобразование плана управления качеством в выполнимые действия по обеспечению качества, которые включают политики и процедуры организации в области качества в стандарты качества проекта.
     - *Приобретение ресурсов (Acquire Resources):* Получение членов команды, оборудования, материалов и других ресурсов, необходимых для выполнения работ по проекту.
     - *Развитие команды проекта (Develop Team):* Улучшение компетенций, взаимодействия членов команды и общей среды команды для повышения эффективности проекта.
     - *Управление командой проекта (Manage Team):* Отслеживание выполнения членами команды, предоставление обратной связи, решение проблем и управление изменениями для оптимизации исполнения проекта.
     - *Управление коммуникациями (Manage Communications):* Обеспечение своевременного и надлежащего сбора, создания, распространения, хранения, извлечения, управления, мониторинга и окончательного размещения информации по проекту.
     - *Осуществление реагирования на риски (Implement Risk Responses).*
     - *Проведение закупок (Conduct Procurements).*
     - *Управление вовлечением заинтересованных сторон (Manage Stakeholder Engagement).*
   - **Взаимодействие:** Руководствуется планом управления проектом. Создает результаты проекта (deliverab и информацию об исполнении, которая используется в процессах мониторинга и контроля.

4. **Группа процессов мониторинга и контроля (Monitoring and Controlling Process Group):**
   - **Цель:** Отслеживание, анализ и регулирование прогресса и исполнения проекта; выявление областей, в которых требуются изменения в плане; и инициирование соответствующих изменений.
   - **Основные процессы:**
     - *Мониторинг и контроль работ проекта (Monitor and Control Project Work):* Отслеживание, анализ и предоставление отчетности об общем прогрессе для достижения целей исполнения, определенных в плане управления проектом.
     - *Осуществление интегрированного контроля изменений (Perform Integrated Change Control):* Анализ всех запросов на изменения; одобрение изменений и управление изменениями в результатах, организационных активах процессов, документах проекта и плане управления проектом; и сообщение о решениях.
     - *Подтверждение содержания (Validate Scope):* Формализация приемки завершенных результатов проекта.
     - *Контроль содержания (Control Scope):* Мониторинг состояния проекта и содержания продукта и управление изменениями базового плана по содержанию.
     - *Контроль расписания (Control Schedule).*
     - *Контроль стоимости (Control Costs).*
     - *Контроль качества (Control Quality).*
     - *Контроль ресурсов (Control Resources).*

  - *Мониторинг коммуникаций (Monitor Communications).*
  - *Мониторинг рисков (Monitor Risks).*
  - *Контроль закупок (Control Procurements).*
  - *Мониторинг вовлечения заинтересованных сторон (Monitor Stakeholder Engagement).*
- **Взаимодействие:** Происходит параллельно с процессами исполнения. Сравнивает фактическое исполнение с планом, выявляет отклонения и инициирует корректирующие или предупреждающие действия, а также запросы на изменения, которые могут повлиять на процессы планирования.

5. **Группа процессов закрытия (Closing Process Group):**
   - **Цель:** Формальное завершение всех активностей по всем группам процессов управления проектом для формального завершения проекта, фазы или контрактных обязательств.
   - **Основные процессы:**
     - *Закрытие проекта или фазы (Close Project or Phase):* Завершение всех операций для проекта или фазы, архивирование информации, извлечение уроков, празднование успеха.
   - **Взаимодействие:** Получает на вход одобренные результаты и информацию об исполнении. Выходом являются финальный продукт, услуга или результат, а также обновление организационных активов процессов (например, архив проекта, извлеченные уроки).

Эти группы процессов не являются дискретными фазами проекта, а скорее наборами взаимосвязанных активностей, которые могут перекрываться и повторяться на протяжении проекта.

2. Особенности управления программным проектом

Управление проектами по разработке программного обеспечения (ПО) имеет ряд специфических особенностей, отличающих его от управления проектами в других отраслях (например, строительство, производство):

1. **Высокая степень неопределенности и изменчивости требований:** Требования к ПО часто меняются в ходе проекта из-за изменения бизнес-потребностей, технологических новшеств или лучшего понимания задачи заказчиком. Это требует гибких подходов к управлению.

2. **Сложность оценки трудоемкости и сроков:** Оценка разработки ПО — сложная задача из-за уникальности многих проектов, зависимости от квалификации команды, неопределенности требований и технологических рисков.

3. **Зависимость от человеческого фактора и командной работы:** Квалификация, мотивация и слаженность команды разработчиков играют ключевую роль в успехе программного проекта. Управление творческими и интеллектуальными ресурсами требует особого подхода.

4. **Необходимость интенсивного тестирования и управления качеством:** Ошибки в ПО могут иметь серьезные последствия. Процессы тестирования (модульное, интеграционное, системное, приемочное) и обеспечения качества являются неотъемлемой частью жизненного цикла разработки.

5. **Повторное использование компонентов (Reusability):** Возможность использования существующих библиотек, фреймворков, компонентов может значительно сократить время и стоимость разработки, но также требует управления зависимостями.

6. **Итеративность и инкрементальность разработки:** Из-за высокой неопределенности многие программные проекты выигрывают от итеративных и инкрементальных подходов (например, Agile-методологии, такие как Scrum, Kanban), где продукт разрабатывается небольшими, работоспособными частями, с регулярной обратной связью от заказчика. Это позволяет адаптироваться к изменениям и снижать риски.

7. **Управление конфигурациями и версиями (Configuration Management):** Важно отслеживать изменения в коде, документации и других артефактах проекта, управлять различными версиями продукта.

8. **Коммуникационные сложности:** В программных проектах часто задействованы распределенные команды, различные заинтересованные стороны (заказчики, пользователи, разработчики, тестировщики, менеджеры), что требует эффективных коммуникаций.

Программный продукт — это совокупность программных средств, процедур, правил, а также связанной с ними документации и данных, предназначенных для поставки пользователям и способных удовлетворить определенные потребности или решить конкретные задачи.

Процесс разработки программного обеспечения — это структурированная последовательность действий (этапов, фаз, активностей), предпринимаемых для проектирования, создания, тестирования и развертывания программного продукта. Процесс разработки ПО включает в себя:

- Определение требований
- Проектирование архитектуры и дизайна
- Кодирование
- Тестирование
- Развертывание
- Сопровождение и обновление

Существуют различные модели процесса разработки ПО (например, каскадная, итерационная, спиральная, гибкие методологии), каждая из которых предлагает свою организацию этих действий.

27 Сетевое планирование проекта. Примеры создания сетевых графиков и назначения ресурсов.

Сетевое планирование — это метод управления проектами, который используется для планирования, составления расписания, координации и контроля выполнения комплекса взаимосвязанных работ проекта. Он основан на представлении проекта в виде сети, состоящей из работ и событий, отражающих технологические и организационные взаимосвязи между ними. Основными инструментами сетевого планирования являются сетевые графики.

1. Основные понятия сетевого планирования

- **Работа (Activity/Task):** Любой процесс или элемент проекта, требующий затрат времени и ресурсов для своего выполнения (например, проектирование, строительство, тестирование). В сетевом графике работа представляется либо дугой (стрелкой), либо узлом (прямоугольником/кругом).
- **Событие (Event):** Момент времени, означающий начало или завершение одной или нескольких работ. Событие не имеет длительности и не требует ресурсов. Обычно представляется узлом (кружком) в методе "работы-дуги". В методе "работы-узлы"события являются неявными (начало/конец работы-узла).
- **Сетевой график (Network Diagram):** Графическое представление работ проекта и их логических зависимостей, показывающее последовательность их выполнения.
- **Зависимость (Dependency/Precedence Relationship):** Логическая связь между работами, определяющая порядок их выполнения. Основные типы зависимостей (особенно актуальны для метода "работа-узел"):
  - **Финиш-Старт (Finish-to-Start, FS):** Последующая работа может начаться только после завершения предшествующей работы. (Самый распространенный тип).
  - **Старт-Старт (Start-to-Start, SS):** Последующая работа может начаться только после начала предшествующей работы.
  - **Финиш-Финиш (Finish-to-Finish, FF):** Последующая работа может завершиться только после завершения предшествующей работы.
  - **Старт-Финиш (Start-to-Finish, SF):** Последующая работа может завершиться только после начала предшествующей работы. (Используется редко).
- **Длительность работы (Duration):** Оценочное время, необходимое для выполнения работы от ее начала до завершения.
- **Фиктивная работа (Dummy Activity):** Работа нулевой длительности, используемая в некоторых типах сетевых графиков (например, "работы-дуги") для корректного отображения логических зависимостей между работами, не требующая затрат времени и ресурсов. Изображается пунктирной стрелкой.
- **Путь (Path):** Любая непрерывная последовательность работ в сетевом графике от начального до конечного события.
- **Критический путь (Critical Path):** Наиболее длительный по времени путь в сетевом графике от начального до конечного события. Работы, лежащие на критическом пути, называются критическими. Они не имеют резерва времени, и любая задержка в их выполнении приведет к задержке завершения всего проекта.
- **Ресурсы (Resources):** Люди (исполнители), оборудование, материалы, финансы и другие средства, необходимые для выполнения работ проекта.

2. Методы построения сетевых графиков

Существует два основных метода представления работ на сетевом графике:

- **Метод "работы-дуги"(Activity-on-Arrow, AOA) или ADM (Arrow Diagramming Method):** Работы изображаются стрелками (дугами), а узлы (кружки) представляют события – моменты начала или окончания работ. Этот метод может требовать использования фиктивных работ для правильного отображения зависимостей.
- **Метод "работы-узлы"(Activity-on-Node, AON) или PDM (Precedence Diagramming Method):** Работы изображаются узлами (обычно прямоугольниками), а стрелки между узлами показывают ло-

гические зависимости между работами. Этот метод более гибок и нагляден, обычно не требует фиктивных работ.

Наиболее известными методами анализа сетевых графиков являются:

- **Метод критического пути (Critical Path Method, CPM):** Используется для определения общей продолжительности проекта и идентификации критических работ. CPM предполагает детерминированные (точно известные) оценки длительности работ.
- **Метод оценки и анализа программ (Program Evaluation and Review Technique, PERT):** Используется для проектов с высокой степенью неопределенности в оценках длительности работ. PERT обычно использует три оценки длительности (оптимистическую, наиболее вероятную, пессимистическую) для расчета ожидаемой длительности и вероятности завершения проекта в срок.

Расчет параметров сетевого графика (Метод критического пути - CPM)

Для каждой работы рассчитываются следующие временные параметры:

- **Ранний срок начала (Early Start, ES):** Самый ранний возможный момент времени, когда работа может быть начата, при условии, что все предшествующие работы завершены в свои ранние сроки. Для первой работы $ES = 0$.
  Для остальных $ES = \max(\text{EF всех непосредственных предшественников})$.
- **Ранний срок окончания (Early Finish, EF):** Самый ранний возможный момент времени, когда работа может быть завершена. $EF = ES +$ Длительность (Dur).
- **Поздний срок окончания (Late Finish, LF):** Самый поздний возможный момент времени, когда работа должна быть завершена, чтобы не вызвать задержку всего проекта. Для последней работы проекта $LF = EF$ (или директивный срок окончания проекта). Для остальных $LF = \min(\text{LS всех непосредственных последователей})$.
- **Поздний срок начала (Late Start, LS):** Самый поздний возможный момент времени, когда работа должна быть начата, чтобы не вызвать задержку всего проекта. $LS = LF -$ Длительность (Dur).
- **Общий (полный) резерв времени (SL):** Максимальное время, на которое можно задержать выполнение данной работы (начало или увеличение длительности) без задержки общего срока завершения проекта. $TF = LS - ES = LF - EF$. Работы на критическом пути имеют $TF = 0$.

Расчет производится в два этапа:

(a) **Прямой проход (Forward Pass):** От начального события к конечному, рассчитываются ES и EF для всех работ. Общая продолжительность проекта – это максимальный EF среди всех работ, завершающих проект.

(b) **Обратный проход (Backward Pass):** От конечного события к начальному, рассчитываются LF и LS для всех работ. LF последней работы принимается равным ее EF (или директивному сроку).

3. Примеры создания сетевых графиков

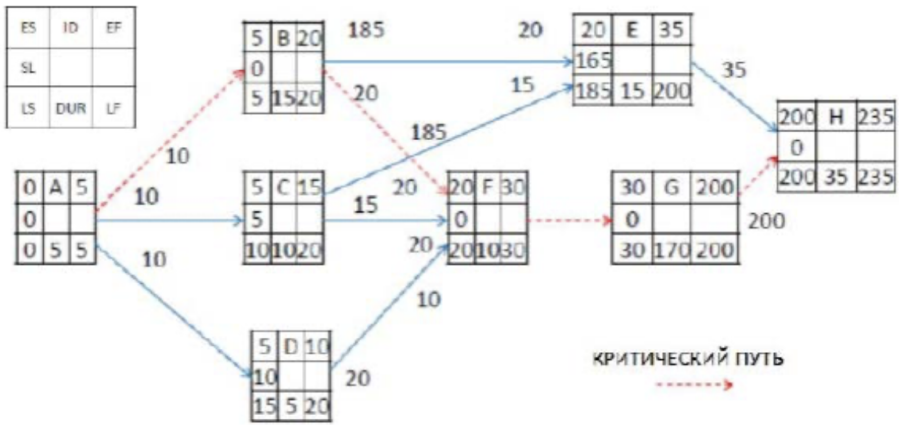

Рис. 2: Изображение 5

Пример 1: Метод "работы-узлы"(PDM/AON) — см. Изображение 5

На **Изображении 5** представлен сетевой график, выполненный методом "работы-узлы"(Activity-on-Node). Каждая работа (A, B, C, D, E, F, G, H) представлена прямоугольным узлом.

ES	ID	EF
SL (Резерв времени)		
LS	DUR	LF

Где: ES - Раннее Начало, ID - Идентификатор работы, EF - Раннее Окончание, SL - Резерв времени (Slack), LS - Позднее Начало, DUR - Длительность (Duration), LF - Позднее Окончание.

Продолжительность проекта по этому пути = 235. Работы C, D, E имеют резервы времени (SL > 0).

Критический путь на Изображении 5: A → B → F → G → H. Работы на этом пути имеют нулевой резерв времени (SL=0). Любая задержка на одной из этих работ приведет к задержке всего проекта.

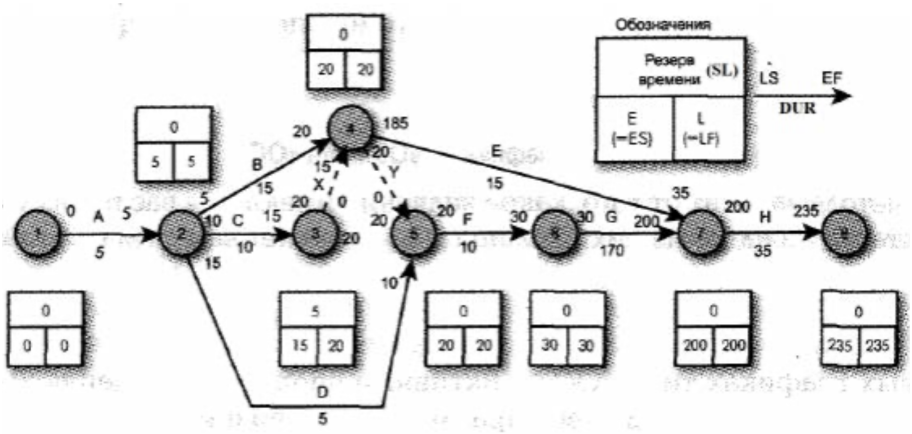

Рис. 3: Изображение 6

На **Изображении 6** представлен сетевой график, выполненный методом "работы-дуги"(Activity-on-Arrow).

- **Работы (A, B, C, D, E, F, G, H, X, Y)** изображены стрелками.
- **События** изображены кружками и пронумерованы (1, 2, 3, 4, 5, 6, 7, 8).
- **Фиктивные работы (X, Y):** Изображены пунктирными стрелками, имеют нулевую длительность.

Критический путь на Изображении 6: A, B, Y, F, G, H.

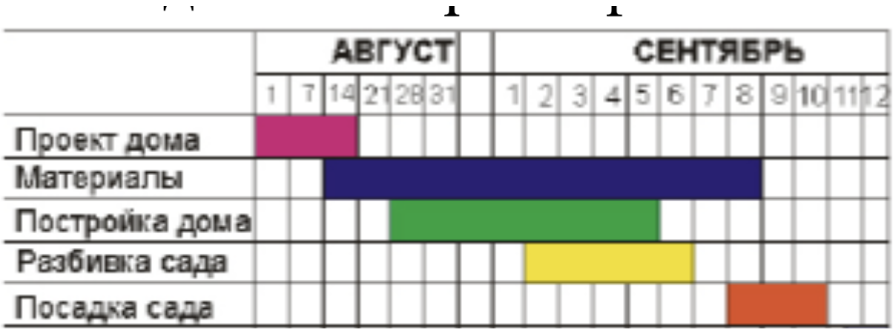

Рис. 4: Изображение 7

На **Изображении 7** показан пример диаграммы Ганта. Это столбчатая диаграмма, иллюстрирующая график проекта.

По вертикальной оси — задачи, по горизонтальной оси — календарные даты, каждая задача — горизонтальная полоса, длина которой равна длительности задачи.

Диаграмма Ганта наглядно показывает длительности, время начала/окончания, параллельные и последовательные задачи.

Почему AOA (работы-дуги) НУЖДАЕТСЯ в фиктивных работах?

В AOA каждая работа – это стрелка. Стрелки соединяют события (кружки). Событие – это момент времени, когда одна или несколько работ завершаются, и одна или несколько других могут начаться.

Вот тут и возникают проблемы, которые решают фиктивные работы:

Проблема 1: Неоднозначность зависимостей

Пример:

Работа C может начаться только после завершения работы A. Работа D может начаться только после завершения работ A И B.

Проблема 2: Уникальная идентификация работ

В классическом AOA каждая работа уникально идентифицируется парой событий: "событие начала – событие конца". Не должно быть двух разных РАБОТ, которые начинаются в одном событии и заканчиваются в одном и том же другом событии.

Пример:

Работа X: Покрасить стену 1 (от события "стены готовы"до события "покраска завершена"). Работа Y: Покрасить стену 2 (тоже от "стены готовы"до "покраска завершена").

Преимущества AOA:

- **Наглядность событий:** Явно показывает события начала и окончания работ, что может быть полезно для отслеживания ключевых событий.
- **Простота для некоторых типов зависимостей:** Основная зависимость "конец-начало"(FS) естественным образом отображается.

Недостатки AOA:

- **Необходимость фиктивных работ (Dummy Activities):** Для корректного отображения сложных зависимостей (например, когда одна работа зависит от двух других, но эти две другие не зависят друг от друга напрямую и не завершаются в одном и том же событии) часто требуются фиктивные работы. Фиктивные работы имеют нулевую длительность и не потребляют ресурсов, но усложняют график и его понимание.
- **Сложность модификации:** Внесение изменений в сетевой график (добавление, удаление работ, изменение зависимостей) может быть более трудоемким.
- **Меньшая интуитивность для сложных проектов:** Графики могут становиться запутанными и трудными для чтения из-за большого количества событий и фиктивных работ.
- **Работы не имеют уникального идентификатора узла:** Идентификация работы происходит по паре событий (начальное и конечное), что может приводить к путанице, если несколько работ соединяют одни и те же два события (требует фиктивных работ для различения).

Преимущества AON/PDM:

- **Наглядность и интуитивность:** Графики обычно легче читать и понимать, так как каждая работа представлена отдельным узлом, содержащим всю информацию о ней (название, длительность, ранние/поздние сроки и т.д.).
- **Отсутствие необходимости в фиктивных работах:** Логические зависимости отображаются напрямую стрелками между узлами-работами.
- **Простота модификации:** Изменения в графике (добавление, удаление, изменение зависимостей) вносить проще.
- **Широкое использование в современном ПО:** Большинство современных программных пакетов для управления проектами используют метод PDM.
- **Каждая работа имеет уникальный узел:** Упрощает идентификацию и анализ.

Недостатки AON/PDM:

- **Менее явное отображение событий:** Ключевые события (например, завершение этапов) не представлены отдельными узлами, как в AOA, хотя их можно обозначать специальными задачами с нулевой длительностью.

4. Назначение ресурсов

После построения графика и определения критического пути назначаются ресурсы.

Процесс назначения ресурсов:

(a) **Определение требуемых ресурсов:** Для каждой работы определяется тип и количество ресурсов.

(b) **Оценка доступности ресурсов.**

(c) **Назначение ресурсов на работы.**

(d) **Анализ загрузки ресурсов (Resource Loading):** Построение гистограмм загрузки для выявления перегрузок/недогрузок.

(e) **Выравнивание ресурсов (Resource Leveling):** Сглаживание пиков и провалов в использовании ресурсов. Методы:

- Сдвиг некритических работ в пределах их резерва.
- Увеличение длительности проекта (если затрагиваются критические работы).
- Разделение работ (Splitting tasks).
- Привлечение дополнительных ресурсов.

(f) **Сглаживание ресурсов (Resource Smoothing):** Оптимизация использования ресурсов в пределах установленных сроков проекта, используя только резервы некритических работ.

Назначение ресурсов и их выравнивание — итеративный процесс для составления реалистичного плана-графика.

Сетевые графики (AON/PDM и AOA/ADM): Инструменты для анализа и расчета временных параметров проекта, определения критического пути. Они фокусируются на логике и зависимостях.

Диаграмма Ганта: Инструмент для визуализации расписания проекта во времени. Она фокусируется на представлении длительностей и сроков работ на временной шкале.